

软件工程基础

第4章：软件概要设计

杨溢龙

北航软件学院

目 录

Contents

软件工程基础

第5章：需求工程

Part 1 软件设计概述

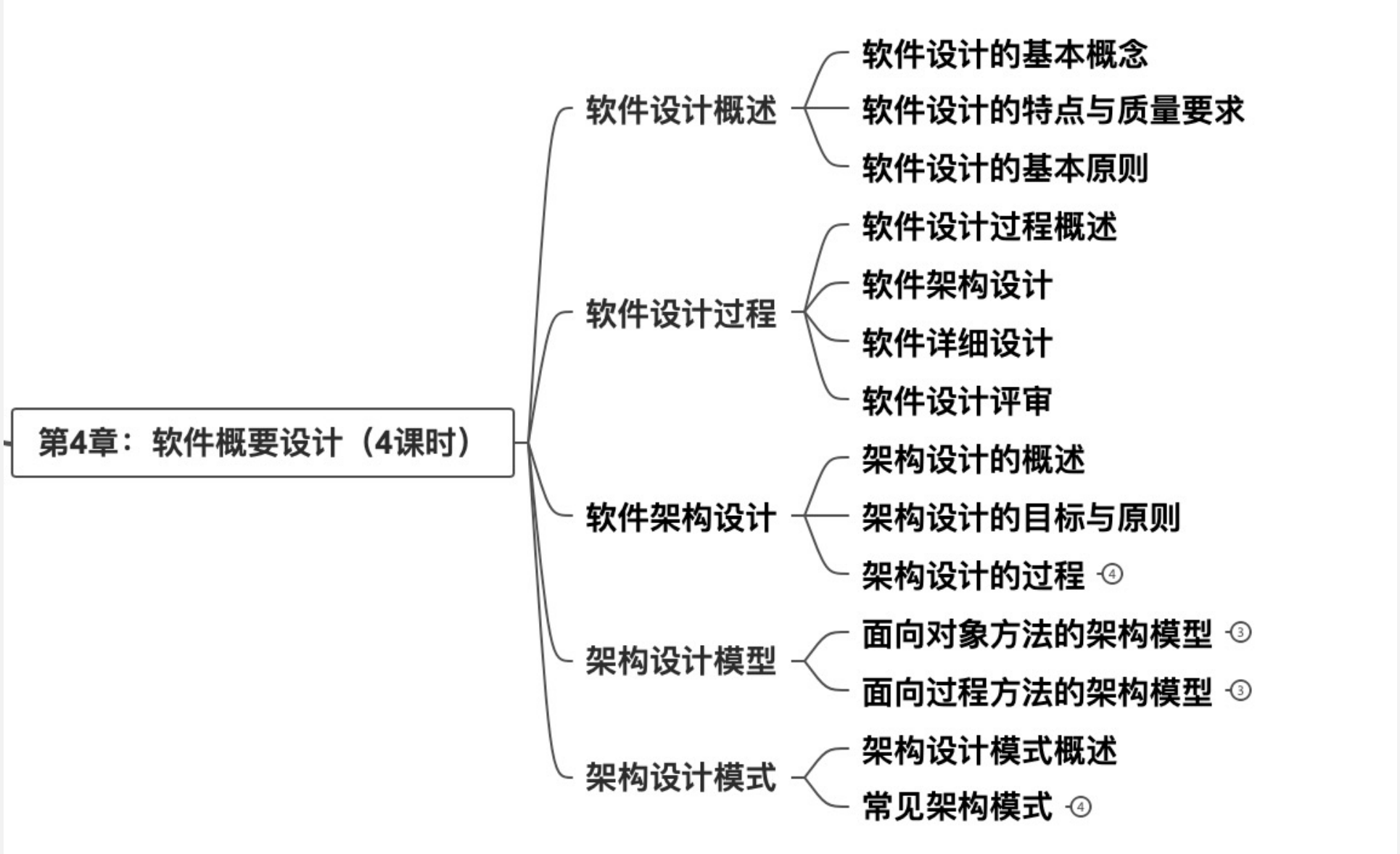
Part 2 软件设计过程

Part 3 软件架构设计（概要设计）

Part 4 架构设计模型（概要设计）

Part 5 架构设计模式

课时安排：软件概要设计


第4章：软件概要设计（4课时）
软件设计概述
软件设计的基本概念
软件设计的特点与质量要求
软件设计的基本原则
软件设计过程
软件设计过程概述
软件架构设计
软件详细设计
软件设计评审
软件架构设计
架构设计的概述
架构设计的目标与原则
架构设计的过程 ④
架构设计模型
面向对象方法的架构模型 ③
面向过程方法的架构模型 ③
架构设计模式
架构设计模式概述
常见架构模式 ④

北京航空航天大學
BEIHANG UNIVERSITY



Part 1

软件设计概述

主观题 10分

设置

思考和讨论：

明确需求后，直接来编写代码行吗？为什么？

如果不行，软件设计主要是设计什么？



Answer

从需求到设计和编码

- **需求 → 设计**
  - ✓回答**如何做（How）→ 设计图纸**
  - ✓根据需求来进行设计，确保设计的**质量**
- **设计 → 实现**
  - ✓基于设计来指导**施工和实现**
  - ✓设计的好坏直接决定了最终产品的好坏！



Part 1 软件设计概述

- 软件设计的基本概念
- 软件设计的特点与质量要求
- 软件设计的基本原则

Part 1 软件设计概述

- 软件设计的基本概念
- 软件设计的特点与质量要求
- 软件设计的基本原则

何为软件设计?

- **软件设计**
  - ✓针对**软件需求**，综合考虑各种**制约因素**，探究软件实现的**解决方案**
- **设计前提：软件需求**
  - ✓定义了要做什么样的软件
- **设计考虑：制约因素**
  - ✓**资源**：时间、人力、财力、开发辅助工具
  - ✓**技术**：技术平台，如DBMS还是文件系统

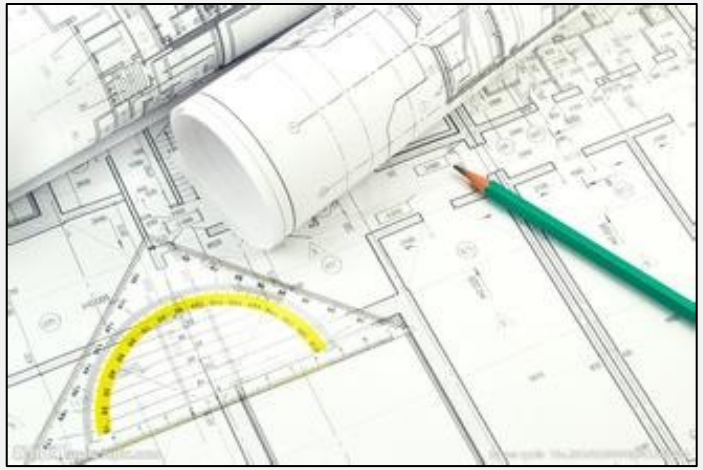


软件设计是要给出软件需求具体实现的解决方案

软件设计的内容

- 描述了如何来构造和实现软件系统
  - ✓模块及其功能和接口
  - ✓模块的组织
  - ✓模块之间的交互
  - ✓模块内部的算法
  - ✓人机交互的界面和方式
  - ✓数据结构设计
  - ✓数据库的设计和组织
  - ✓……

- 不同设计内容
- 不同设计层次
- 不同设计视角



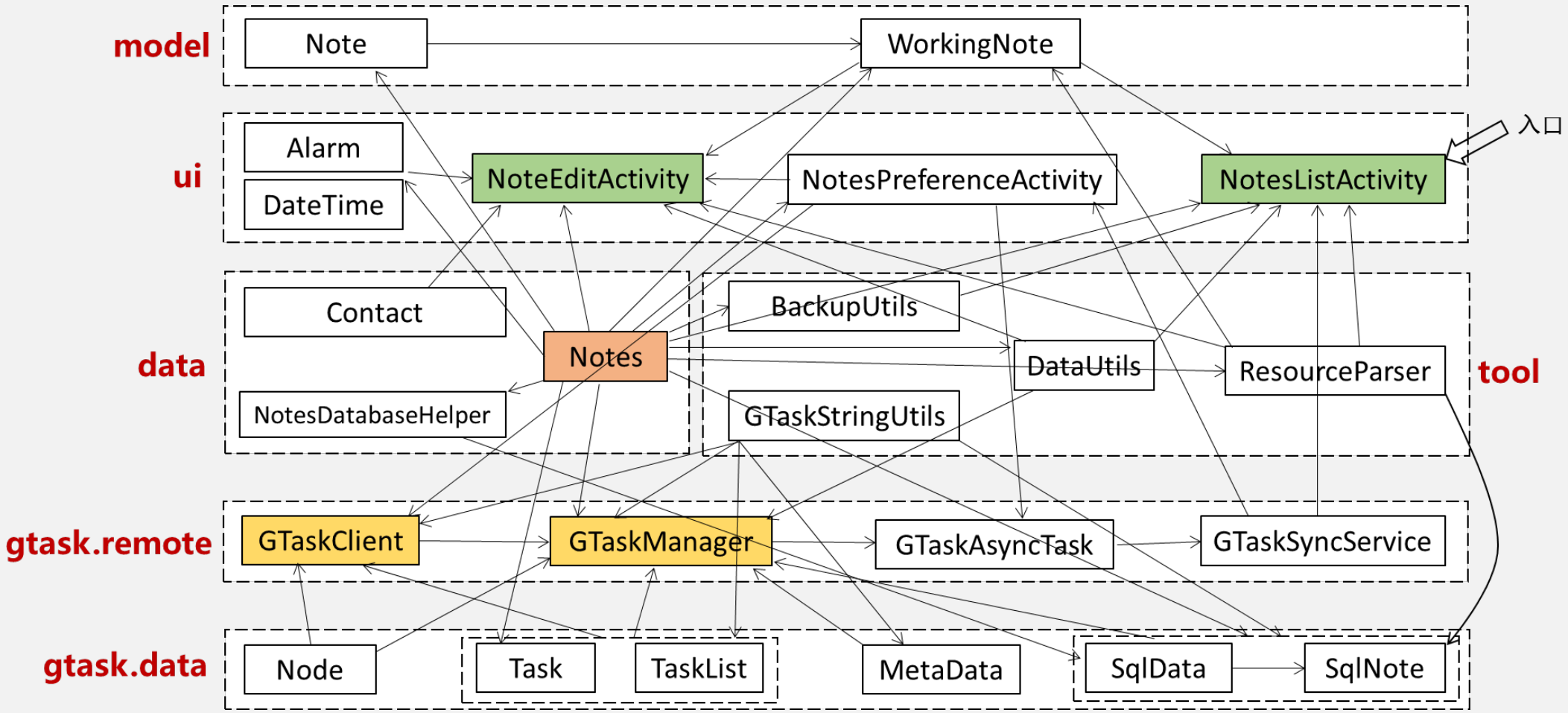
设计实现需求
设计指导实现

- 软件系统的解决方案类似于软件实现图纸

从实现的角度，软件设计应该提供什么样的解决方案?



示例：“小米便签”的软件实现解决方案



体系架构设计

Part 1 软件设计概述

- 软件设计的基本概念
- 软件设计的特点与质量要求
- 软件设计的基本原则

软件设计特点：多样性和差异性

- ✓ 如何区分设计的**差异性**?
- ✓ 如何评价设计的**优劣**?
- ✓ 除了满足需求之外，设计还需要**注意哪些要素**?

用户需求 → 软件设计



设计结果1



设计结果2



设计结果n

- □ 软件设计是一个创作的过程
- □ 一个软件需求会有多种软件设计方案

主观题 10分

设置

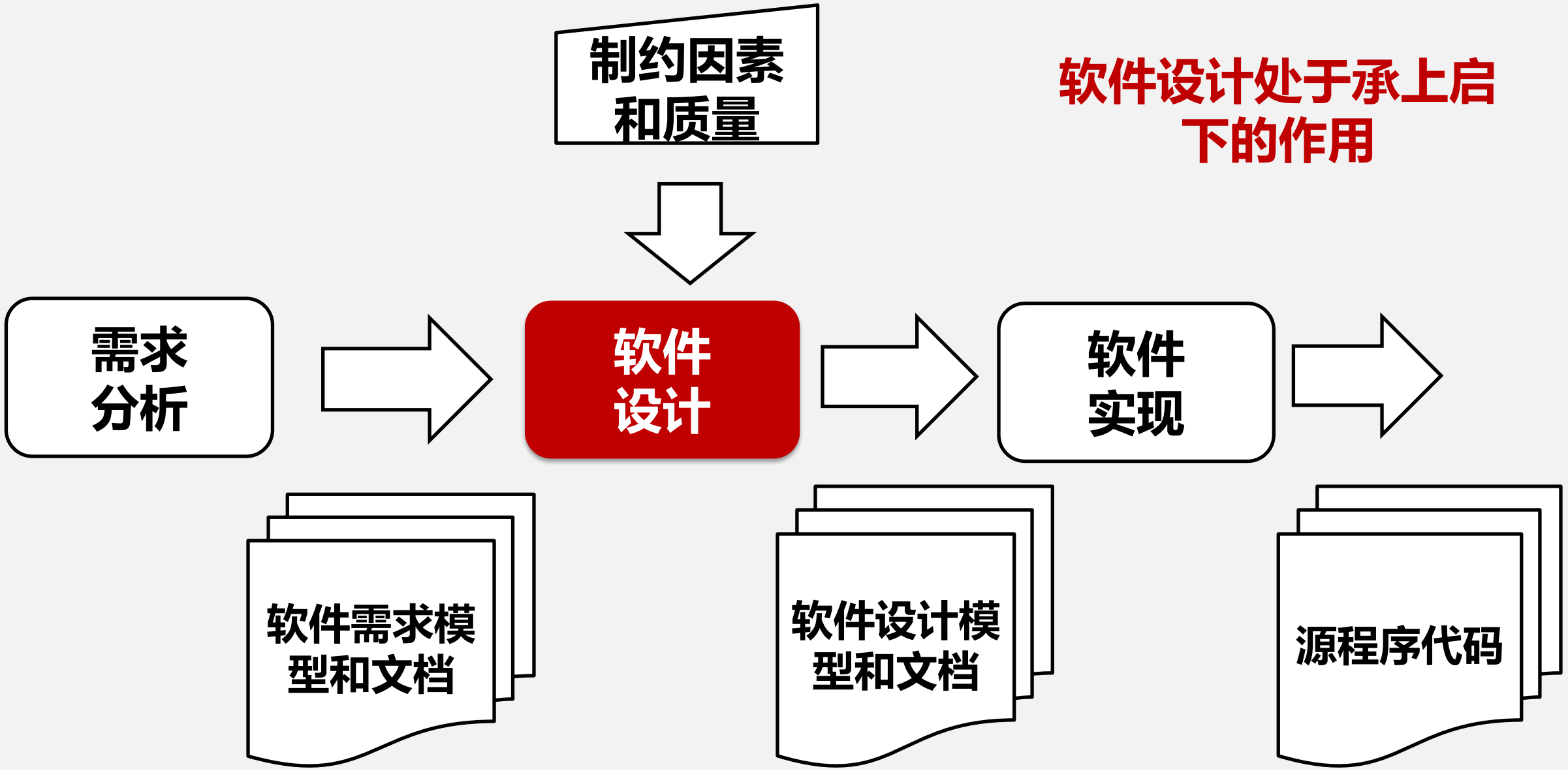
思考和讨论：

确保软件设计的质量重要吗？为什么？



Answer

需求分析、软件设计、软件实现间的关系



主观题 10分

设置

思考和讨论：

“好的” “高质量”的软件设计应满足什么要求?



Answer

软件设计的质量要求

- **正确性**
  - ✓ 正确**实现**所有的软件**需求项**；设计元素间**无逻辑冲突**；在技术平台和软件项目的可用资源约束条件下，采用程序设计语言可完整地实现设计模型
- **充分性**
  - ✓ 所有的设计元素**已充分细化**，模型易于理解，编程人员勿需再面对影响软件功能和质量的技术抉择或权衡
- **优化性**
  - ✓ 以合理的、充分优化的方式实现软件需求模型，目标软件产品能够表现出良好的软件质量属性，尤其是**正确性、有效性、可靠性和可修改性**
- **简单性**
  - ✓ 模型中的**模块的功能**或职责尽可能**简明易懂**，**模块间的关系简单直观**，模型的结构尽可能自然地反映待解软件问题的结构

如何才能得到高质量的软件设计呢？==》设计原则

Part 1 软件设计概述

- 软件设计的基本概念
- 软件设计的特点与质量要求
- 软件设计的基本原则

软件设计基本原则

① 抽象
② 模块化
③ 信息隐藏
④ 关注点分离
⑤ 软件重用
⑥ 迭代
⑦ 可追踪性

抽象原则

❑何为抽象?

- ✓在认识事物、分析和解决问题的过程中，**忽略**那些与当前研究目标**不相关的部分**，以便将**注意力集中于与当前目标相关的方面**
- ✓**不要过早地考虑细节**
- ✓抽象是**管理和控制复杂性**的基本策略

❑抽象在软件设计中的应用

- ✓软件开发就是一个**从高层抽象到低层抽象逐步过渡过程**
- ✓高层设计 ➔ 底层设计　“逐步求精”

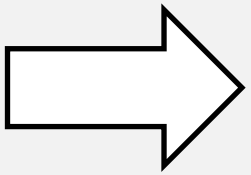
软件设计抽象层次的变化

- 结构性
- 全局性
- 关键性

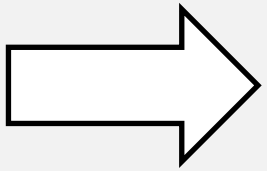


- 过程性
- 局部性
- 细节性

体系结构设计抽象



类设计抽象



算法设计抽象



逐步求精

主观题 10分

设置

思考和讨论：

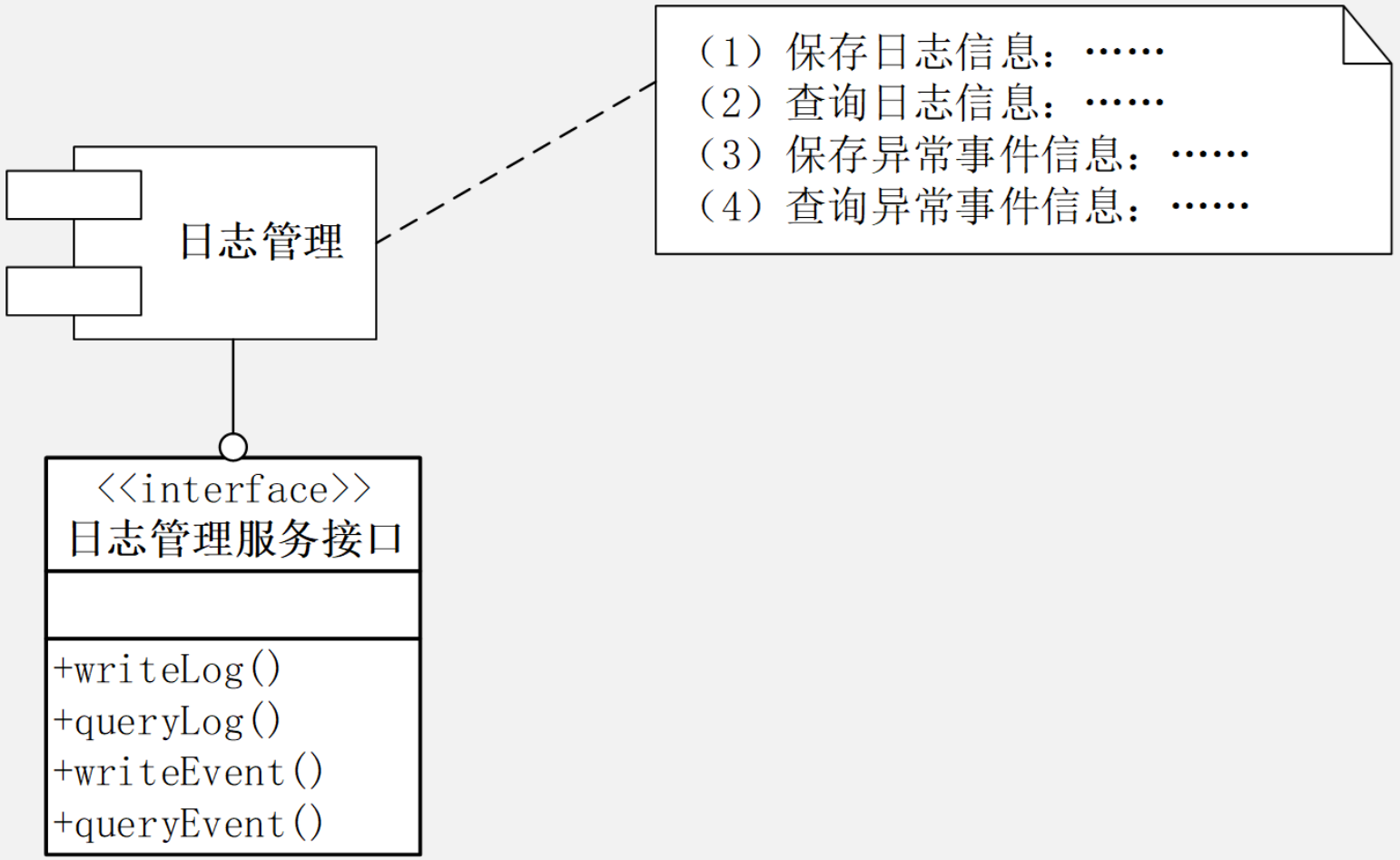
软件设计过程中不采用抽象的原则会产生什么样的结果?



Answer

示例：体系结构层次的设计抽象

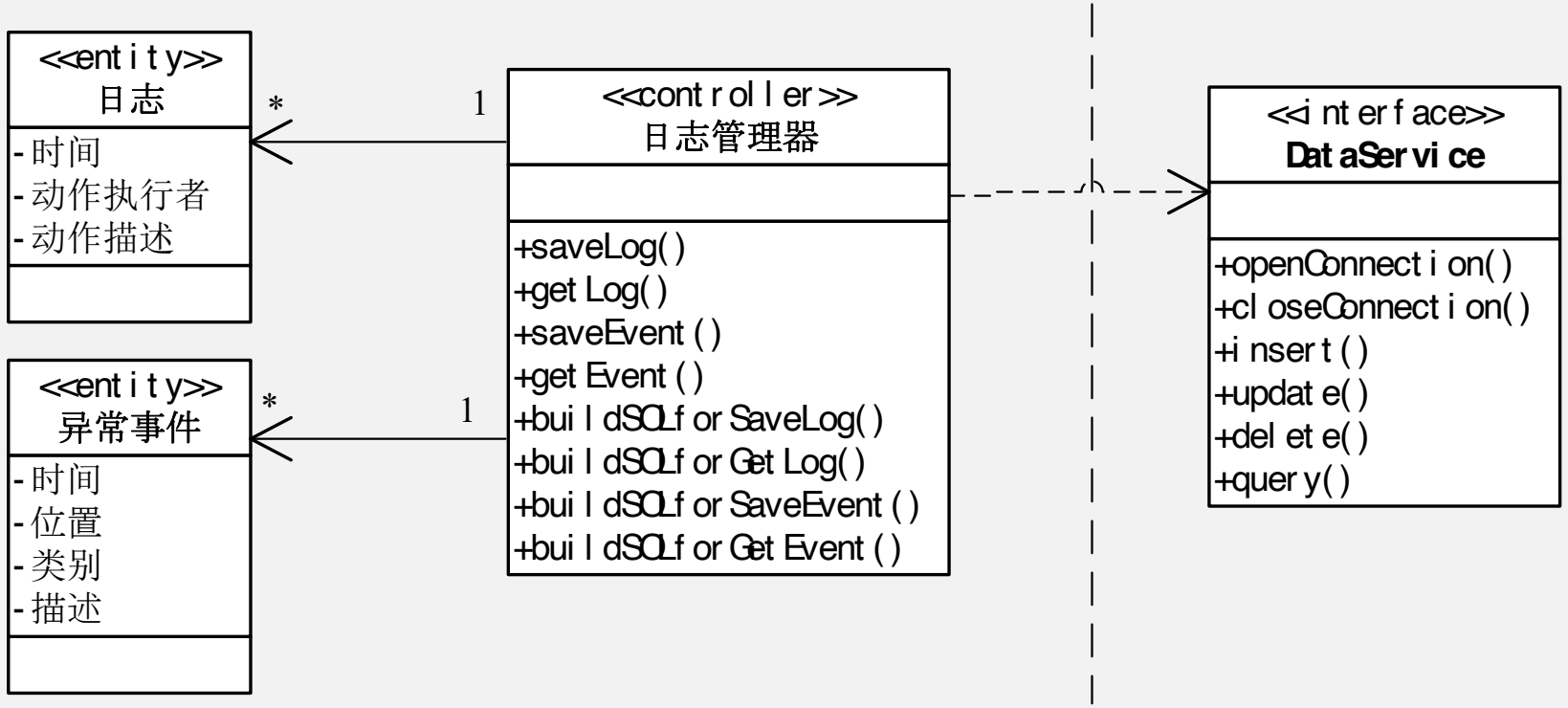
❑关注组件的职责和接口



外部的功能和服务

示例：构件层次的设计抽象

❑考虑构件内部设计元素的职责和协同



内部的结构和构成

模块化

- **将软件系统的整体结构分解为一组相关模块**
  - ✓模块：包、子系统、构件、类、方法等等
  - ✓**每个模块实现单一的功能**
  - ✓通过模块之间的交互来组装模块，形成整体框架
  - ✓体现了“分而治之”思想
- **如何达成模块化**
  - ✓模块内部强内聚
  - ✓模块之间低耦合



主观题 10分

设置

思考和讨论：

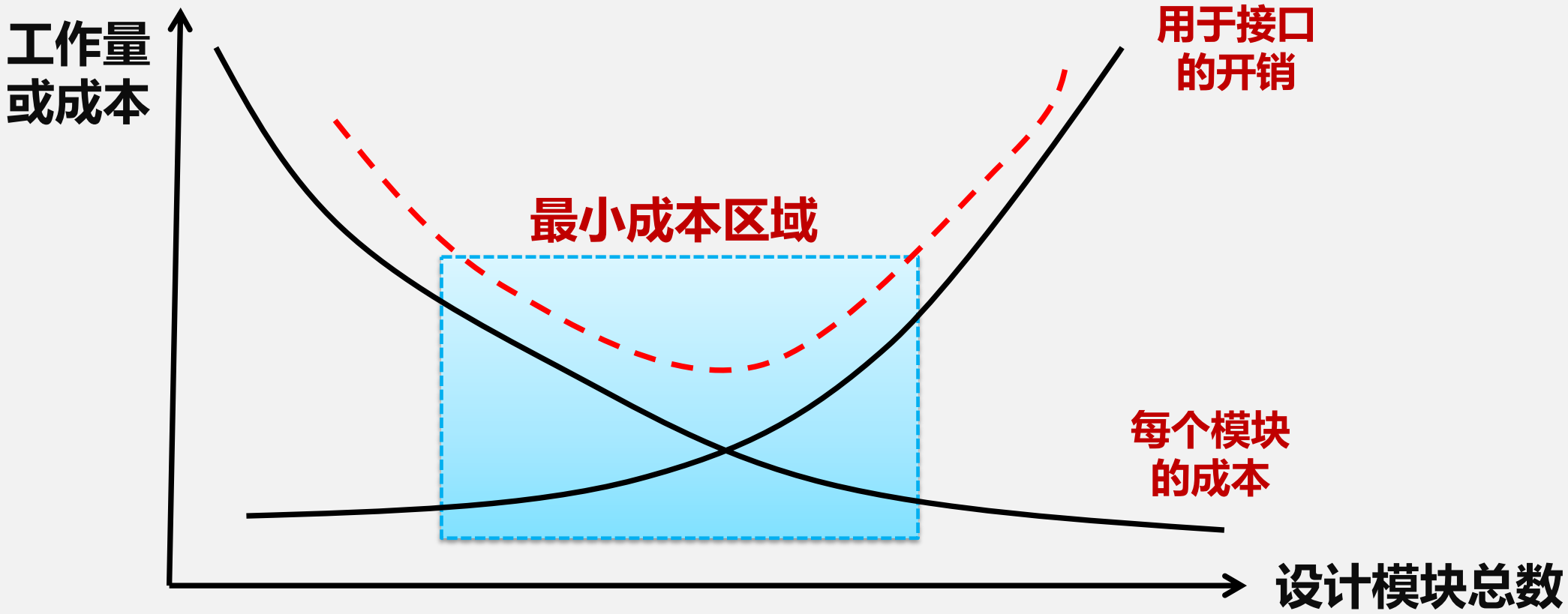
不按照模块化的原则来进行软件设计会怎样？





Answer

模块分解与开发成本之间的关系



按照模块化原则开展软件设计，有助于得到模块数量适中、开发工作量和成本较低、易于维护的设计结果

高内聚度原则

❑何为模块的内聚度?

✓指该模块内各成分间彼此结合的紧密程度，越高越好，高内聚

❑内聚度分类

✓**偶然性内聚**：模块内各成分为**完成一组功能而结合在一起**，关系松散

✓逻辑性内聚：模块完成的诸**任务逻辑上相关**

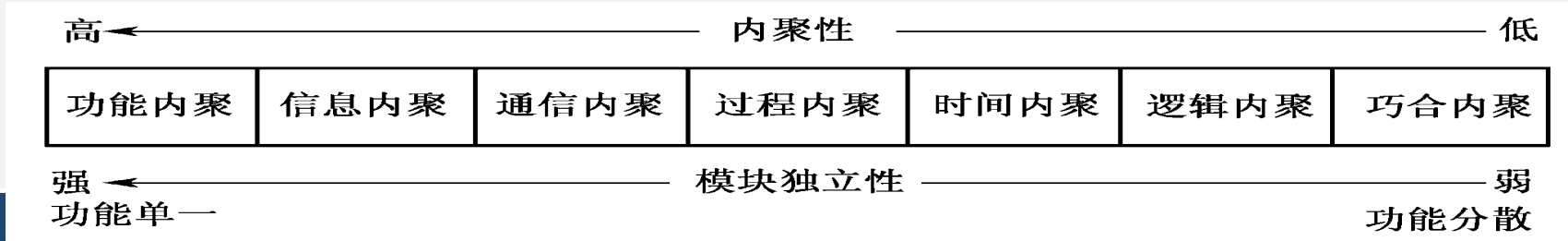
✓时间性内聚：模块内诸任务必须**在同一时间段内**执行

✓过程性内聚：模块内各成分相关且**必须按特定次序**执行

✓通讯性内聚：模块内各成分**对数据结构的同一区域**操作

✓顺序性内聚：模块内各成分与**同一功能相关且顺序执行**

✓**功能性内聚：**模块内各成分是**一整体，完成单个功能**



低耦合度原则

❑何为模块间的耦合度?

✓模块间的相关程度，越低越好，低耦合

❑耦合度分类

✓**非直接耦合**：二个模块都**不依赖对方**而独立存在

✓数据耦合：二个模块通过**参数交换信息**且仅限于**数据**

✓控制耦合：二个模块通过**参数交换信息**包含**控制信息**

✓特征耦合：介于数据耦合和控制耦合之间

✓外部耦合：二个模块与**同一外部环境相关联**(文件等)

✓公共耦合：模块间通过**全局数据环境**相互作用

✓**内容耦合**：一个模块使用另一**模块内的数据和控制信息**，或者直接转移到另一模块内执行



信息隐藏原则

❑何为信息隐藏?

- ✓模块应该设计得使其所含的信息对那些**不需要这些信息的模块不可访问**；模块间**仅仅交换**那些为完成系统功能所**必需交换的信息**

❑优点

- ✓模块的**独立性更好**
- ✓支持模块的**并行开发**（设计和编码）
- ✓**便于测试和维护**，减少错误向外传播
- ✓便于增加新的功能

信息隐藏示例

- 模块只提供**对外接口**，不提供内部**实现细节**
  - ✓**Public 方法对外可访问**
- 某些方法或属性设计为不可访问
  - ✓**Private不可访问**

```
package net.micode.notes.data;


import ...


public class NotesProvider extends ContentProvider {
    private static final UriMatcher mMatcher;

    private NotesDatabaseHelper mHelper;

    private static final String TAG = "NotesProvider";

    private static final int URI_NOTE            = 1;
    private static final int URI_NOTE_ITEM       = 2;
    private static final int URI_DATA            = 3;
    private static final int URI_DATA_ITEM       = 4;

    private static final int URI_SEARCH          = 5;
    private static final int URI_SEARCH_SUGGEST  = 6;

    static {
        mMatcher = new UriMatcher(UriMatcher.NO_MATCH);
        mMatcher.addURI(Notes.AUTHORITY, "note", URI_NOTE);
        mMatcher.addURI(Notes.AUTHORITY, "note/#", URI_NOTE_ITEM);
```

关注点分离原则

❑何为关注点

- ✓针对概念、任务和目标的**某个部分或者侧面的聚焦**
- ✓关注点：结构、行为等

❑何为关注点分离

- ✓设计师将若干性质不同的关注点分离开来，以便**在适当的时间处理不同的关注点**，随后将这些关注点整合起来，形成局部或者全局性的设计结果
- ✓防止“胡子眉毛一把抓”

主观题 10分

设置

思考和讨论：

关注点不分离会产生什么样的后果？





Answer

软件重用原则

- **尽可能地重用已有的软件资产来实现软件系统的功能，同时要确保所开发的软件系统易于为其他软件系统所重用**
- **支持软件重用的技术手段**
  - ✓封装、接口、继承、多态等
- **软件重用的对象**
  - ✓过程和函数、类、软构件、开源软件
  - ✓软件设计模式、软件开发知识

北京航空航天大學
BEIHANG UNIVERSITY



Part 2

软件设计过程

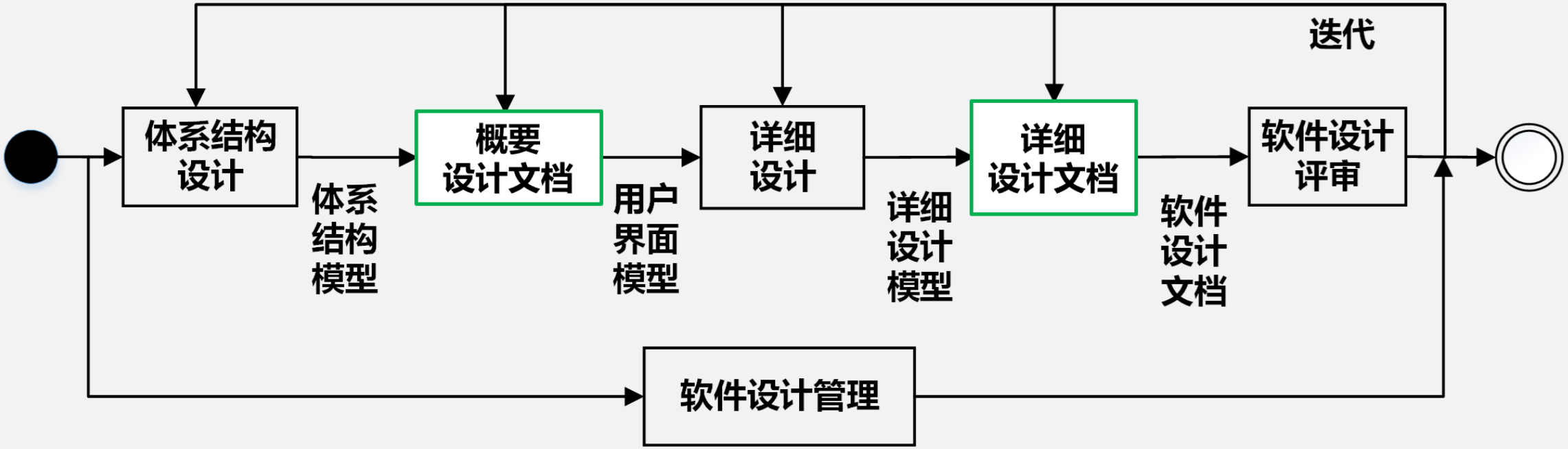
Part 2 软件设计过程

- 软件设计过程概述
- 体系架构设计
- 软件详细设计
- 软件设计评审

Part 2 软件设计过程

- 软件设计过程概述
- 体系架构设计
- 软件详细设计
- 软件设计评审

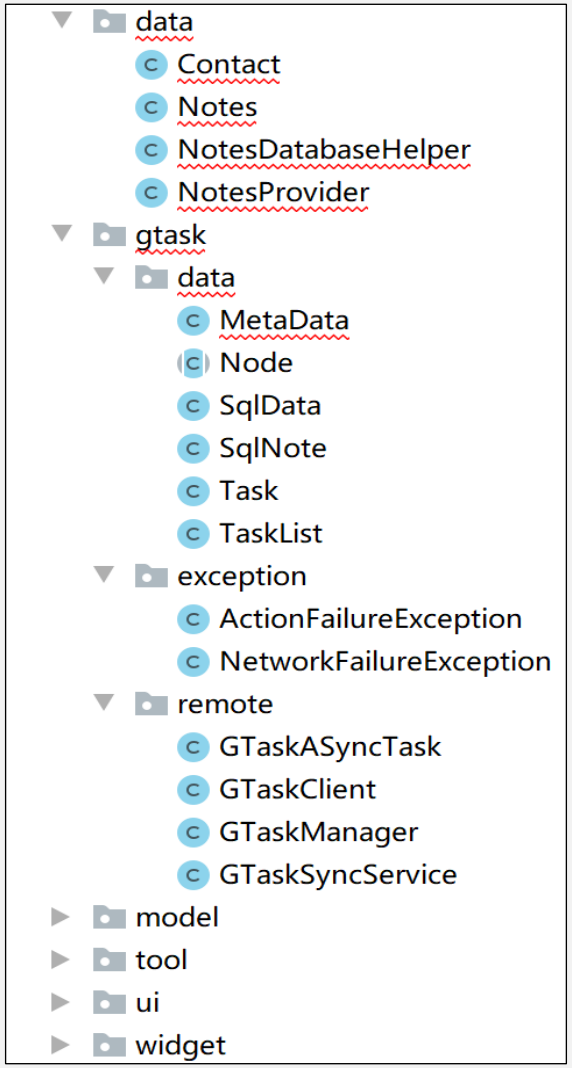
软件设计的过程


迭代
体系结构设计
概要设计文档
详细设计
详细设计文档
软件设计评审
体系结构模型
用户界面模型
详细设计模型
软件设计文档
软件设计管理

软件设计元素

- **子系统**
  - ✓完成特定功能、逻辑上相互关联的一组模块集合
  - ✓有助于管理软件系统的复杂度，简化软件设计和实现
- **软件组件**
  - ✓可分离、可独立部署和执行、可单独重用的一类设计元素
  - ✓如动态链接库（.DLL）、可运行的Java JAR包、微服务镜像等就属于软构件
- **设计类/设计函数**
  - ✓**面向对象**：类既是最基本的设计单元，也是最基本的模块单元
  - ✓**面向过程**：函数既是最基本的设计单元，也是最基本的模块单元

示例：软件设计元素




```java
package net.micode.notes.data;


import ...


public class NotesProvider extends ContentProvider {
    private static final UriMatcher mMatcher;

    private NotesDatabaseHelper mHelper;

    private static final String TAG = "NotesProvider";

    private static final int URI_NOTE            = 1;
    private static final int URI_NOTE_ITEM       = 2;
    private static final int URI_DATA            = 3;
    private static final int URI_DATA_ITEM       = 4;

    private static final int URI_SEARCH          = 5;
    private static final int URI_SEARCH_SUGGEST  = 6;

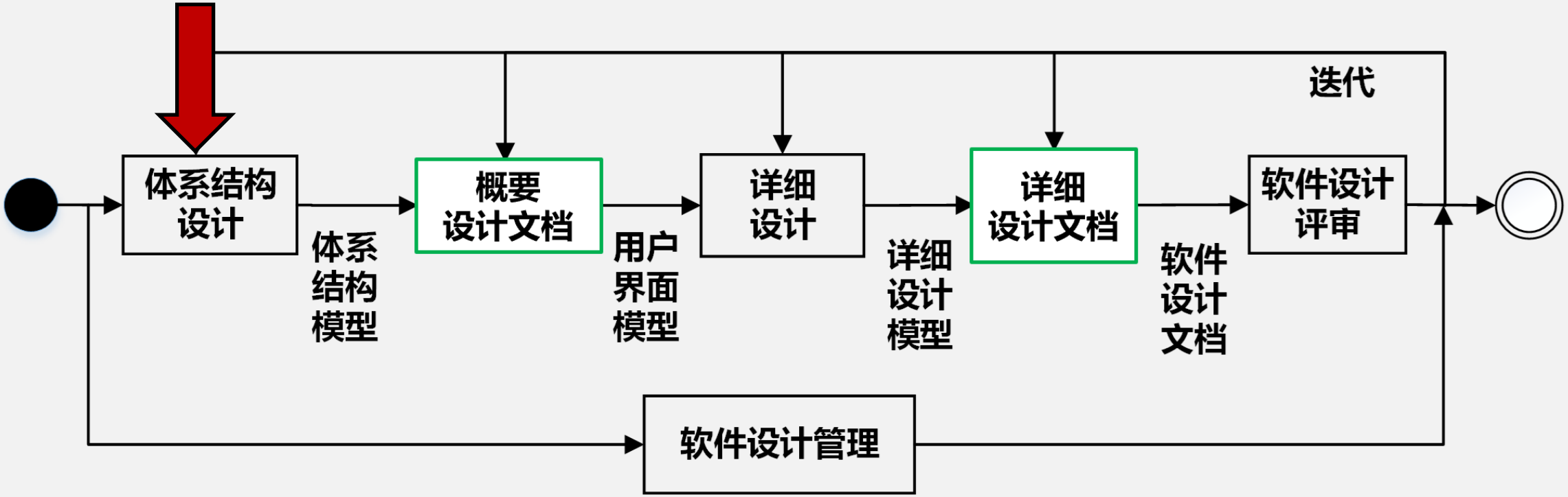
    static {
        mMatcher = new UriMatcher(UriMatcher.NO_MATCH);
        mMatcher.addURI(Notes.AUTHORITY, "note", URI_NOTE);
        mMatcher.addURI(Notes.AUTHORITY, "note/#", URI_NOTE_ITEM);
```

所有的设计元素在编码实现时都有相应的对应物

Part 2 软件设计过程

- 软件设计过程概述
- 体系架构设计
- 软件详细设计
- 软件设计评审

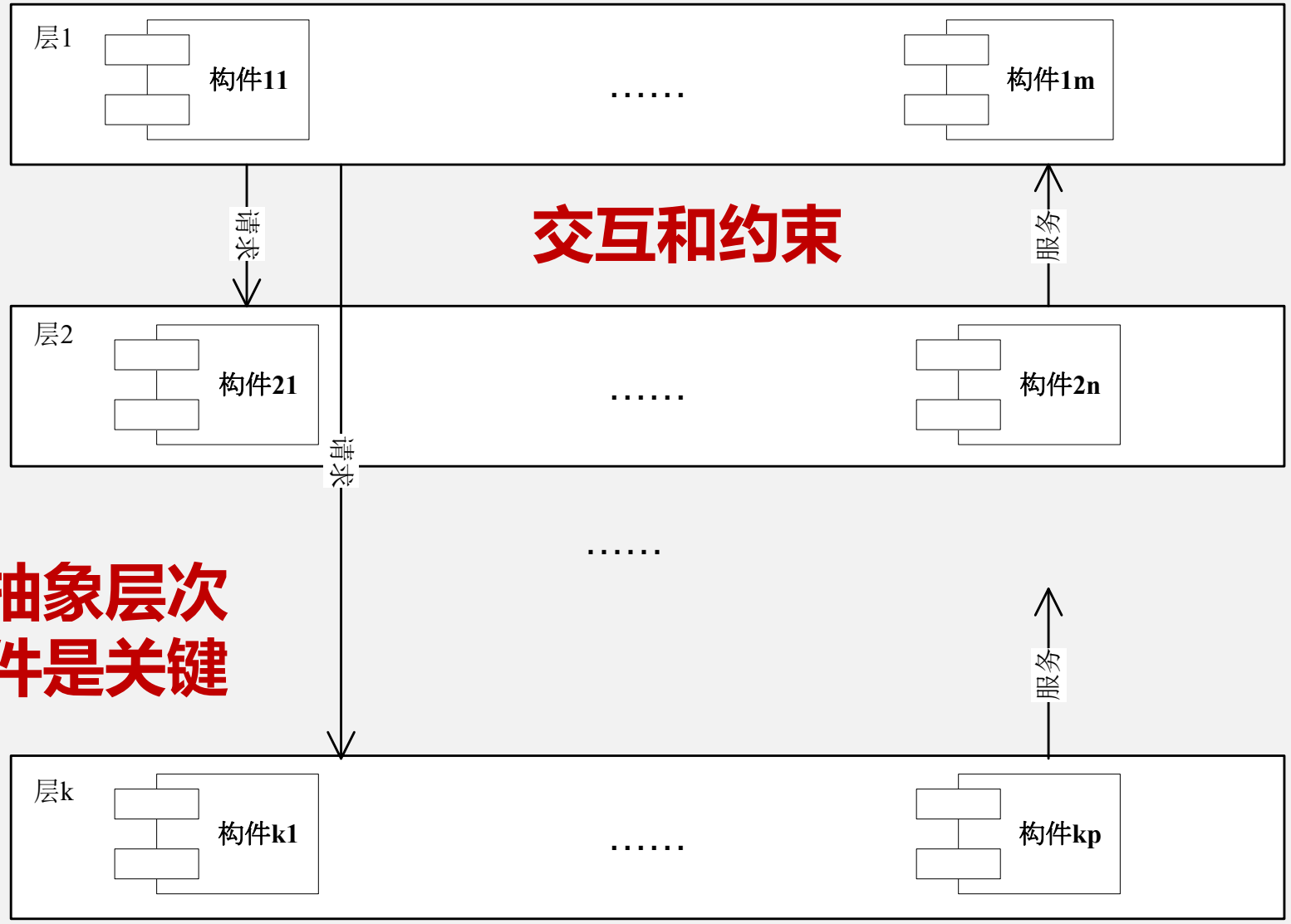
软件设计的过程


迭代
体系结构设计
概要设计文档
详细设计
详细设计文档
软件设计评审
体系结构模型
用户界面模型
详细设计模型
软件设计文档
软件设计管理

软件设计的过程 – 软件体系结构设计

- 从全局和宏观视角、站在最高抽象层次来设计软件系统
  - ✓模块及其关系
  - ✓职责分派、接口定义
  - ✓相互交互及协作行为
- 每个模块为“黑盒子”
- 设计关注的质量要素
  - ✓可扩展、可维护、可重用、可移植、可互操作等等

示例：分层体系结构风格



界面

业务

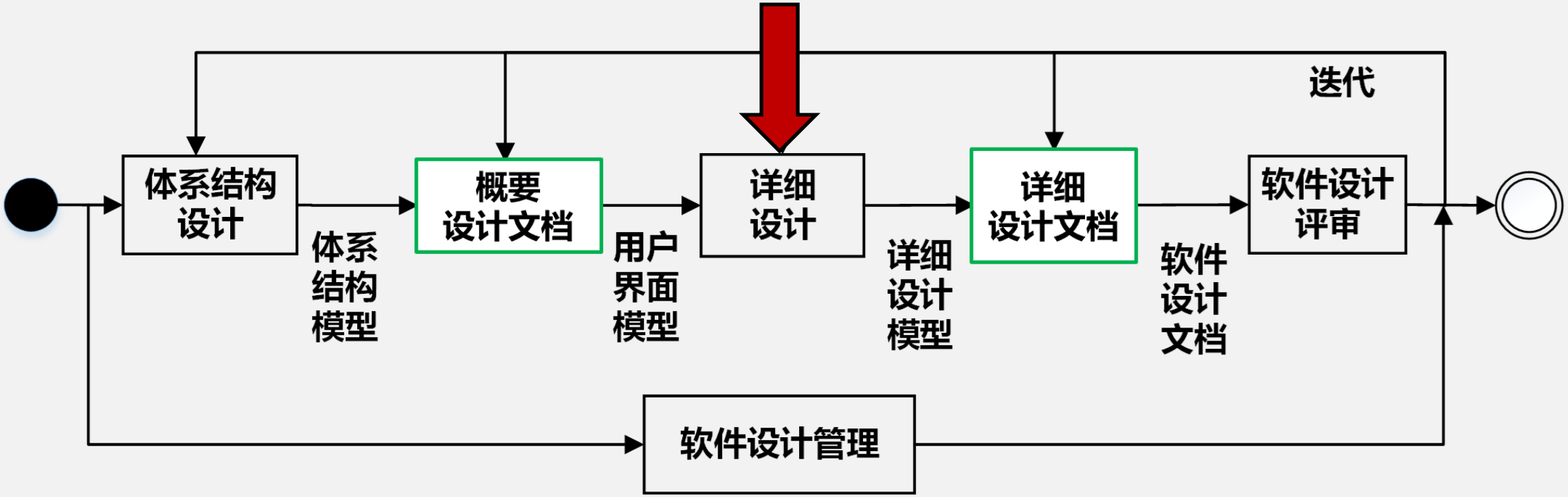
服务

合理地设计抽象层次和组织软构件是关键

Part 2 软件设计过程

- 软件设计过程概述
- 体系架构设计
- 软件详细设计
- 软件设计评审

软件设计的过程


迭代
体系结构设计
概要设计文档
详细设计
详细设计文档
软件设计评审
体系结构模型
用户界面模型
详细设计模型
软件设计文档
软件设计管理

软件设计的过程 – 软件详细设计

- **对体系结构设计成果进行细化和精化，获得高质量的、充分细化的软件设计模型**
  - ✓**面向对象：**
    - **类设计**：细化各个类设计，包括属性、**操作**等
    - **对象之间的交互设计（算法）**：实现特定功能的具体执行流程和算法
  - ✓**面向过程：**
    - 函数设计：详细的计算过程
    - 函数调用：函数之间的复合关系
  - ✓**界面设计**
- **设计关注的质量要素**
  - ✓有效、高效、可靠、易于维护等等

主观题 10分

设置

思考和讨论：

不按照上述过程来进行软件设计行吗？先进行详细设计，再进行体系结构设计会产生什么样的问题?



Answer

软件设计的输出

❑软件设计模型

- ✓它从多个不同的视角、不同的抽象层次描述了软件的设计信息，并采用诸如UML、模块图、层次图等图形化的方式来加以刻画

❑软件设计文档

- ✓它采用自然语言的形式，结合软件设计模型，详细描述软件系统的各项设计，包括体系结构设计、子系统和构件设计、用户界面设计、用例设计、数据设计等等

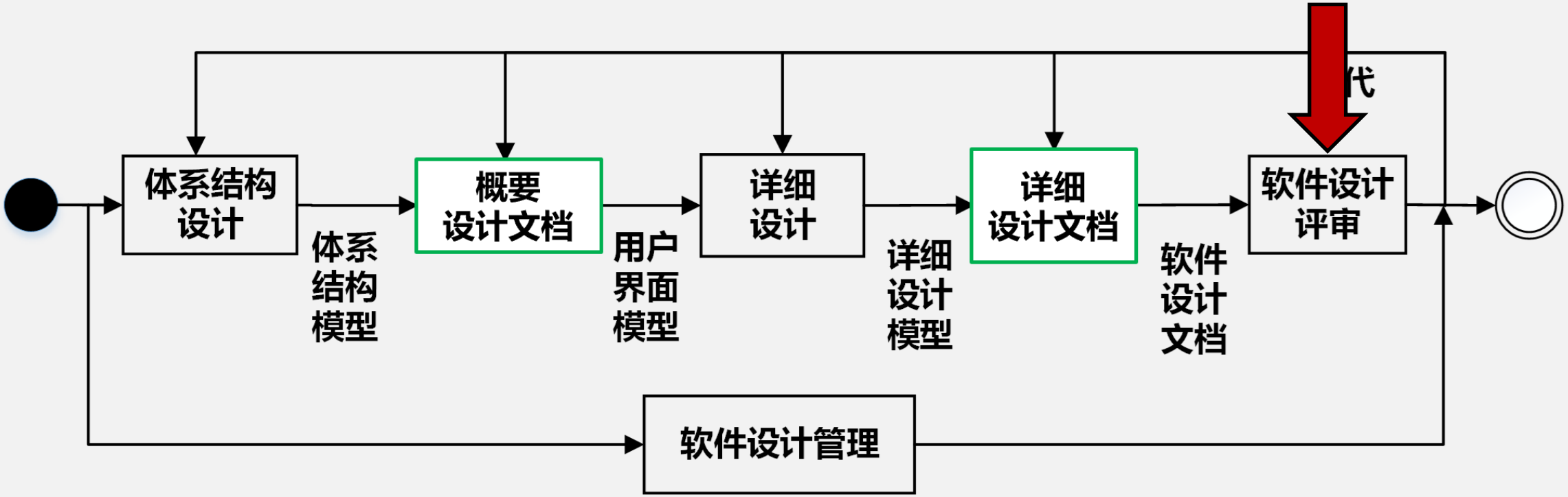
软件设计文档的规范

- 文档概述
- 系统概述
- 设计目标和原则
- 设计约束和现实限制
- 体系结构设计
- 用户界面设计
- 子系统/构件设计
- 类与对象交互设计
- 函数设计

Part 2 软件设计过程

- 软件设计过程概述
- 体系架构设计
- 软件详细设计
- 软件设计评审

软件设计的过程


体系结构
设计
概要
设计文档
详细
设计
详细
设计文档
软件设计
评审
代
体系
结构
模型
用户
界面
模型
详细
设计
模型
软件
设计
文档
软件设计管理

软件设计中的缺陷

- **设计未能满足需求**
  - ✓对软件需求的理解存在偏差，未能正确地理解用户的软件需求，导致所设计的软件无法满足用户的需要
- **设计质量低下**
  - ✓设计过程中未能遵循设计原则、缺乏设计经验，导致软件设计质量低下，如设计的软件不易于维护和扩展
- **设计存在不一致**
  - ✓不同软件设计制品对同一个设计有不同的描述，或者存在不一致甚至相冲突的设计内容；多个不同软件设计要素之间存在不一致
- **设计不够详尽**
  - ✓未能提供设计细节性信息，导致程序员无法根据设计来开展编码工作

主观题 10分

设置

思考和讨论：

软件设计评审谁来评？怎么评？



Answer

软件设计的评审

❑谁参与评审

✓程序员、软件测试工程师、用户、质量保证人员、设计工程师等

❑评审什么内容

✓**文档规范性**，软件设计文档是否符合软件设计规格说明书

✓**设计制品的可理解性**，是否简洁、易于理解

✓**设计内容的合法性**，设计结果是否符合相关的标准、法律和法规

✓**设计的质量水平**，软件设计是否遵循设计原则，质量如何

✓**设计是否满足需求**，设计是否完整和正确地实现了软件需求

✓**设计优化性**，软件设计是否还有待优化的内容

软件设计的CASE工具

- **软件设计文档撰写工具**
  - ✓如借助于Microsoft Office、WPS、**LLM-based Agent**等
- **软件设计建模工具**
  - ✓如Microsoft Visio、StarUML、Visual Paradigm等工具
- **软件设计分析和转换工具**
  - ✓如IBM Rational Rhapsody等软件工具
- **配置管理工具和平台**
  - ✓如Git、Github、Gitlab、PVCS、Microsoft SourceSafe等，支持软件需求制品（如模型、文档等）的配置、版本管理、变化跟踪等

北京航空航天大學
BEIHANG UNIVERSITY



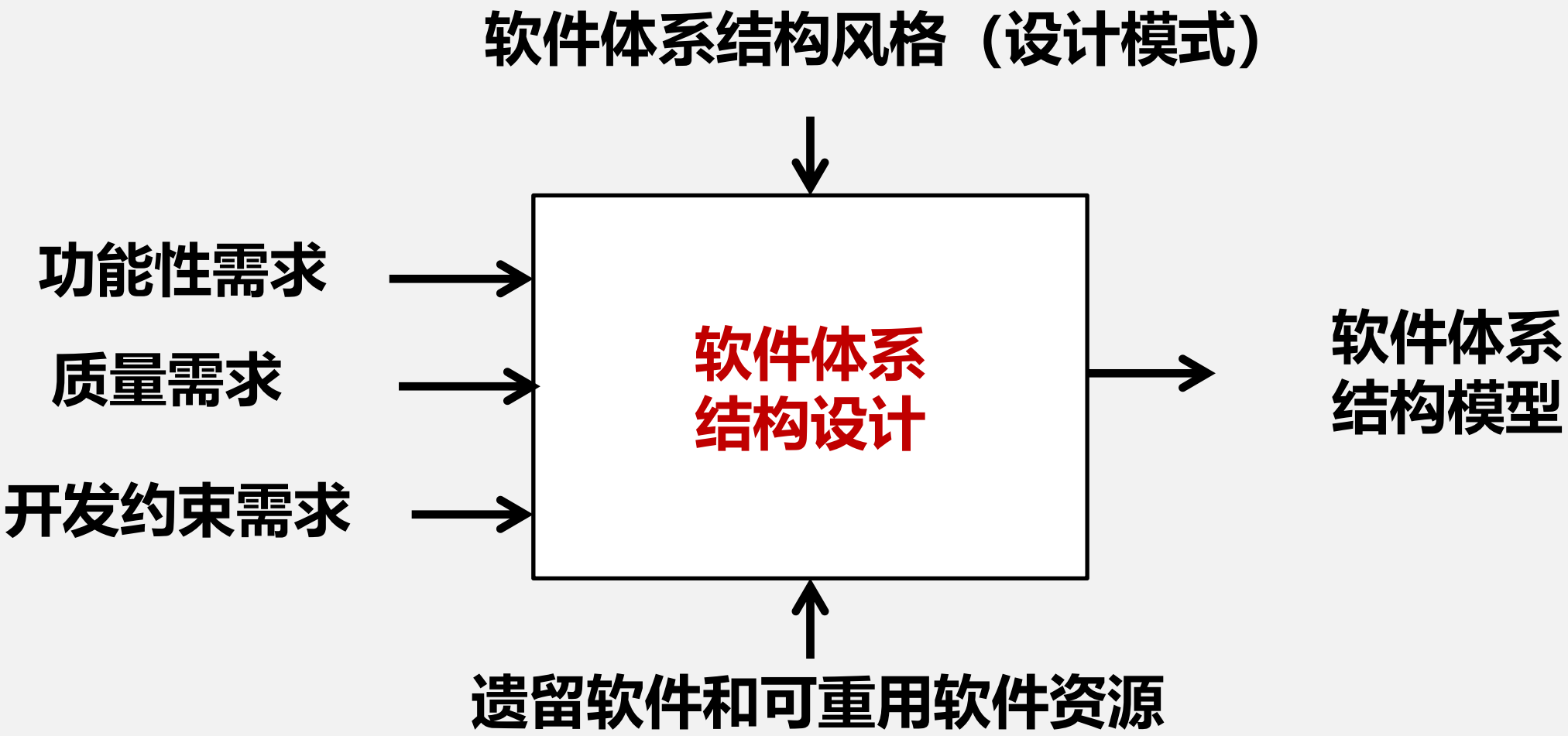
Part 3

软件架构设计

Part 3 软件架构设计

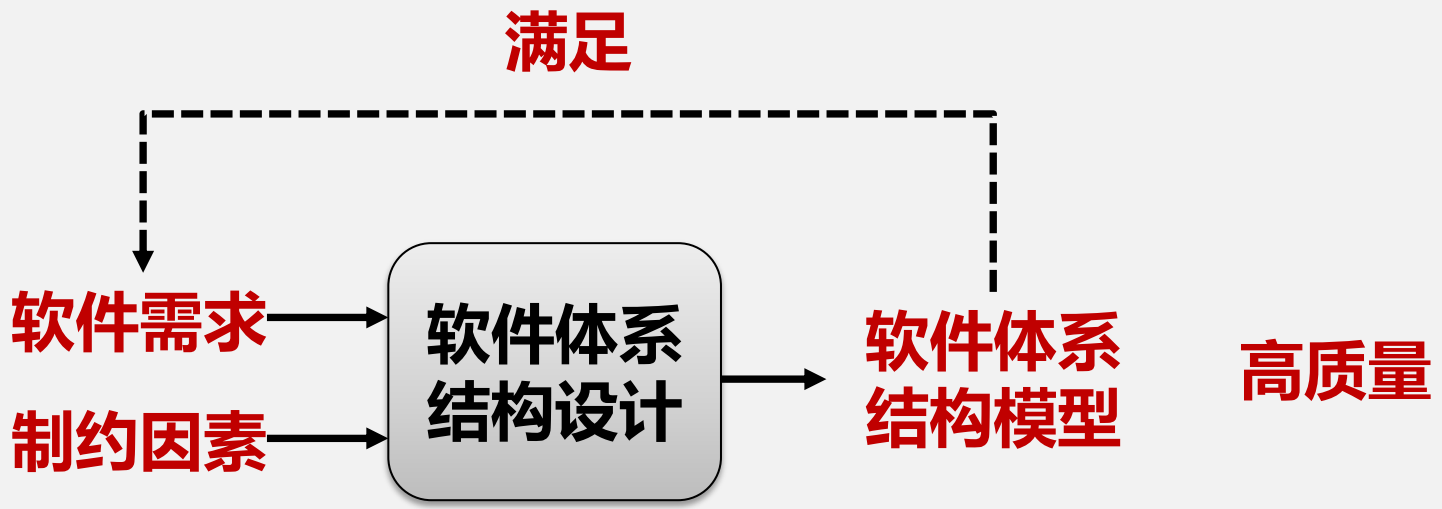
- 软件架构设计概述
- 软件架构设计目标与任务
- 软件架构设计过程

软件体系结构设计示意图



软件体系结构设计的目标

- **满足软件需求**
  - ✓实现软件需求
  - ✓……
- **追求高质量**
  - ✓可扩展
  - ✓可伸缩
  - ✓易维护
  - ✓……



非功能需求在体系结构设计中起着非常重要的作用

主观题 10分

设置

思考和讨论：

想一下架构设计都设计什么？



Answer

软件体系结构设计的任务

- **建立满足软件需求的软件体系结构**
  - ✓明确定义各模块的**职责划分及协作关系**
  - ✓明确它们在物理运行环境下的部署

- **特点**
  - ✓针对软件系统**全局性、基础性技术问题**给出技术解决方案
  - ✓宏观、全局、层次、战略、多视点、关键性

与软件需求关系?

- 体系结构是以软件需求实现为目标的**软件设计蓝图**
- **软件需求**是体系结构设计的基础和驱动因素
- 软件需求，尤其是**非功能需求**，对软件体系结构具有关键性的塑形作用



与详细设计关系?

- 体系结构设计为详细设计提供**可操作指导**
- 详细设计是对体系结构设计中设计要素的**局部设计**
- 详细设计须**遵循体系结构设计**：如接口和约束
- 详细设计**只能实现、不能更改**体系结构设计中规定模块的接口和行为

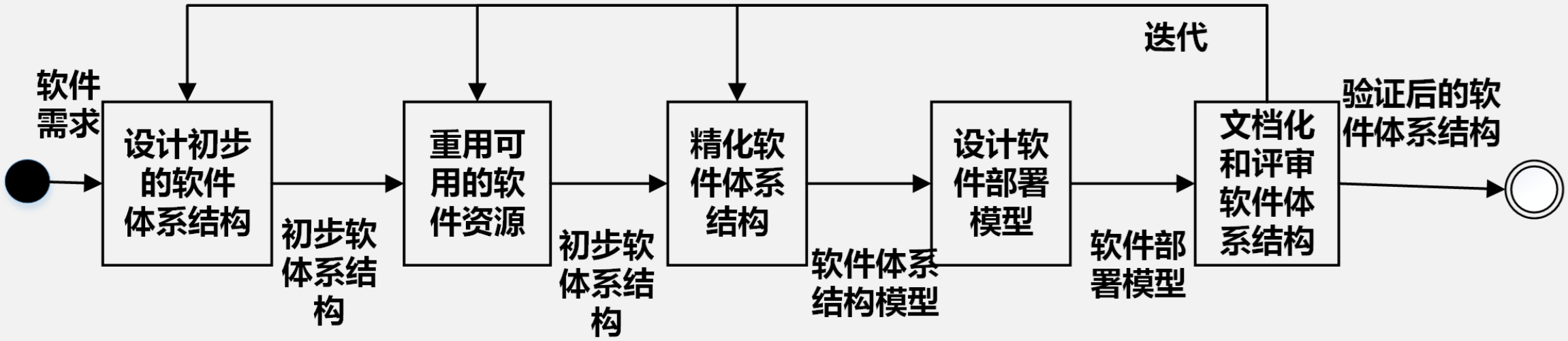


基于体系结构设计的结果来指导详细设计

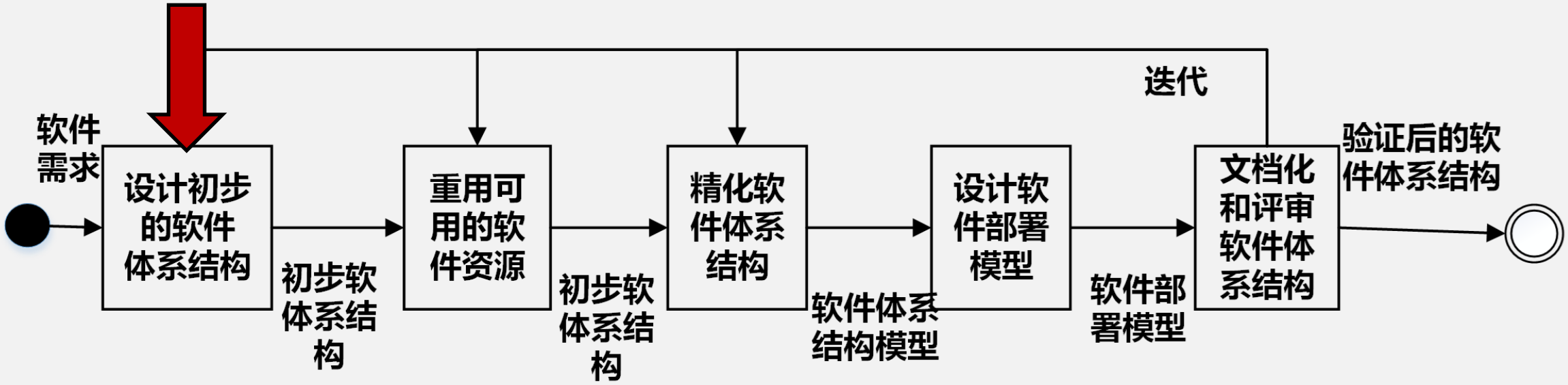
Part 3 软件架构设计

- 软件架构设计概述
- 软件架构设计目标与原则
- 软件架构设计过程

软件体系结构设计的过程



软件体系结构设计的过程



设计初步的软件体系结构

❑任务

- ✓基于功能性和非功能性**软件需求**，参考业界已有的**软件体系结构设计模式**，设计出目标软件系统的初始体系结构，明确**每个构件的职责**以及构件间的**通信和协作关系**

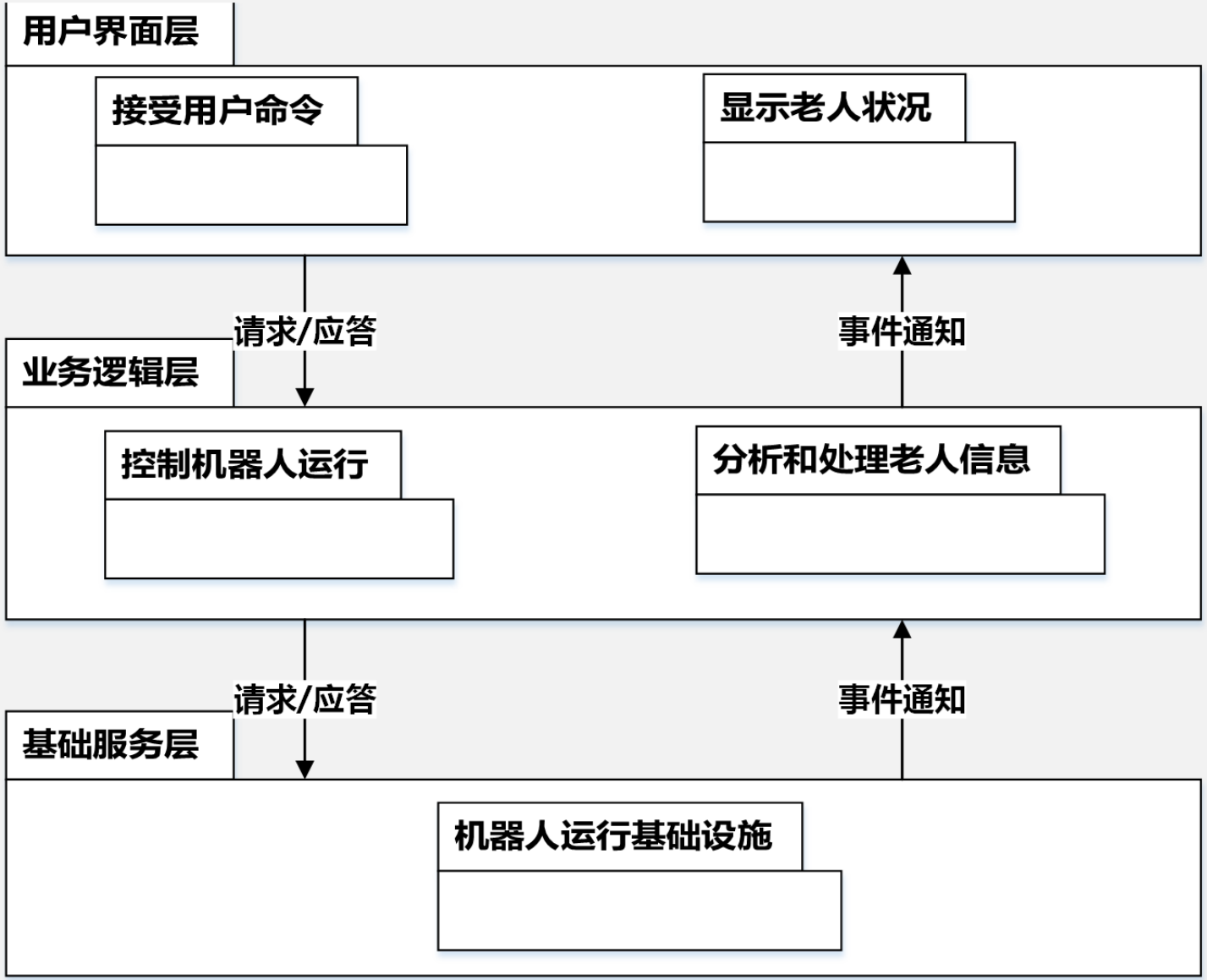
❑输出：初步的软件体系结构

- ✓针对软件需求，寻求体系结构风格，给出**初步和粗糙**的顶层架构，以供后续设计阶段的精化和细化
- ✓无需关注架构中各个子系统或构件内部的实现细节
- ✓用**线框图/UML中的包图**对所设计的顶层架构进行直观的表示

不同体系结构模式适合的应用

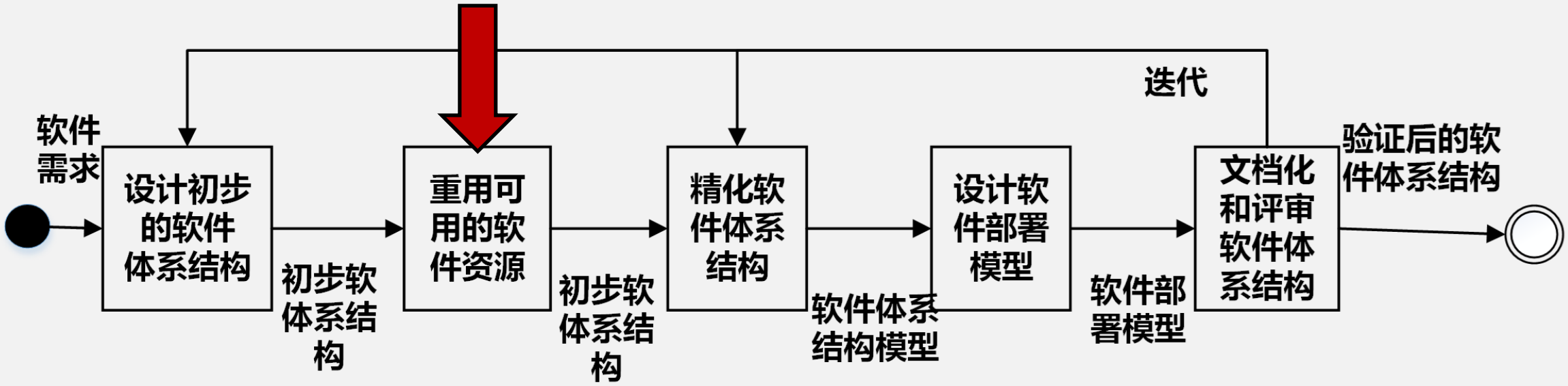
类别	特点	典型应用
管道/过滤器风格	数据驱动的分级处理，处理流程可灵活重组，过滤器可重用	数据驱动的事务处理软件，如编译器、Web服务请求等
层次风格	分层抽象、层次间耦合度低、层次的功能**可重用和可替换**	绝大部分的应用软件
MVC/MVP/MVVM风格	模型、处理和显示的职责明确，构件间的关系局部化，各个软构件**可重用**	单机软件系统，Web应用软件系统
SOA风格	以服务作为基本的构件，支持异构构件之间的互操作，服务的**灵活重用和组装**	部署和运行在云平台上的软件系统
消息总线风格	提供统一的消息总线，支持**异构构件**之间的消息传递和处理	异构构件之间消息通信密集型的软件系统

示例：初步软件体系结构的设计(2)



“空巢老人看护软件”初步软件体系结构设计

软件体系结构设计的过程



重用开源软件及已有软件资产

- **搜索**已有的粗粒度软件资产
  - ✓**可重用的软件开发包**，如函数库、类库、构件库等
  - ✓**互联网上的云服务**，为特定问题提供独立功能，如身份验证、图像识别、语音分析等等
  - ✓**遗留软件系统**，已存在的软件系统
  - ✓**开源软件**，提供针对特定功能的完整程序代码
- **寻找**可用于支持目标软件系统构建的软件制品
- **实现**软件架构中某个（些）子系统或组件的功能

到哪里找开源软件?

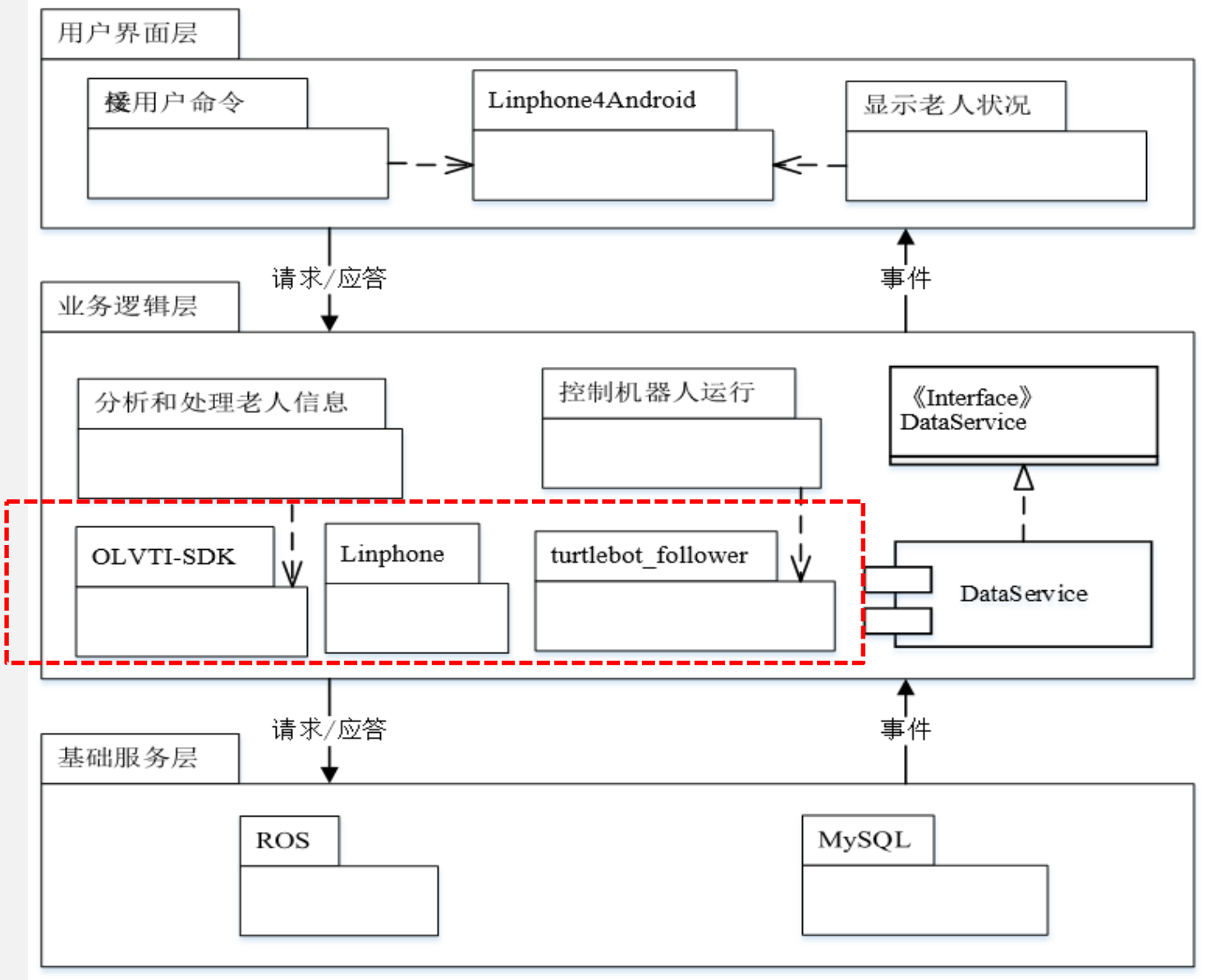
- **Github(github.com/)**
  - ✓开源软件托管平台和开源社区，180万商业组织，2700万开发者，8000万个版本库
- **SourceForge(sourceforge.net/)**
  - ✓开源软件仓库和开发平台，370万用户，43万项目
- **Gitee(www.gitee.com/)**
  - ✓ 中国人的开源社区
- **……**







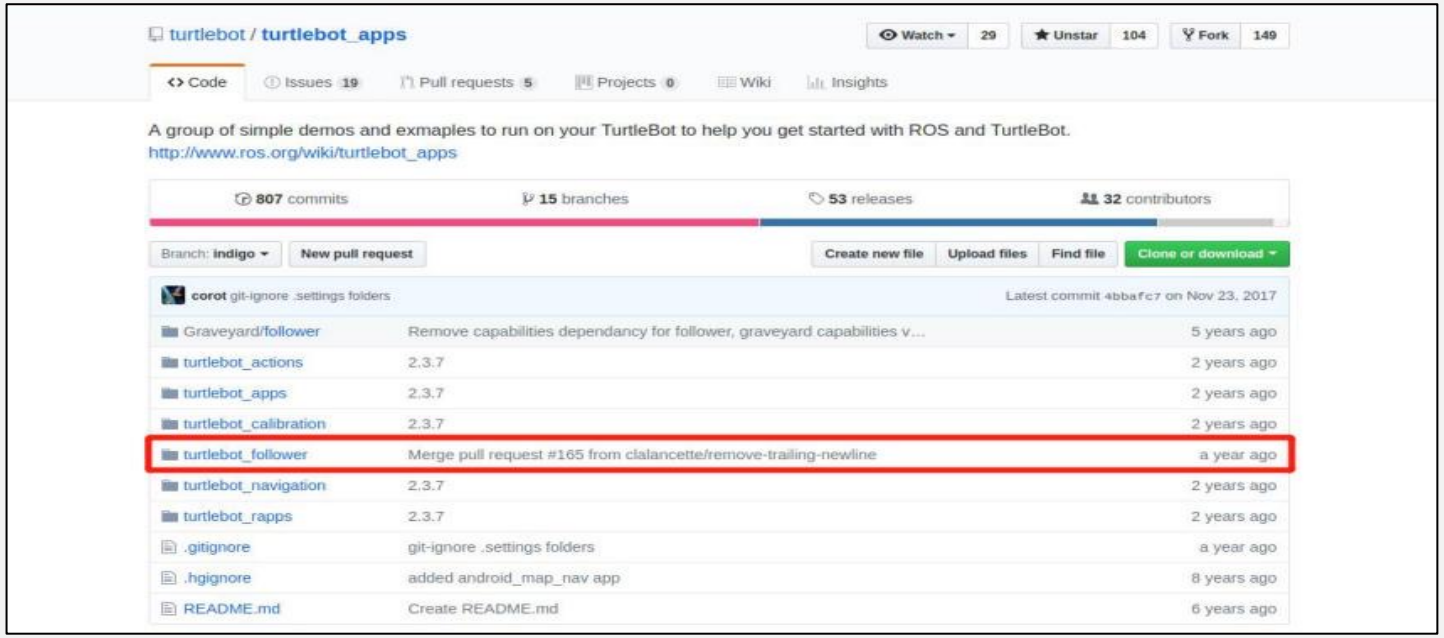
示例：“空巢老人看护软件”的开源软件重用



示例：搜寻和重用软件资产

❑机器人的自主跟随功能

- ✓搜寻和重用ROS代码“turtlebot_follower”来实现
- ✓在Github搜索“turtlebot app”
- ✓点击“turtlebot_follower”，阅读和下载其开源代码



示例：搜寻和重用软件资产

- **搜寻和重用“离线语音合成”软件包OLVTI-SDK实现文字到语音转换**
  - ✓讯飞开放平台（www.xfyun.cn/）提供“离线语音转换和合成”功能
  - ✓平台提供软件开发包SDK



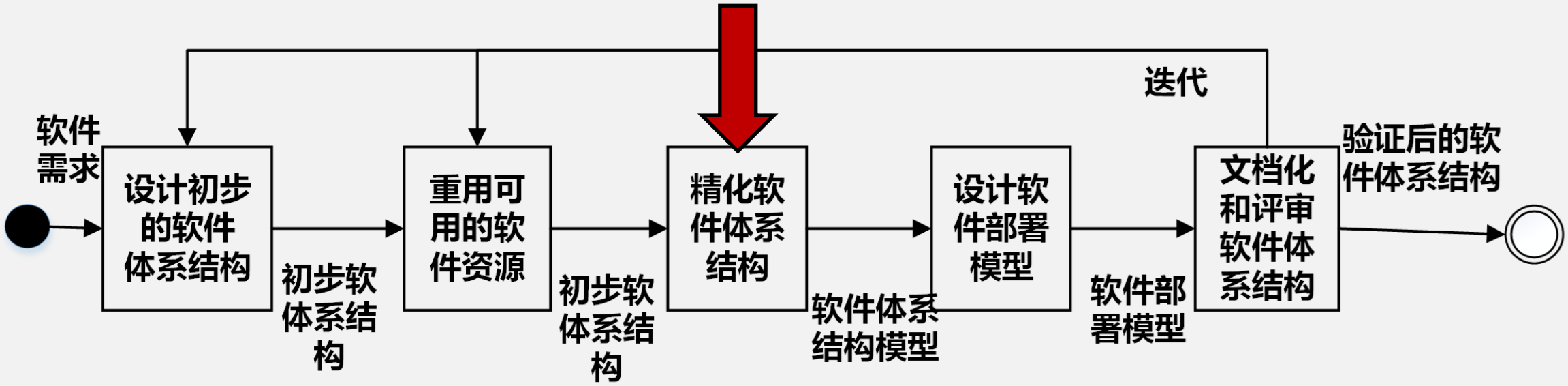
主观题 10分

设置

- 为什么软件体系结构设计时要考虑重用粗粒度软件资产?
- 如果在此过程中没有考虑软件重用，会带来什么样问题?

Answer

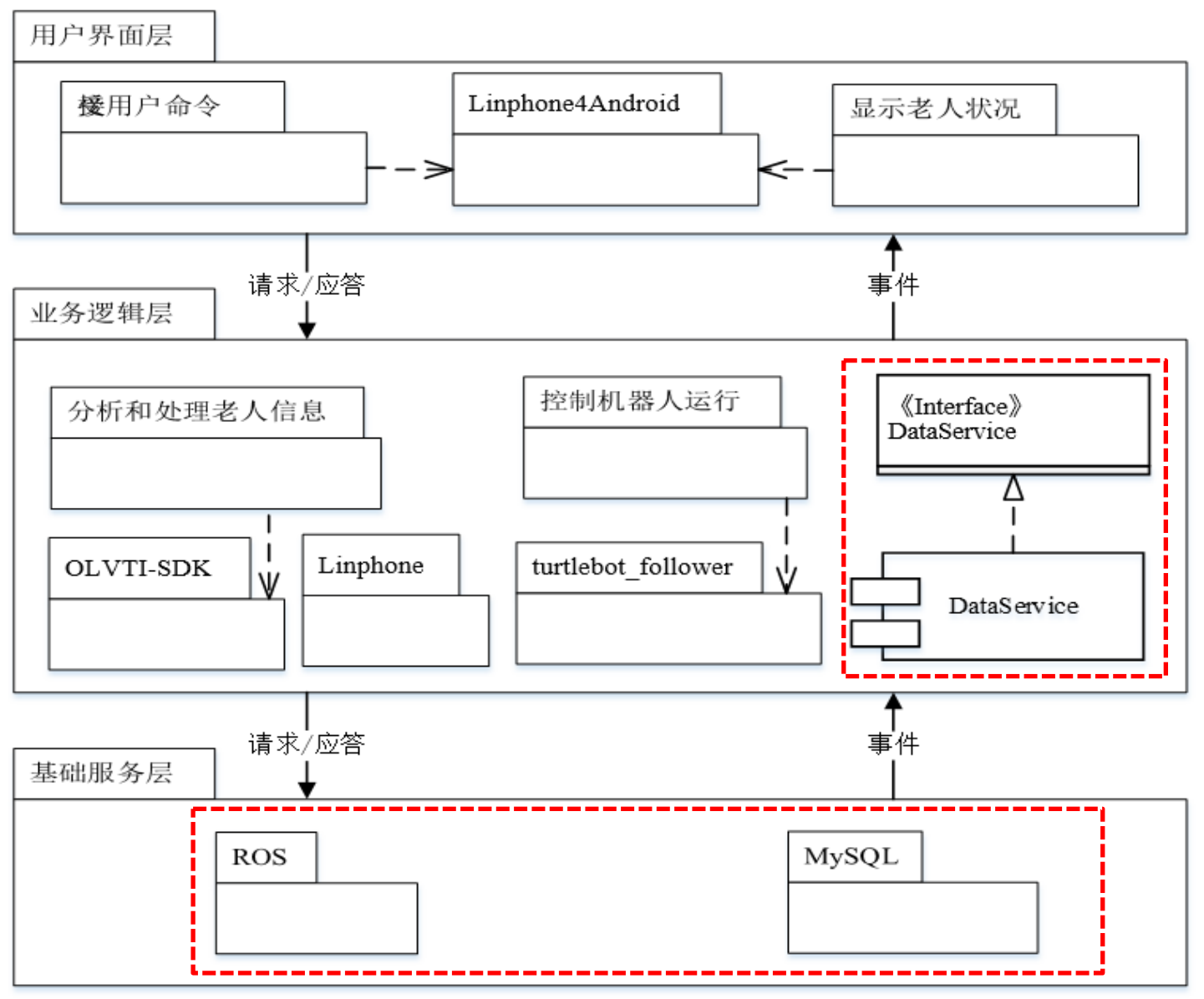
软件体系结构设计的过程



精化软件体系结构

- **选择软件体系结构所依赖的公共基础设施**
  - ✓确定其中的**基础性服务**，从而为目标软件系统的**运行**提供基础性的技术支撑，如**操作系统、软件中间件、数据库管理系统、软件开发框架、安全服务**等
- **确定软件体系结构中的设计元素（层次结构）**
  - ✓包括子系统、软构件和关键设计类等，**明确其职责和接口**，从而为开展详细设计奠定基础

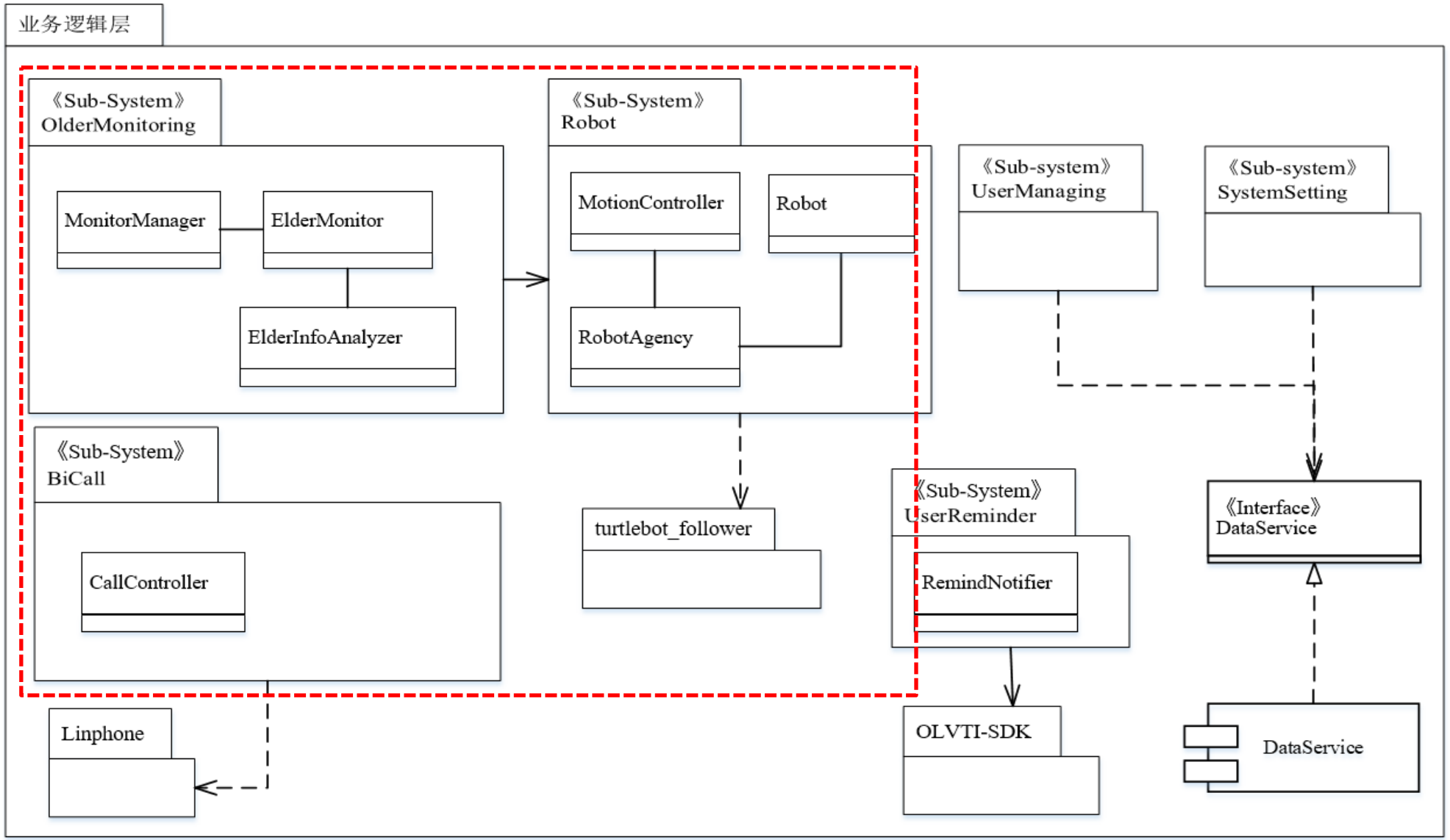
示例：“空巢老人看护软件”的基础设施



确定子系统及其接口

- **遵循问题分解和系统组织思想，将分析类按相关性原则加以组织和归类，形成软件系统的若干子系统**
- **面向对象方法：**
  - ✓ **用例相关性原则：**将用例按业务相关性或相似性进行分组，每组用例组成一个子系统，参与这些用例的实现的分析类均成为此子系统的“设计类”
  - ✓ **实现途径相关性原则**：分析用例的交互图，将具有相关或相似业务处理职责的控制类归为一个子系统；或将所有这些控制类的职责归并后按某种业务上的相关性或相似性进行分组，每组职责归为一个子系统
  - ✓ **实体类相关性：**从便于管理和控制的角度，将分析模型中的实体类进行分组，每组对应于一个子系统，由子系统对所属的实体类加以管理

示例： 确定“空巢老人智能看护系统”子系统



子系统

OlderMonitoring

Robot

BiCall

UserReminder

……

确定构件及其接口

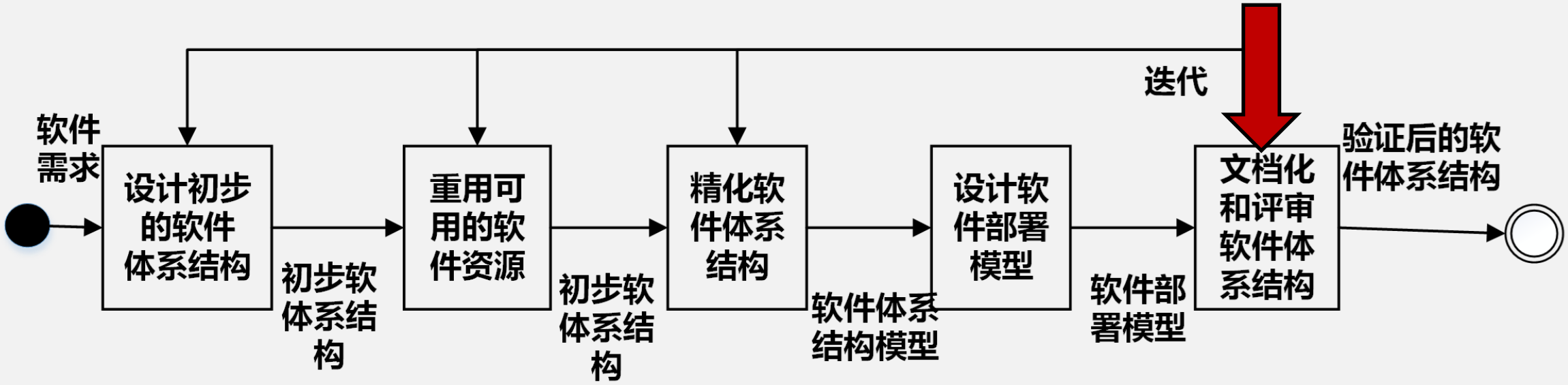
- **将一些分析类的相似或相关职责封装为一个构件，通过接口向软件系统的其他部分提供功能和服务**
  - ✓**构件**的内聚度应该高于**子系统**
  - ✓**构件**的设计应该追求**可复用性**目标

- **构件的规模小于子系统**
  - ✓子系统中可以包含构件，但构件中不会包含子系统

优化体系结构设计

- 分解一些规模较大的子系统
- 合并若干职责相同或相似的设计元素
- 以软件重用为目的适当的调整设计元素所封装的功能、承担的职责和对外提供的接口
- 用简洁的软件元素，以尽可能高效、合理的方式实现所有软件需求
- 将不同设计元素中的公共职责提取成为服务构件或服务类

软件体系结构设计的过程



撰写软件体系结构设计文档

1. 文档概述
2. 系统概述
3. 设计目标和原则
4. 设计约束和现实限制
5. 逻辑视点的体系结构设计（功能架构）
6. 开发视点的体系结构设计（技术架构）
7. 部署视点的体系结构设计（物理架构）
8. 运行视点的体系结构设计（运行架构）

评审软件体系结构设计

- **满足性**
  - ✓体系结构是否能够满足软件需求，体系结构怎样满足软件需求
- **优化性**
  - ✓体系结构是否以充分优化方式实现所有软件需求项
- **可扩展性**
  - ✓是否易于扩展，以应对软件需求的变化
- **可追踪性**
  - ✓软件体系结构中的所有设计元素是否有相对应的软件需求项
- **详尽程度**
  - ✓体系结构的详略程度是否恰当

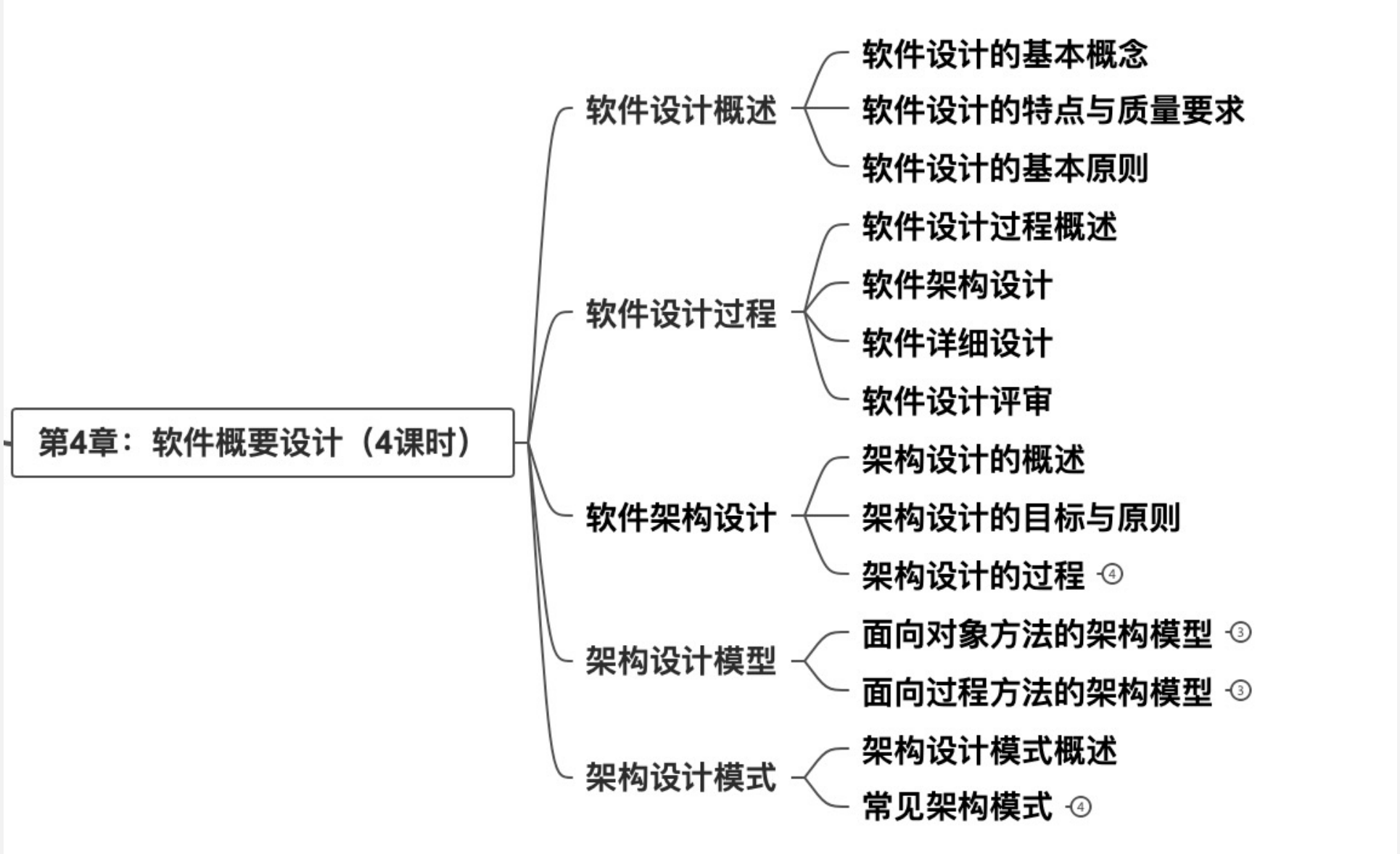
北京航空航天大學
BEIHANG UNIVERSITY



Part 4

架构设计模型

课时安排（回顾）：软件概要设计



主观题 10分

设置

思考和讨论：

软件体系结构设计的最大挑战是什么？

如何分析和评价软件体系结构设计的质量？



Answer

Part 4 架构设计模型

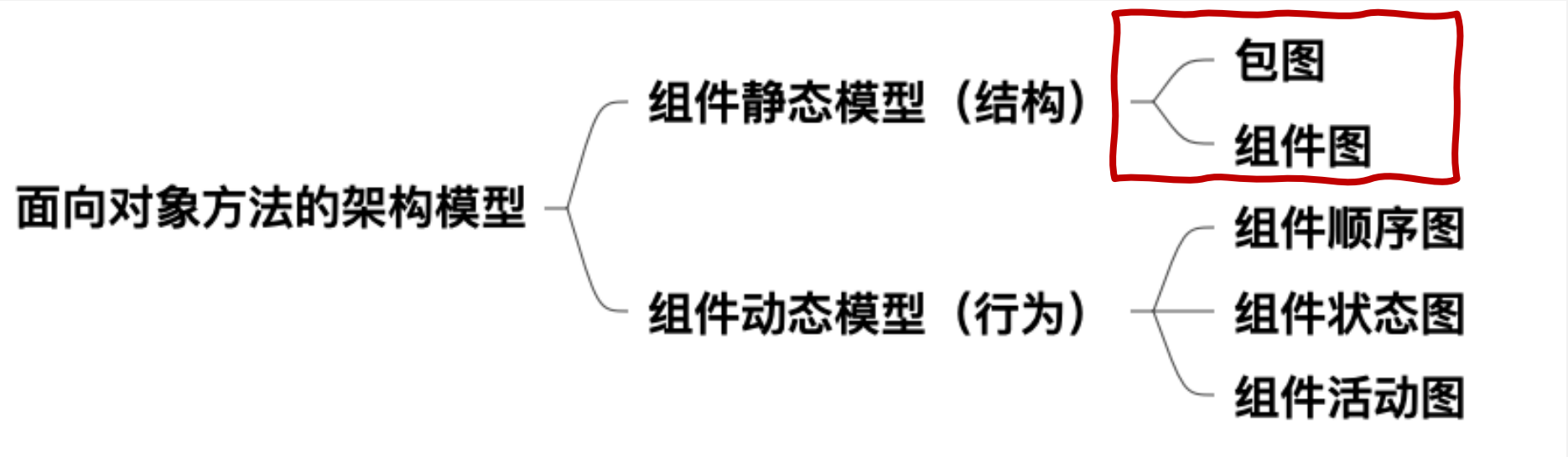
- 面向对象方法的架构模型
- 面向过程方法的架构模型

Part 4 架构设计模型

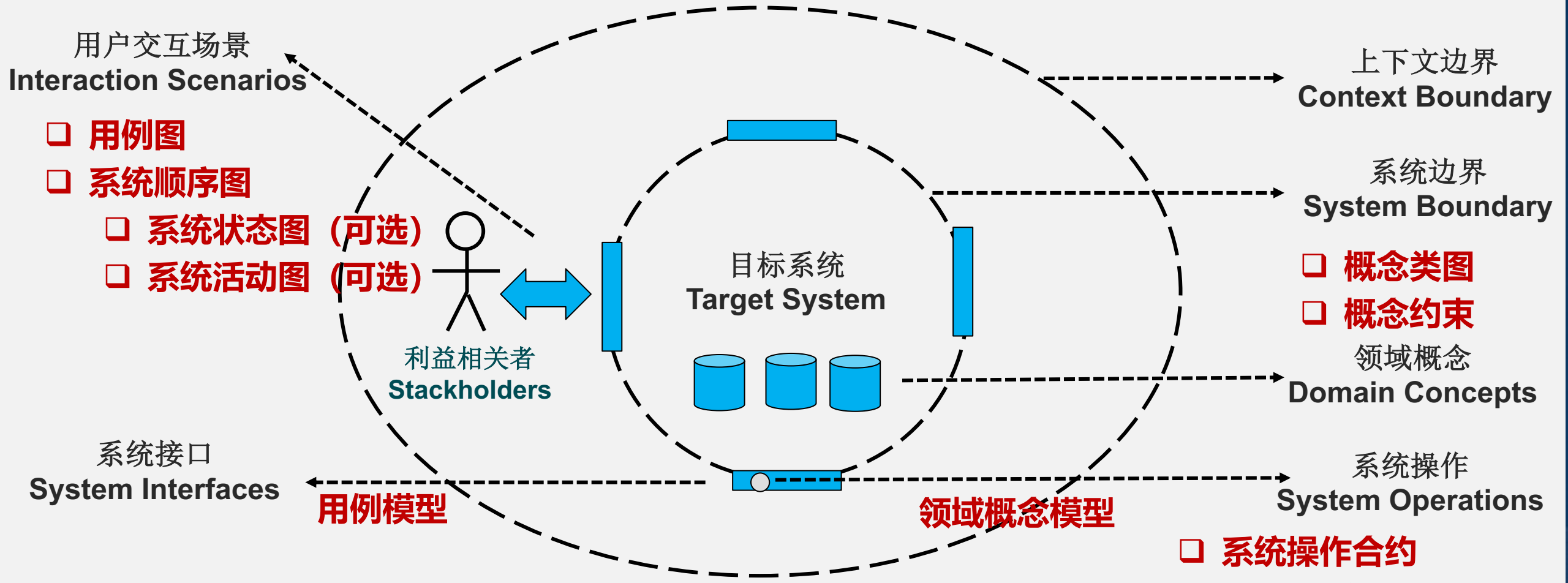
- 面向对象方法的架构模型
- 面向过程方法的架构模型

Part 4 架构设计模型

❑面向对象方法的架构模型



回顾：系统需求的建模


用户交互场景
Interaction Scenarios
❑ 用例图
❑ 系统顺序图
❑ 系统状态图（可选）
❑ 系统活动图（可选）
利益相关者
Stackholders
目标系统
Target System
上下文边界
Context Boundary
系统边界
System Boundary
❑ 概念类图
❑ 概念约束
领域概念
Domain Concepts
系统接口
System Interfaces
用例模型
领域概念模型
系统操作
System Operations
❑ 系统操作合约
❑ 一阶逻辑
❑ 对象约束语言

回顾：面向对象的需求模型

- 用例图
- 系统顺序图
  - 系统状态图（可选）
  - 系统活动图（可选）
- 系统操作合约
  - 一阶逻辑
  - 对象约束语言

用例模型
（行为）

- 概念类图
- 概念复杂约束

领域概念模型
（结构）

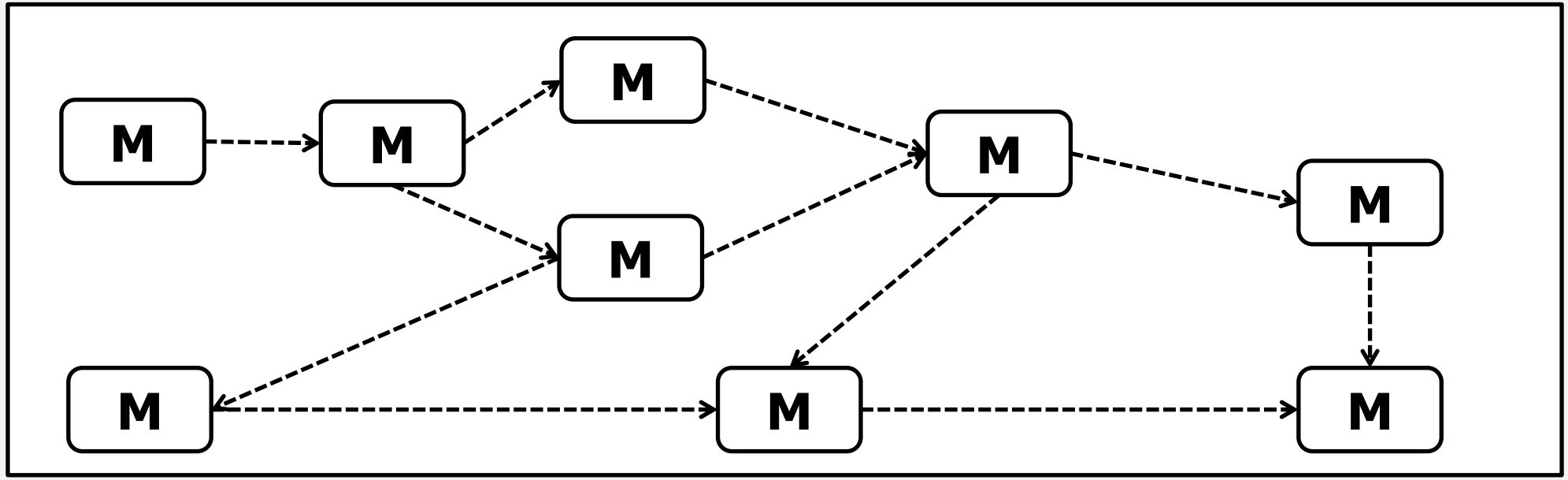


OO需求模型

软件体系结构的概念

❑软件体系结构(Software Architecture，SA)

✓也称软件架构，从**高层抽象**角度刻画组成软件系统的**设计元素**及它们之间的**逻辑关联**



软件体系结构如何体现了高层和抽象的特征？这里的设计元素是什么？



面向对象软件设计过程

需求分析

用例模型

概念模型

概要设计

组件模型

组件图（包图）+
组件顺序图（组件状态图）

详细设计

类模型

类图+顺序图（状态图）

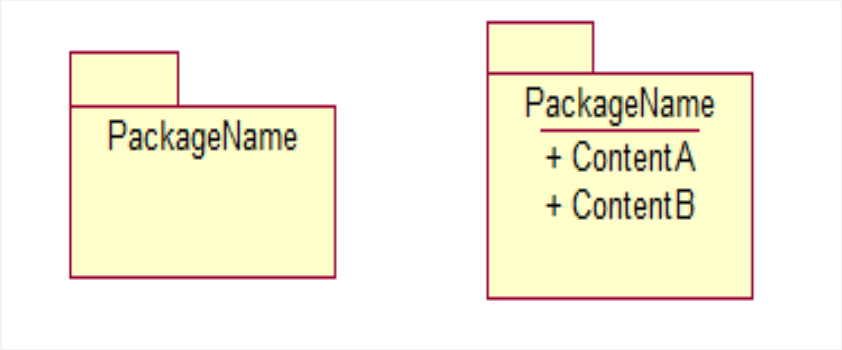
包图

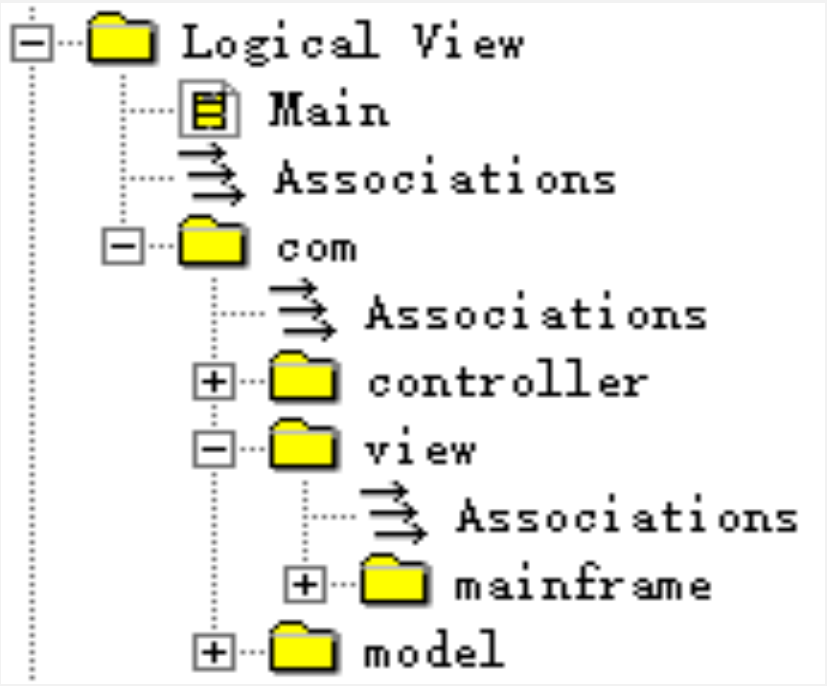
视点	图（diagram）	说明
结构	**包图（package diagram）**	**从包层面描述系统的静态结构**
	类图（class diagram）	从类层面描述系统的静态结构
	对象图（object diagram）	从对象层面描述系统的静态结构
	构件图(component diagram)	描述系统中构件及其依赖关系
行为	状态图(statechart diagram)	描述状态的变迁
	活动图(activity diagram)	描述系统活动的实施
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)	描述对象间的消息传递与协作
部署	部署图（deployment diagram）	描述系统中工件在物理运行环境中的部署情况
用例	用例图（use case diagram）	从外部用户角度描述系统功能

©Copyright Xinjun Mao

包图的基本概念

- ✓包是用于把模型本身组织成**层次结构**的通用机制，它不能执行。
- ✓包名：与类图相似，包有简单名与路径名两种命名方法。
- ✓包中的元素：**包中可以容纳各种高级的模型元素**，如类和类的关系、状态机、用例图、交互、协作等，甚至是一个完整的UML图。
- ✓另外，**包中还可以含有包，这被称为包的嵌套。**





软件体系结构的逻辑视图：包图

❑包模型

- ✓刻画**包间**的**构成和依赖**关系
- ✓包可以包含一组具有逻辑关联的UML模型元素
- ✓包间关系：**构成关系、依赖关系**

❑包图可描述软件系统的高层结构

- ✓包图以结构化、层次化方式组织、管理大型的软件模型，使得分别处理不同包的开发团队之间的相互干扰程度降至最低

包图：系统（子系统）、构件及关系

包的作用

- **作为软件模型的组织单元**
  - ✓将大型软件系统划分成不同包，以构建各类模型
- **作为模型管理的基本单元**
  - ✓以包为单位分派开发任务安排计划，包是天然的基本处理单元
- **作为系统高层结构中的组成元素。**
  - ✓逐级细分的包才是软件高层结构中恰当的组成元素
- **作为访问控制的基本手段**
  - ✓将包视为名字空间，每个模型元素可通过在其名称之前拼接包名，构成全系统范围内的唯一限定名来指称它，正如在文件系统中以文件的全路径名来唯一地指称一个文件

包的依赖关系

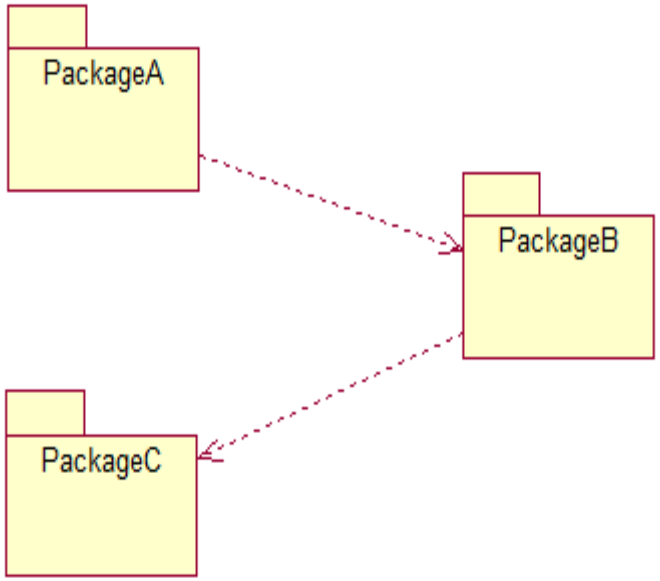
- ✓包之间的依赖关系实际上是从一个更高的层次来描述包内某些元素之间的依赖关系。也就是说，**如果不同包中任何元素之间存在着一个依赖，则两个包之间就存在着依赖关系**。
- ✓包之间的**依赖关系**首先需要包中的某些元素具有某种**外部可见性，即可以被包外部的元素所引用**。
- ✓包之间的依赖关系没有传递性。
- ✓包之间有**依赖关系，也就存在了耦合关系**。好的设计要求体现**高内聚、低耦合**的特性。



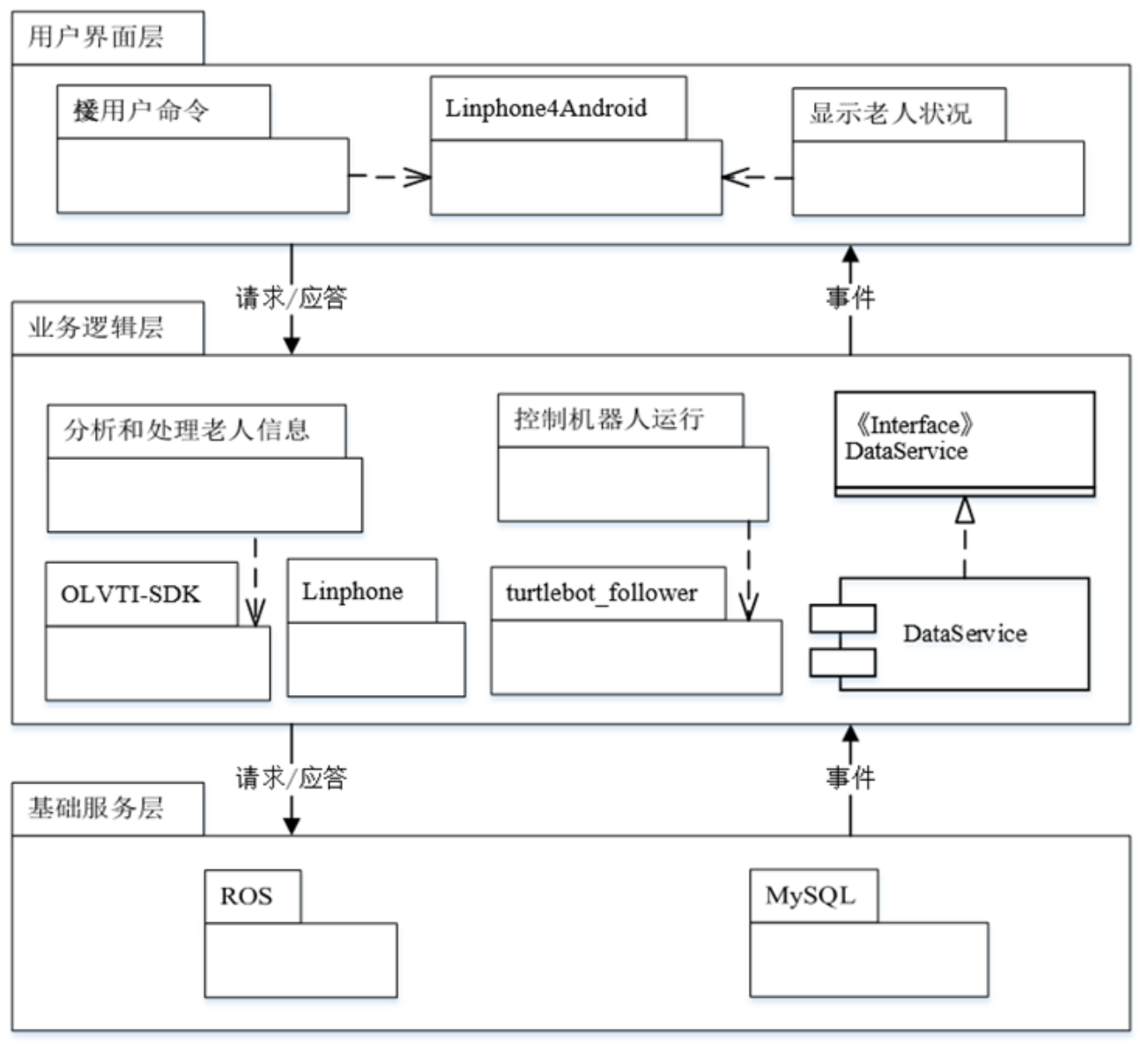
包的依赖关系

- ✓**容易出现的问题：循环依赖**
- ✓循环依赖的出现是令人困惑、也是非常容易产生错误的。尤其是当依赖关系表示包的引入时，循环依赖会导致将模型转化成代码后因为包之间互相引入而出现错误。
- ✓**解决方案：重新分包**，引入第三个包，重新建立依赖关系。





示例：软件体系结构的包图表示



包图表示软件体系结构

构件图

视点	图（diagram）	说明
结构	包图（package diagram）	从包层面描述系统的静态结构
	类图（class diagram）	从类层面描述系统的静态结构
	对象图（object diagram）	从对象层面描述系统的静态结构
	组件图(component diagram)	**描述系统中组件及其依赖关系**
行为	状态图(statechart diagram)	描述状态的变迁
	活动图(activity diagram)	描述系统活动的实施
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)	描述对象间的消息传递与协作
部署	部署图（deployment diagram）	描述系统中工件在物理运行环境中的部署情况
用例	用例图（use case diagram）	从外部用户角度描述系统功能

组件

❑何为组件

- ✓软件系统中的**模块**，具有特定的功能和精确定义的对外接口，外界可通过接口来访问它

❑特点

- ✓**可分离**：一个或数个可独立部署执行码文件
- ✓**可替换**：构件实例可被其它任何实现了相同接口的另一构件实例所替换
- ✓**可配置**：可通过配置机制修改构件配置数据，影响构件对外服务的功能或行为
- ✓**可复用**：构件可不经源代码修改，无需重新编译，即可应用于多个软件项目或软件产品

组件及其接口

❑构件

- ✓一个或者多个可独立部署的**执行码文件**
- ✓具有精确定义的**供给接口和需求接口**
- ✓**可分离**，接口和实现分离
- ✓**可替换**，构件实例可被任何实现相同接口的同一构件实例所替换

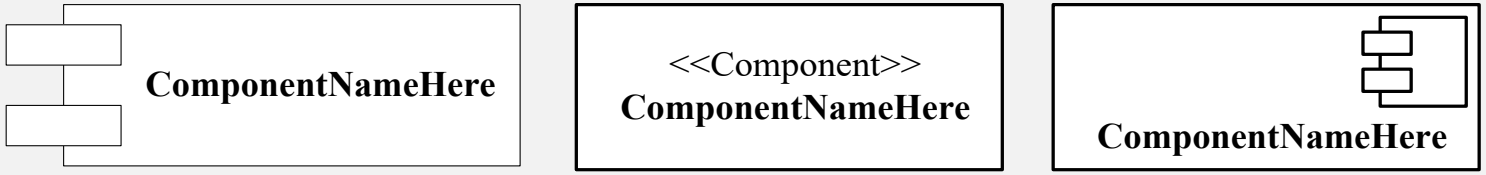
❑接口

- ✓一组操作 和/或 属性的说明（不含操作的实现），它用作服务提供方和使用方之间的协议
- ✓每个构件还可以定义一些端口（port），每个端口绑定了一组**供给接口和/或需求接口**
- ✓接口由类或构件实现

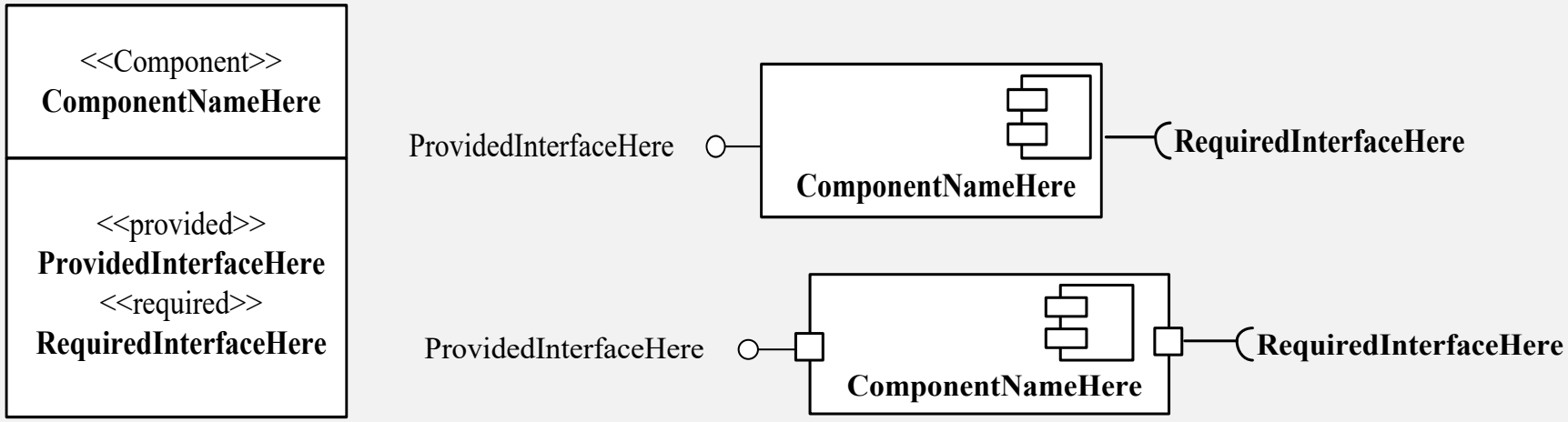
©Copyright Xinjun Mao

组件和接口的表示

❑组件的三种图元



❑组件及其接口的两种表示方法



软件体系结构的逻辑视图：组件图

❑描述软件系统中组件及组件间的关系

- ✓**结点**：组件、类和包
- ✓**关系**：构成关系、依赖关系

❑对软件体系结构的描述

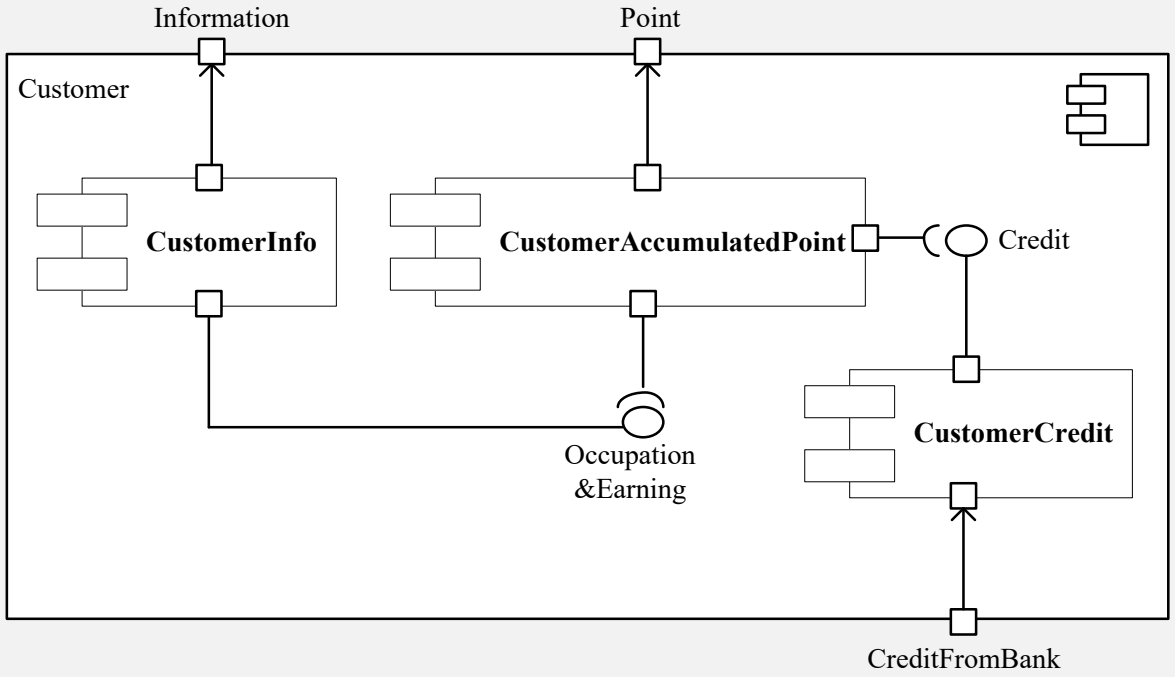
- ✓描述软件系统中**组件的接口定义及组件间的依赖关系**，以便评估软件变更的影响范围
- ✓描述软件系统或其中某个局部的**组件化设计**，为在后续开发阶段实现构件化的软件模块订立设计规约

连接和接口

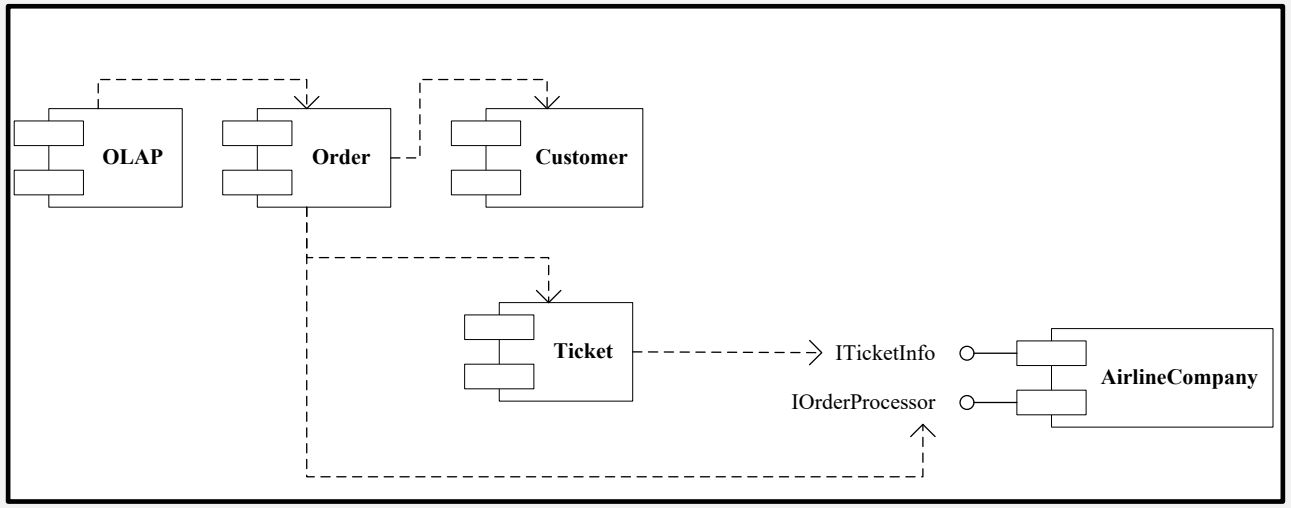
- **二种接口(Interface)**
  - ✓**供给接口**：对外提供的接口
  - ✓**需求接口**：请求其他构件帮助所需的接口
- **连接端口(Port)**
  - ✓每个**端口**绑定了一组供给接口和/或需求接口
  - ✓构件通过端口与外部世界交互
  - ✓当外部请求到达端口时，构件的端口知道如何将外部请求路由至合适的接口的**实现体**
  - ✓当构件通过端口请求外部服务时，端口也知道如何分辨该请求所对应的需求接口

- **构件通过接口对外提供服务**
- **构件通过接口与其他构件进行交互**

示例：软件体系结构的组件图表示



- 组件
- 接口和端口
- 关系(依赖)



MedShare Architecure - Component

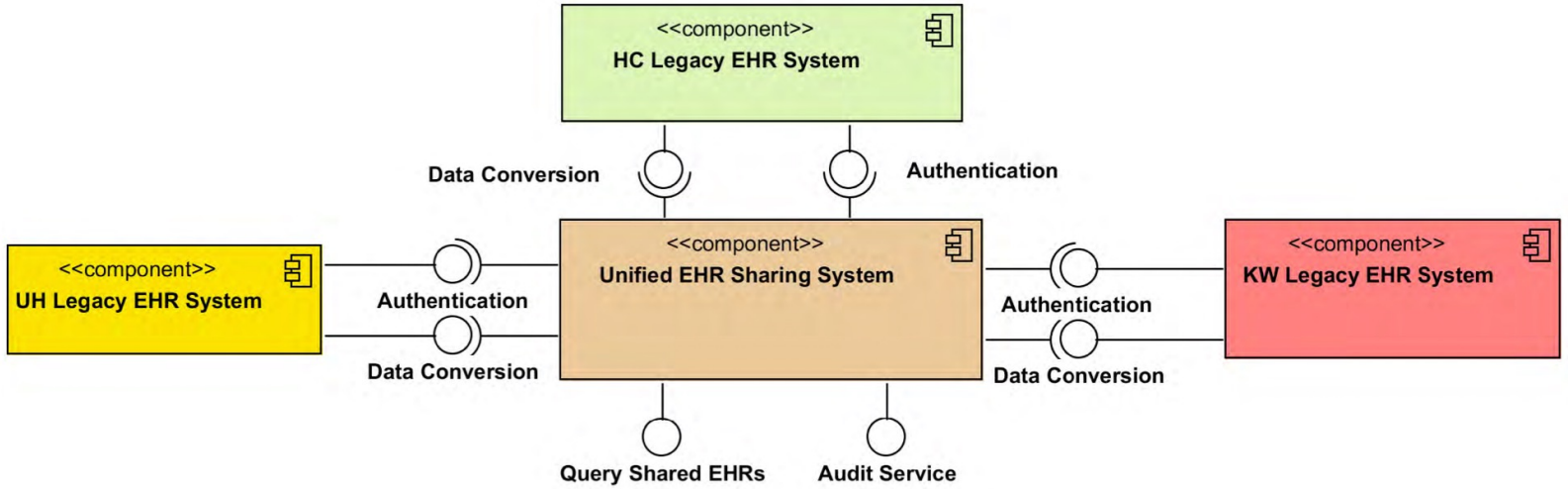


FIGURE 4. External view of distributed EHR sharing architecture.

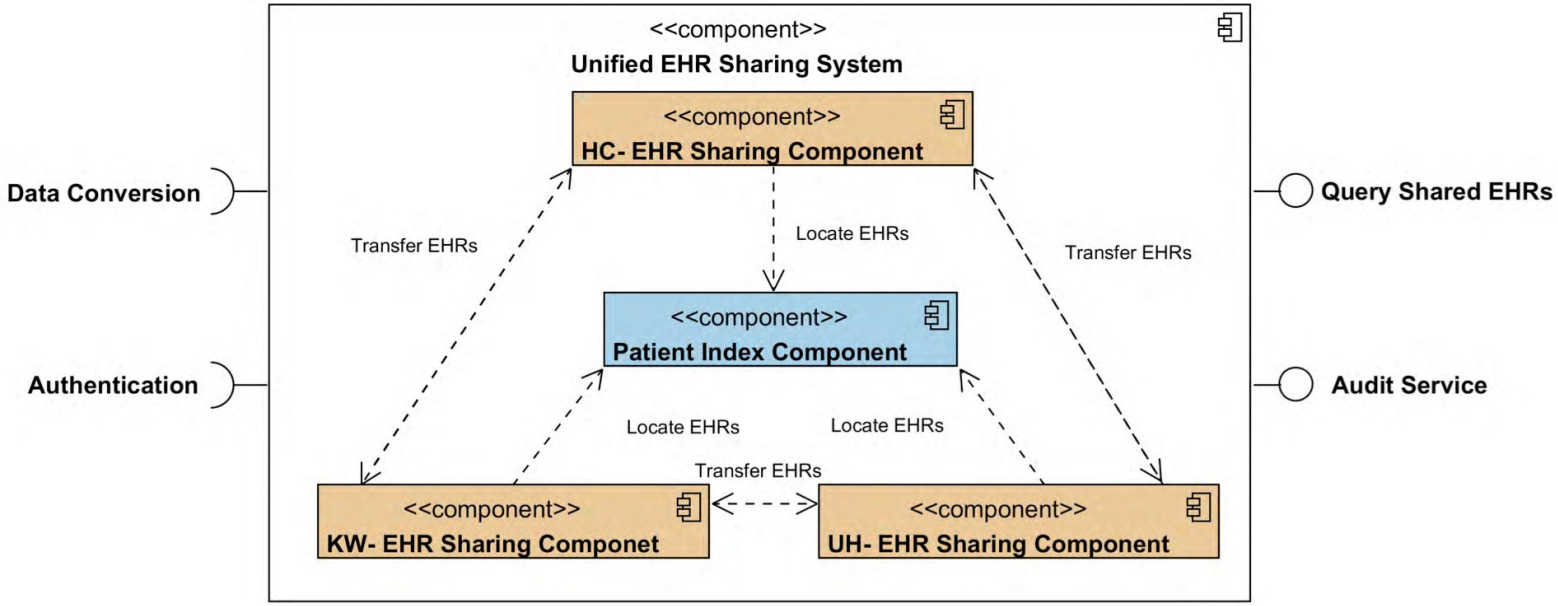
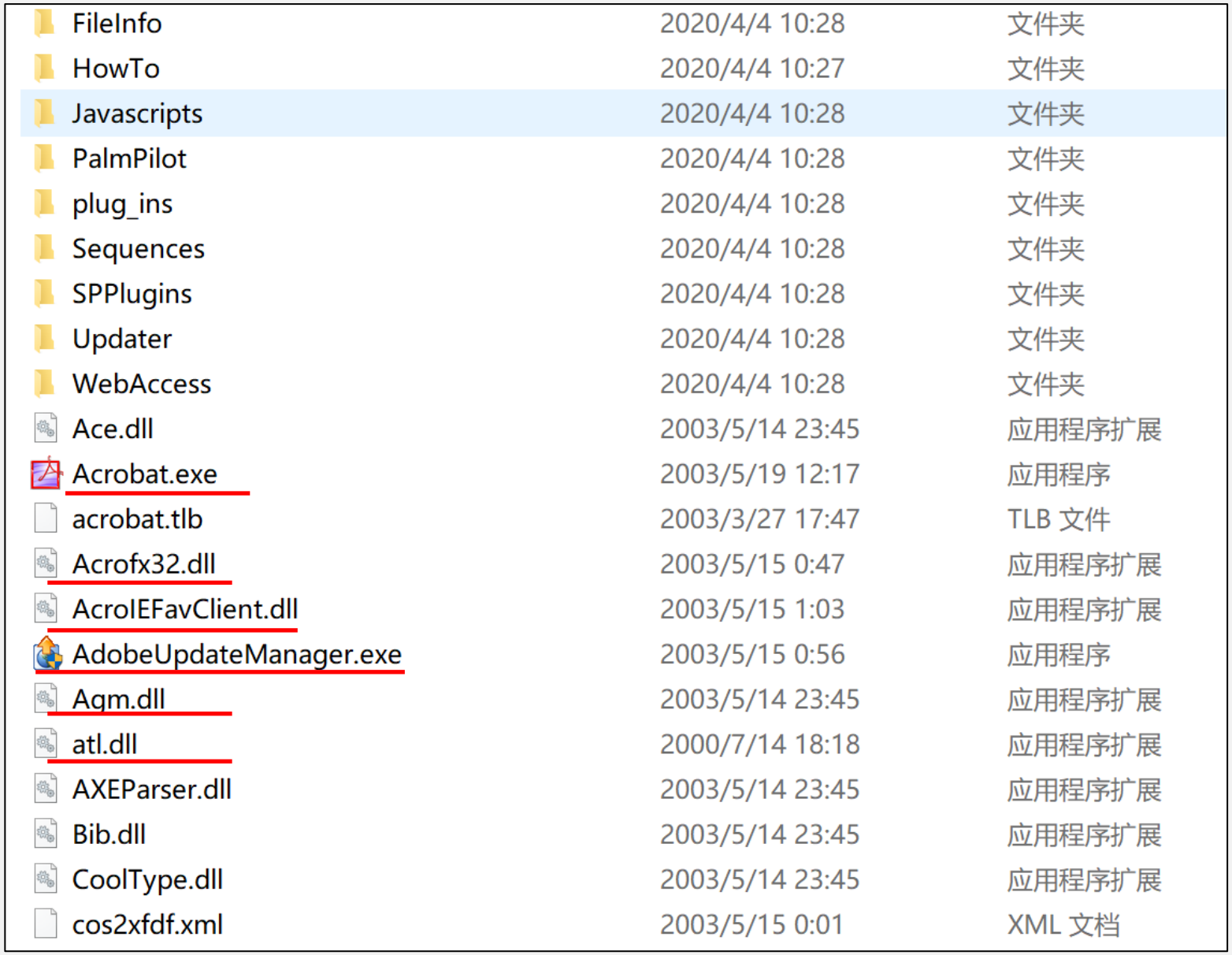


FIGURE 5. Internal view of distributed EHR sharing architecture.

示例：软件组件



FileInfo	2020/4/4 10:28	文件夹
HowTo	2020/4/4 10:27	文件夹
Javascripts	2020/4/4 10:28	文件夹
PalmPilot	2020/4/4 10:28	文件夹
plug_ins	2020/4/4 10:28	文件夹
Sequences	2020/4/4 10:28	文件夹
SPPlugins	2020/4/4 10:28	文件夹
Updater	2020/4/4 10:28	文件夹
WebAccess	2020/4/4 10:28	文件夹
Ace.dll	2003/5/14 23:45	应用程序扩展
Acrobat.exe	2003/5/19 12:17	应用程序
acrobat.tlb	2003/3/27 17:47	TLB 文件
Acrofx32.dll	2003/5/15 0:47	应用程序扩展
AcroIEFavClient.dll	2003/5/15 1:03	应用程序扩展
AdobeUpdateManager.exe	2003/5/15 0:56	应用程序
Agm.dll	2003/5/14 23:45	应用程序扩展
atl.dll	2000/7/14 18:18	应用程序扩展
AXEParser.dll	2003/5/14 23:45	应用程序扩展
Bib.dll	2003/5/14 23:45	应用程序扩展
CoolType.dll	2003/5/14 23:45	应用程序扩展
cos2xfdf.xml	2003/5/15 0:01	XML 文档

Acrobat软件中的组件

示例：组件

❑.dll文件



❑.jar文件

- 构件不是源程序，而是可运行的二进制代码
- 构件是客观物理存在的（即有实际的文件），而非是逻辑存在的
- 构件是可访问的，以获取其功能和服务
- 构件应该是粗粒度的，而非细粒度的

思考和讨论

- 举例和示范你计算机中安装的软件及其构件
- 对比软构件与源代码、软构件与.class之间的区别和联系
- 为什么要将软件组织为一组构件



主观题 10分

设置

思考和讨论：

如果一个组件没有对外的接口是否可以？没有接口的组件会带来什么问题?



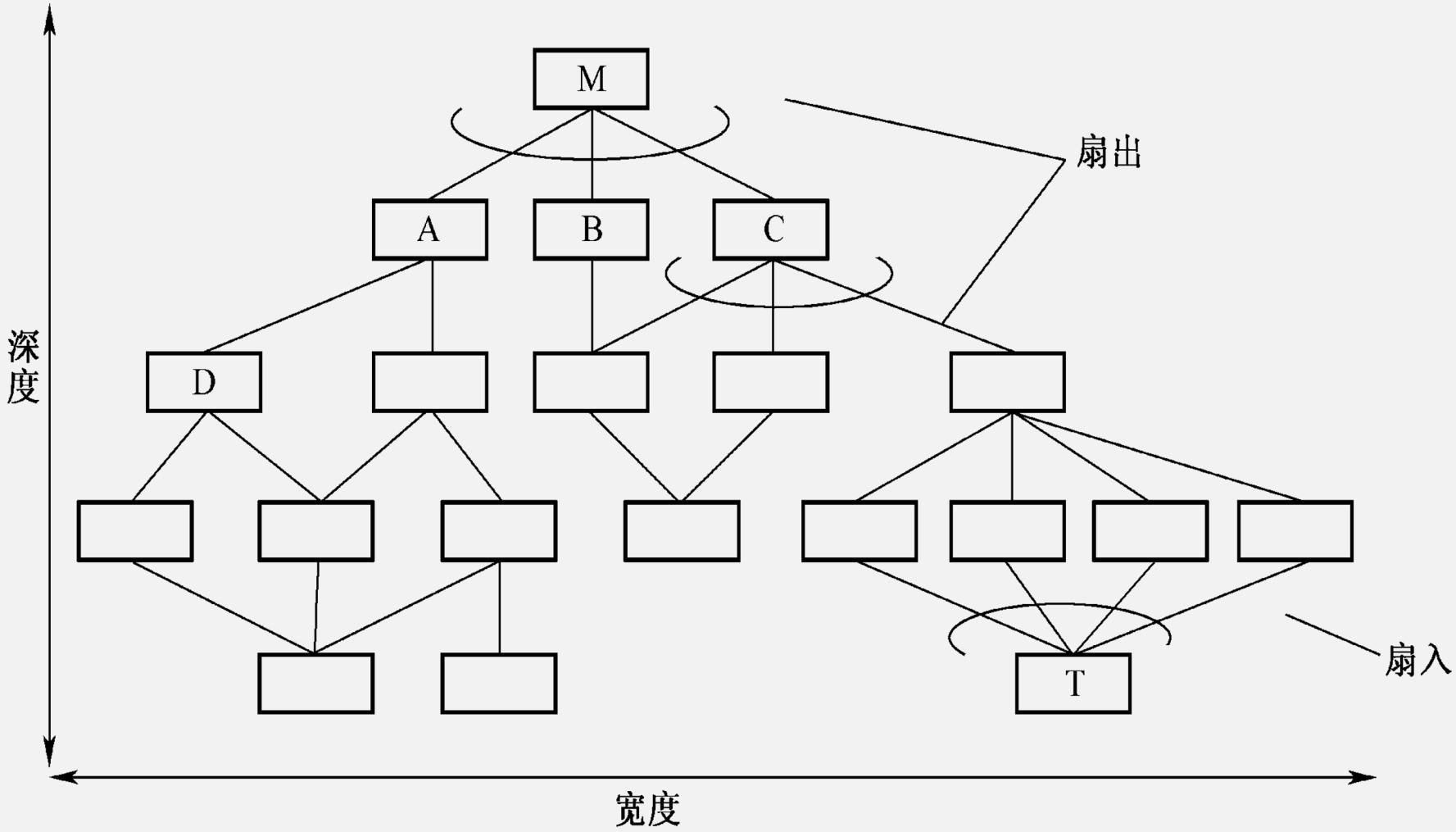
Answer

Part 4 架构设计模型

- 面向对象方法的架构模型
- 面向过程方法的架构模型



面向过程概要设计的主要内容

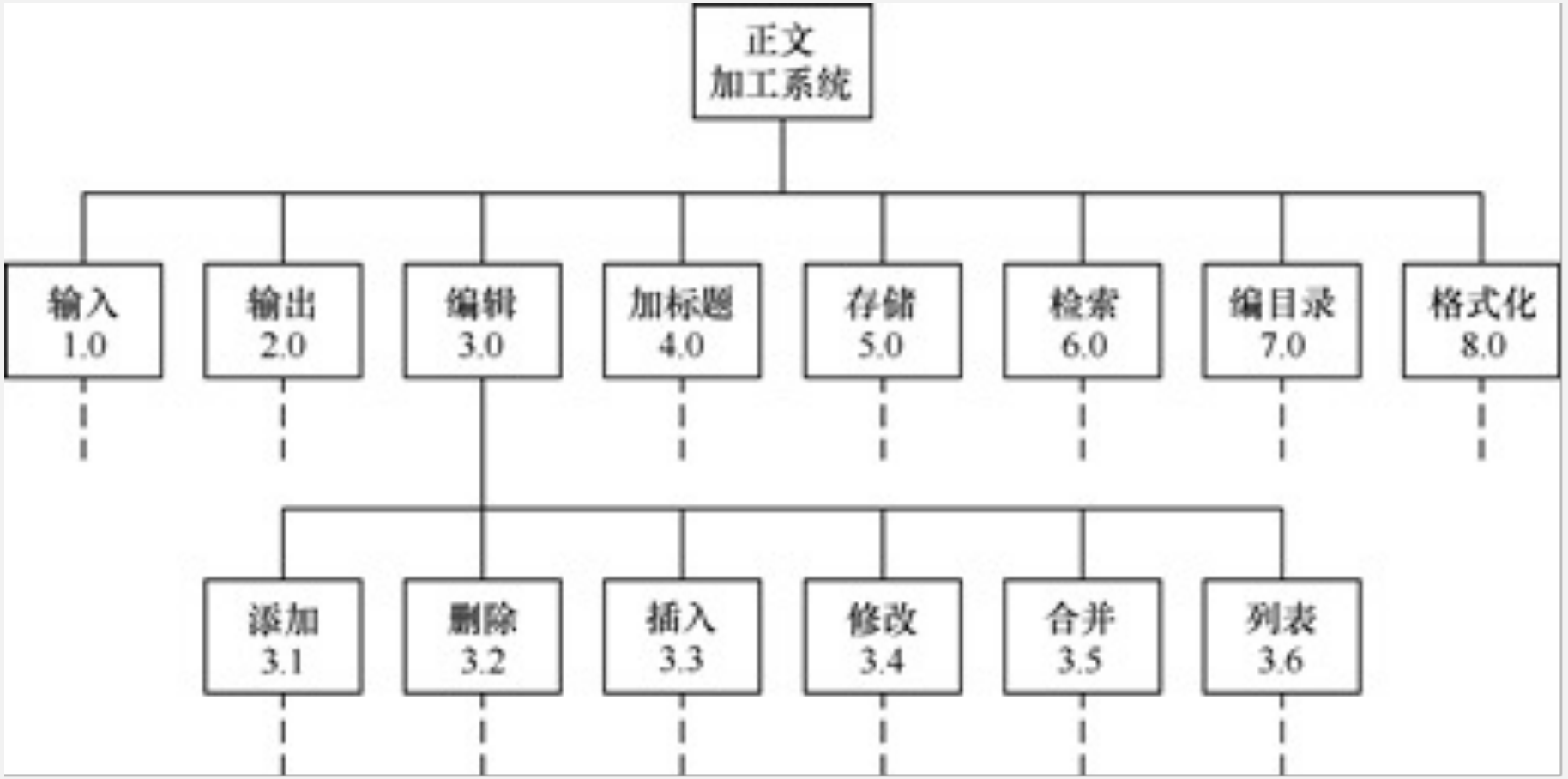


两种架构模型

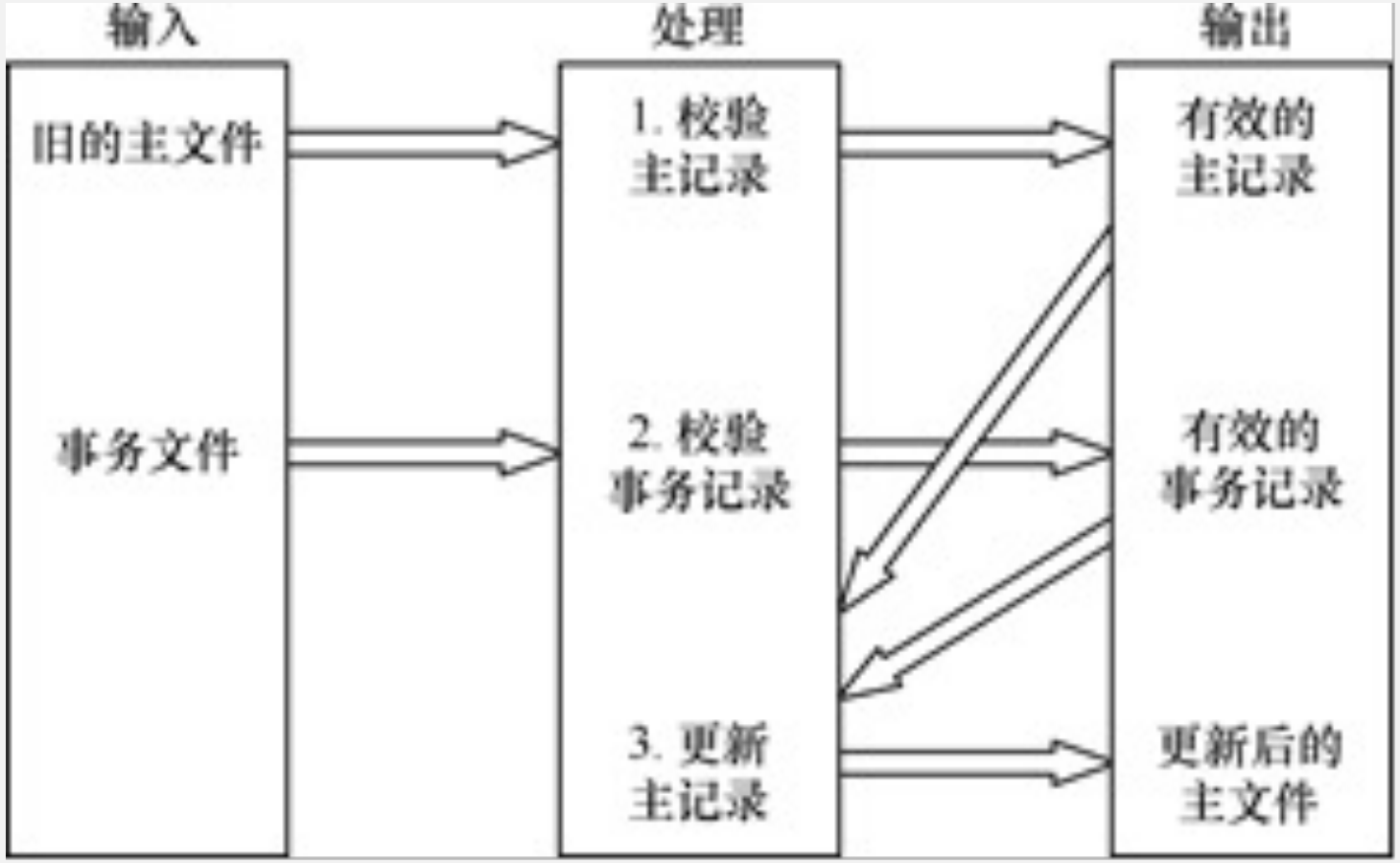
- ✓ **HIPO图**是由美国IBM公司发明的“**层次图+输入/处理/输出图**”的英文缩写，它是层次结构图的变种，是对层次结构图的补充。HIPO图实际上由**H图**和**IPO图**组成，**H图**就是上面提到的**层次结构图**，为了使HIPO图有**可跟踪性**，在H图里除了最顶层的方框之外，每个方框都**加了编号**。

- ✓ **层次结构图**是**软件概要设计阶段**最常使用的表示形式之一，用来描绘软件的层次结构，图中的**每个方框代表一个模块**，方框间的**连线表示模块的调用关系**，层次结构图适合于在“**自顶向下、逐步求精**” 面向过程设计中使用

H图

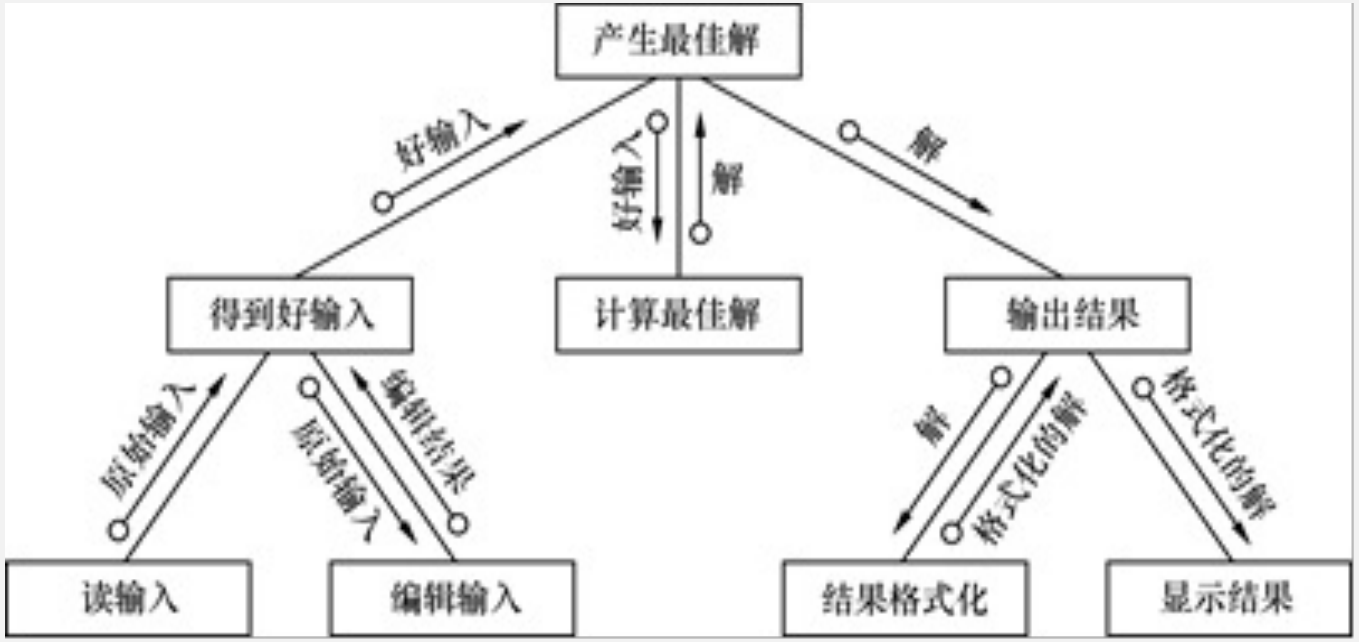

正文
加工系统
输入
1.0
输出
2.0
编辑
3.0
加标题
4.0
存储
5.0
检索
6.0
编目录
7.0
格式化
8.0
添加
3.1
删除
3.2
插入
3.3
修改
3.4
合并
3.5
列表
3.6

IPO图


输入
处理
输出
旧的主文件
事务文件
1. 校验
主记录
2. 校验
事务记录
3. 更新
主记录
有效的
主记录
有效的
事务记录
更新后的
主文件

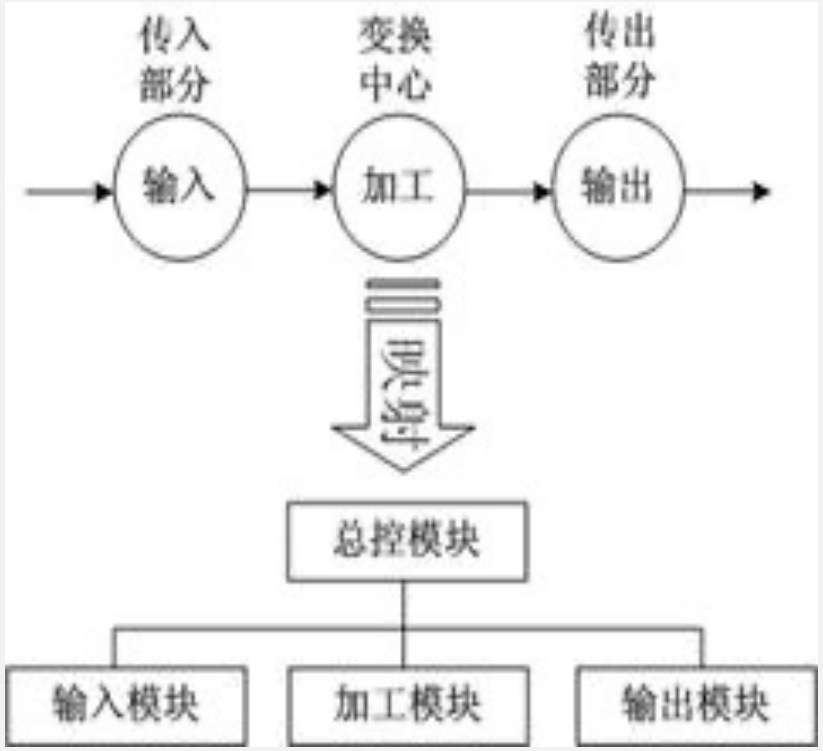
结构图

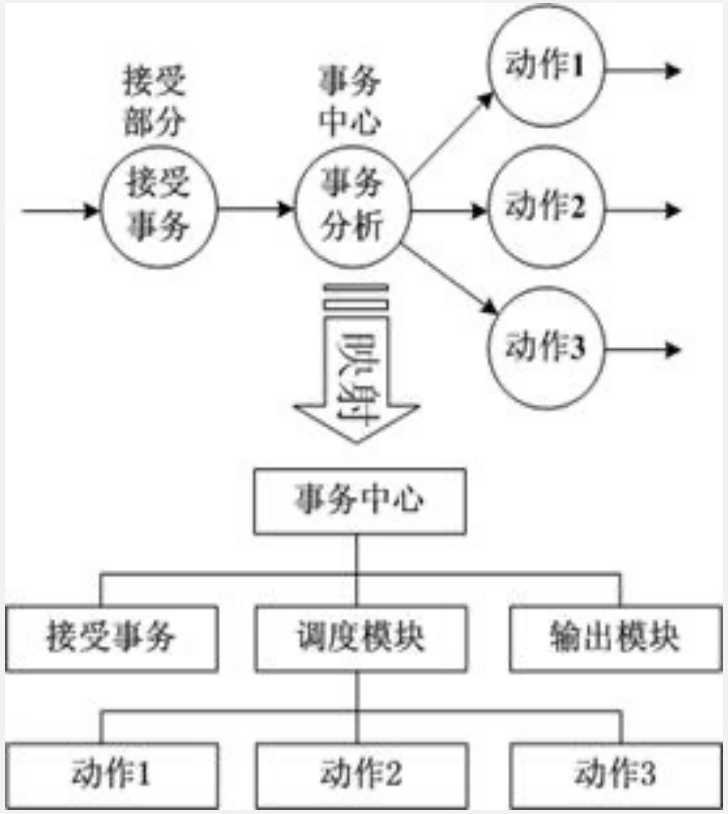
- 在结构图中通常还用**带注释的箭头**表示**模块调用过程**中来回**传递的信息**。如果希望进一步标明传递的信息是数据还是控制信息，则可以利用注释箭头尾部的形状来区分：**尾部是空心圆**表示**传递的是数据**，**实心圆**表示传递的是**控制信息**。



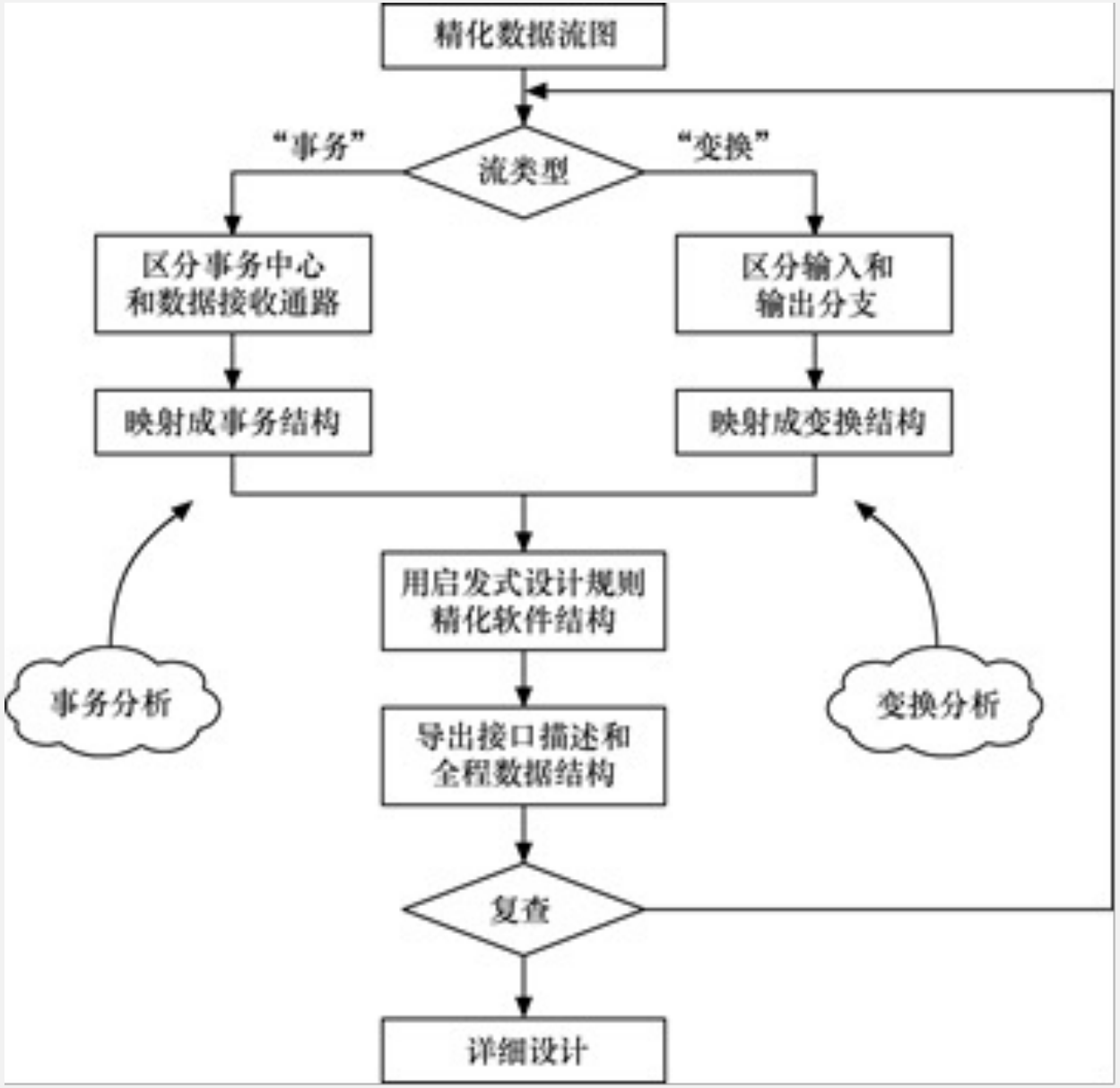
面向数据流的设计方法

- 例如：在结构化设计过程中，要将**数据流图(DFD)映射成系统结构图(SC)**，分别画出**变换型数据流**和**事务型数据流**的映射方式。

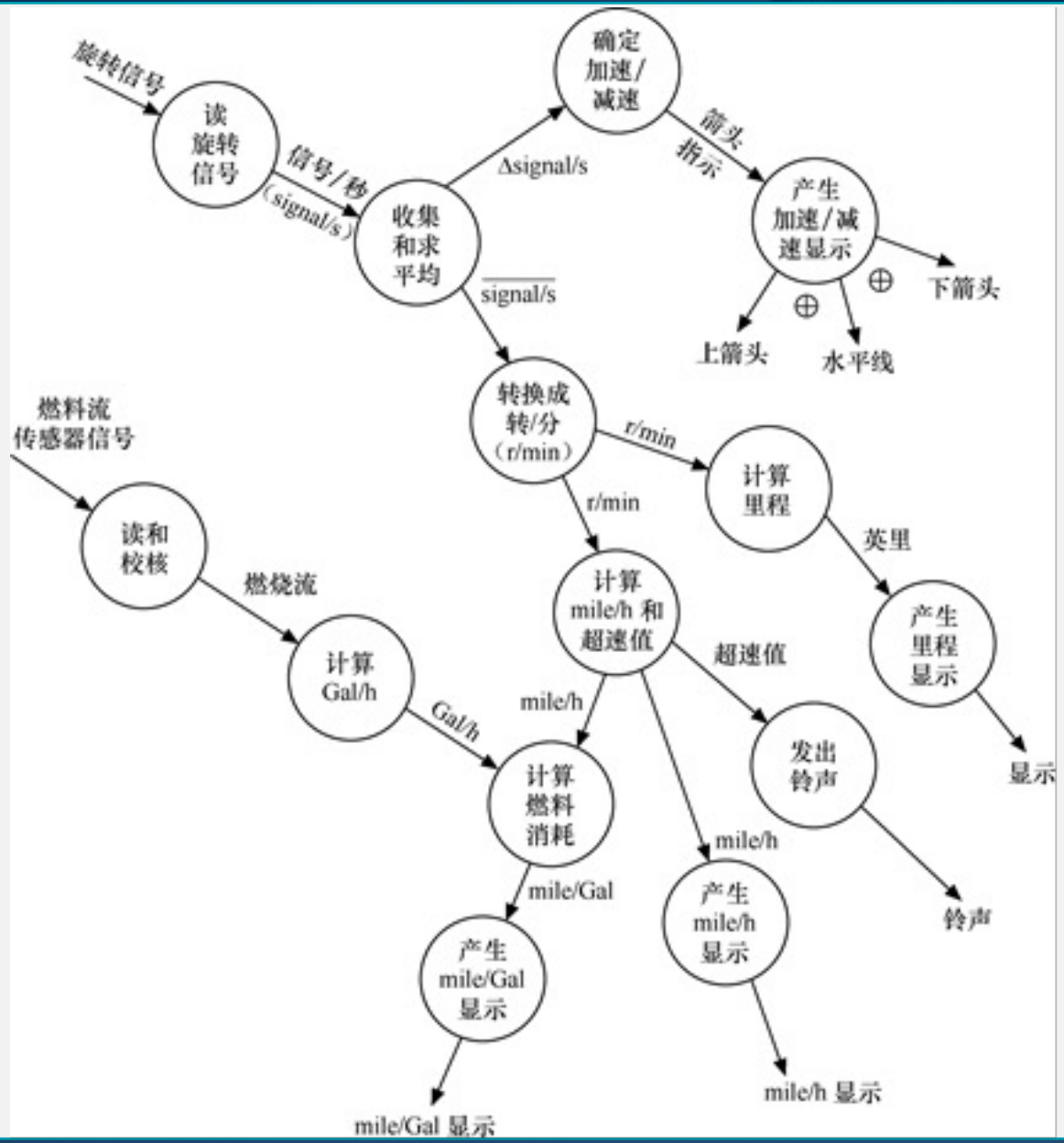


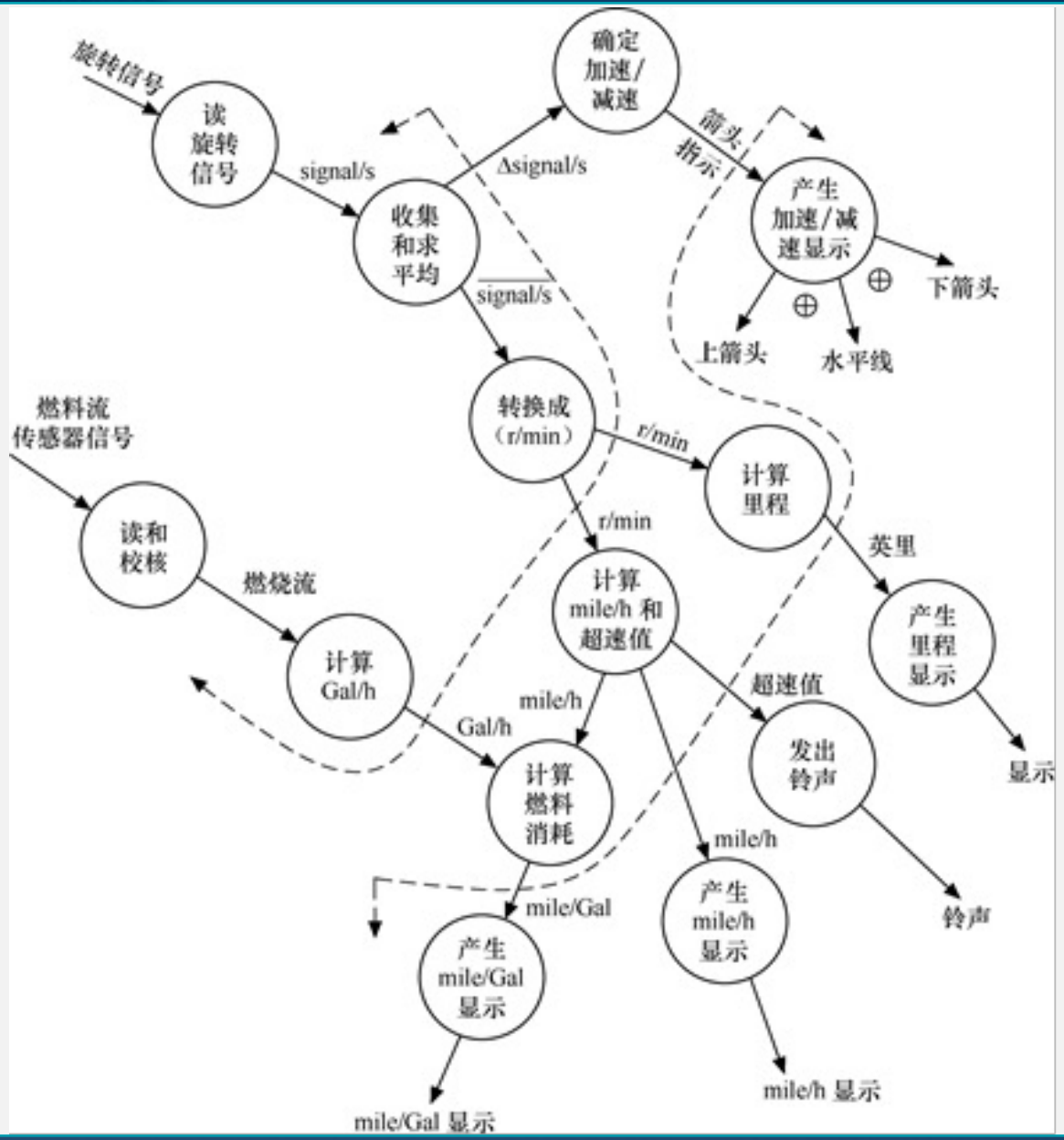


面向数据流方法的设计过程

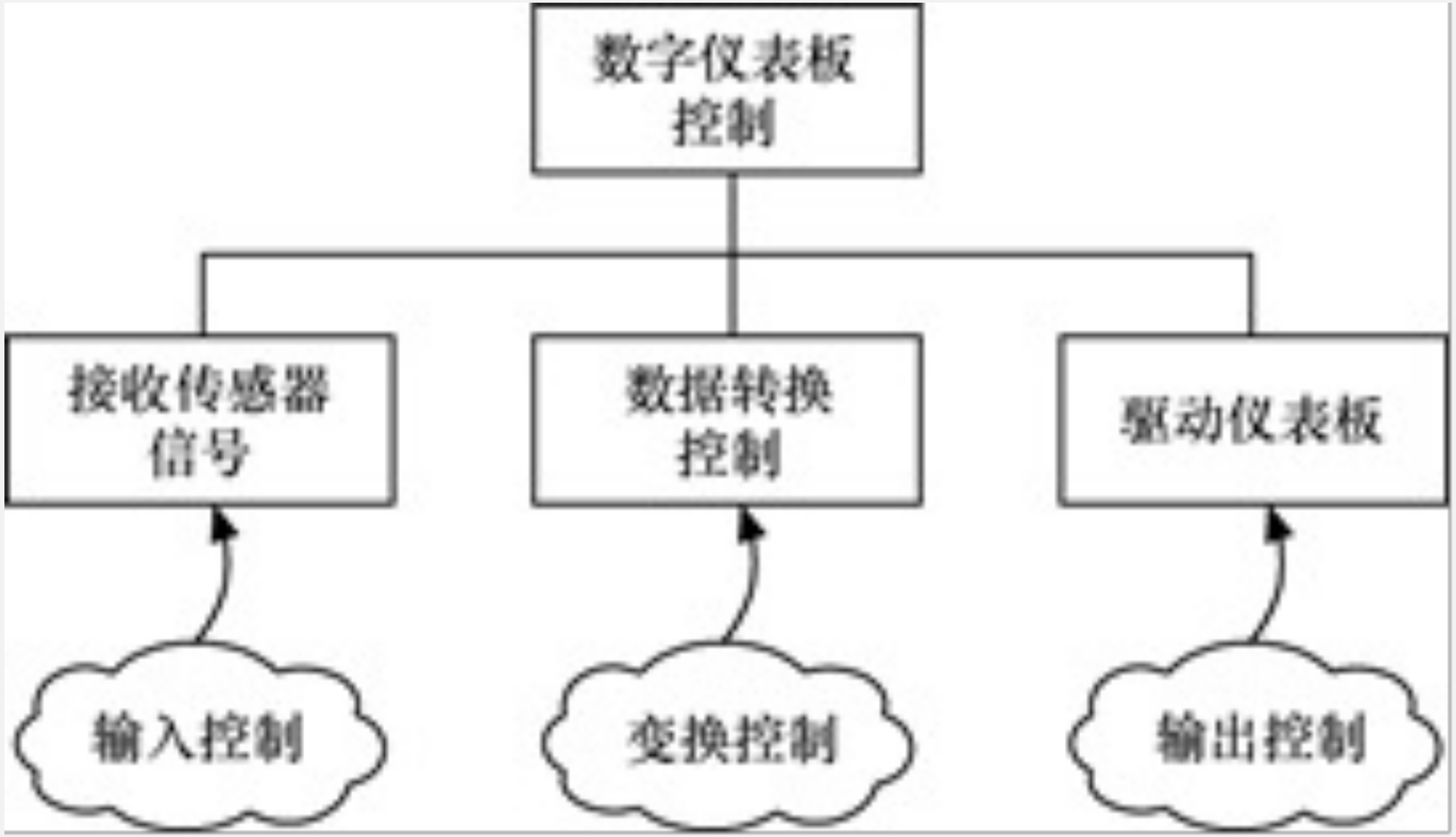


变换分析

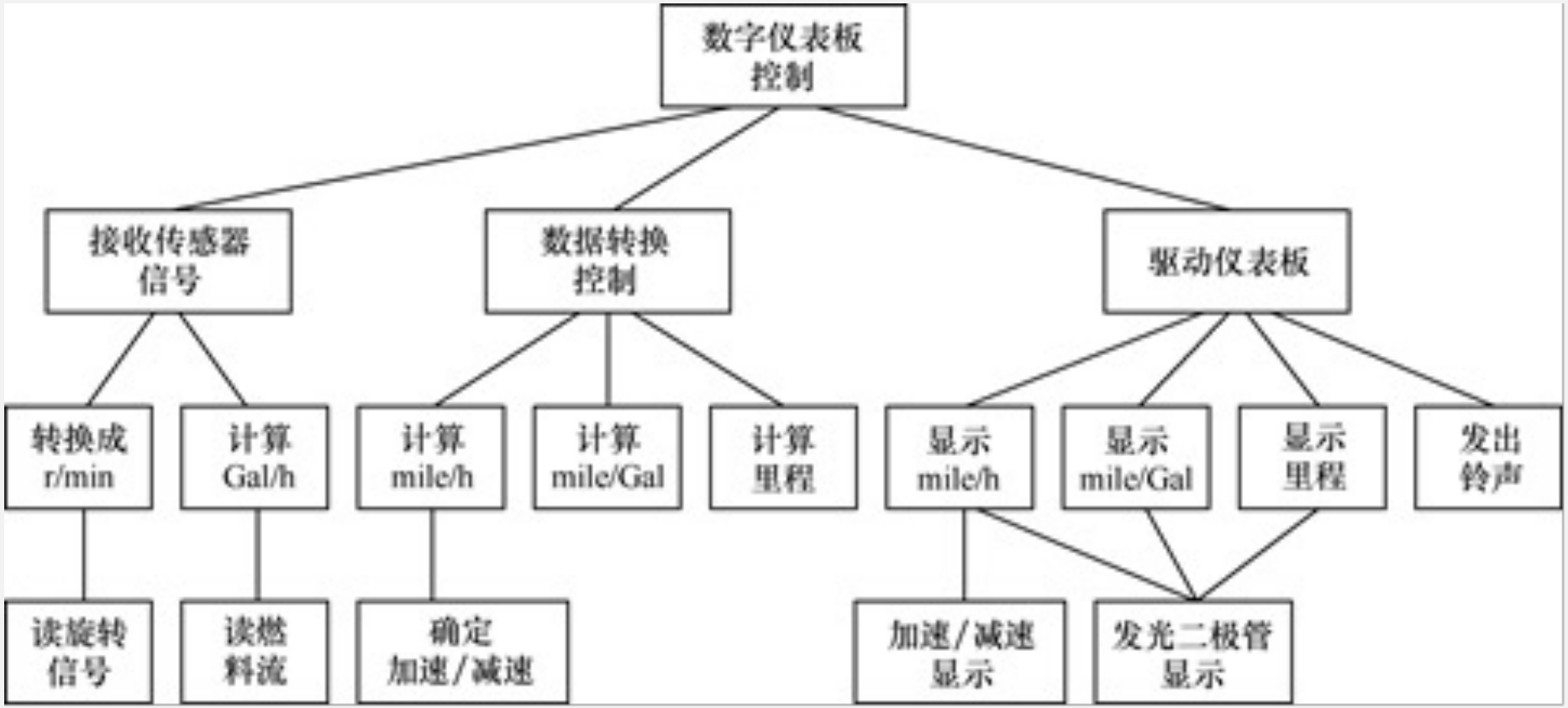

旋转信号
读旋转信号
信号/秒（signal/s）
收集和求平均
Δsignal/s
确定加速/减速
箭头指示
产生加速/减速显示
下箭头
上箭头
水平线
signal/s
转换成转/分（r/min）
r/min
计算里程
英里
产生里程显示
显示
燃料流传感器信号
读和校核
燃烧流
计算Gal/h
Gal/h
计算mile/h和超速值
mile/h
超速值
发出铃声
铃声
计算燃料消耗
mile/Gal
产生mile/Gal显示
mile/Gal 显示
产生mile/h显示
mile/h
mile/h 显示


旋转信号
读旋转信号
signal/s
收集和求平均
Δsignal/s
确定加速/减速
箭头指示
产生加速/减速显示
下箭头
上箭头
水平线
signal/s
转换成（r/min）
r/min
计算里程
英里
产生里程显示
显示
燃料流传感器信号
读和校核
燃烧流
计算Gal/h
Gal/h
计算mile/h和超速值
mile/h
超速值
发出铃声
铃声
计算燃料消耗
mile/Gal
产生mile/Gal显示
mile/Gal 显示
产生mile/h显示
mile/h
mile/h 显示

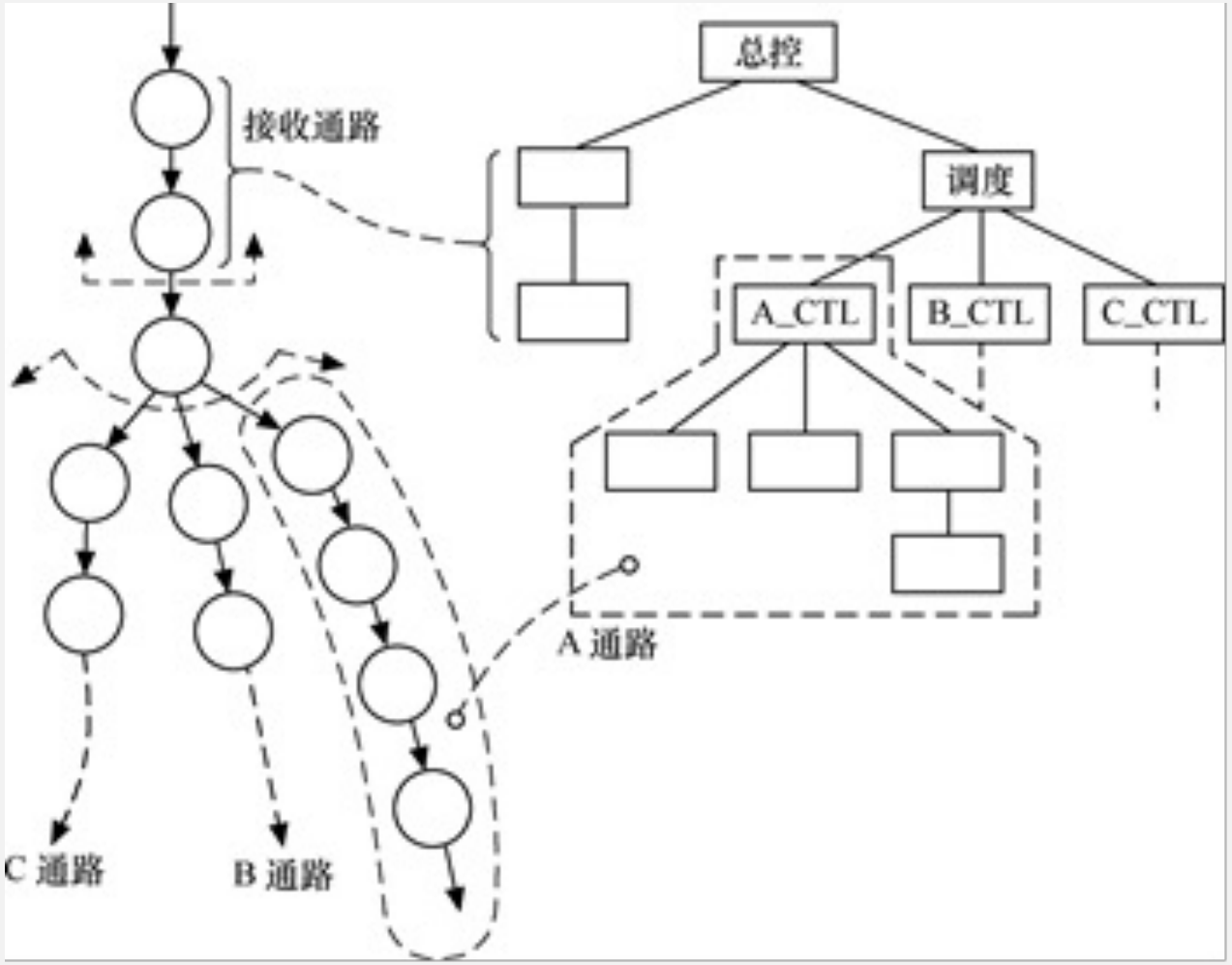
变换分析



变换分析


数字仪表板
控制
接收传感器
信号
数据转换
控制
驱动仪表板
转换成
r/min
计算
Gal/h
计算
mile/h
计算
mile/Gal
计算
里程
显示
mile/h
显示
mile/Gal
显示
里程
发出
铃声
读旋转
信号
读燃
料流
确定
加速/减速
加速/减速
显示
发光二极管
显示

事物分析



北京航空航天大學
BEIHANG UNIVERSITY



Part 5

软件架构设计模式

建筑风格：传统中式建筑



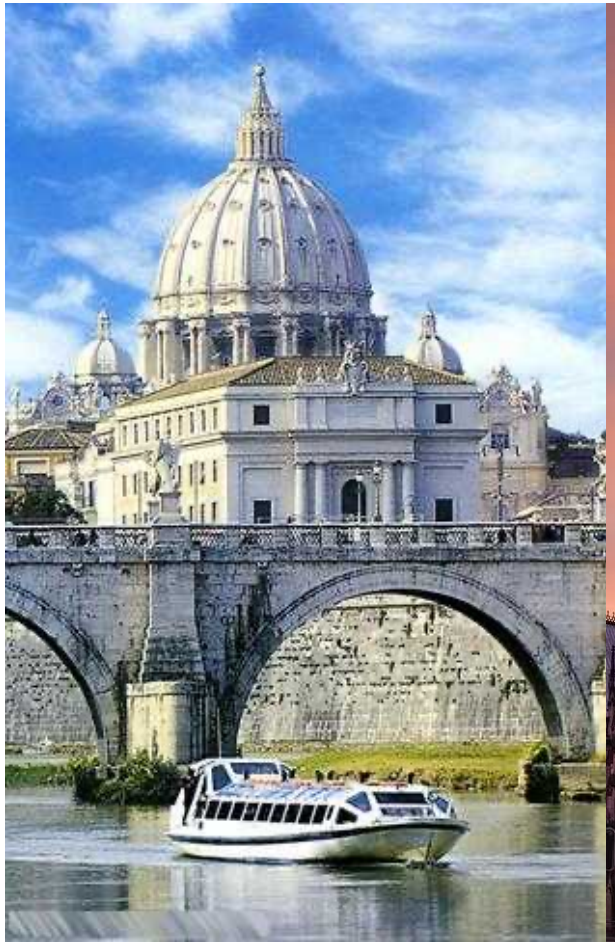




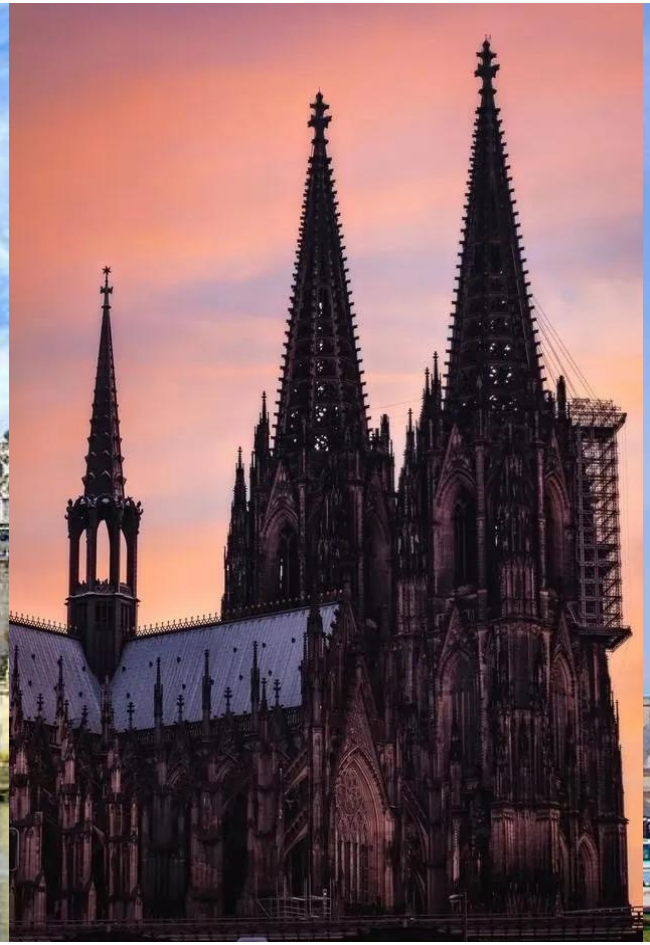


徽式、京式、晋式、苏式

建筑风格：传统西方建筑



罗马式



哥特式



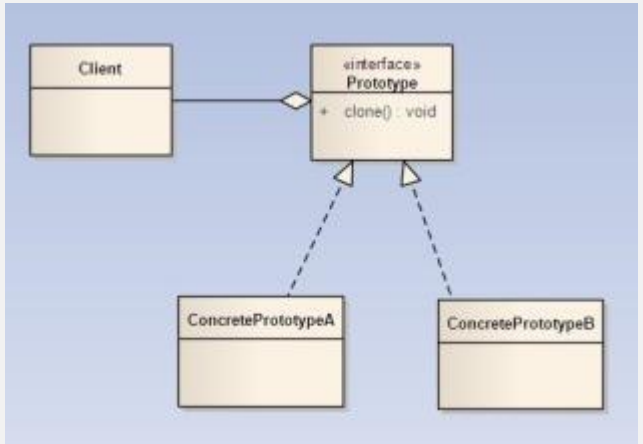
巴洛克式

软件体系结构

- **建筑风格：中国古代建筑风格、西方建筑风格**
- **程序设计样式、设计模式、体系结构风格**
- **软件体系结构 (风格 style)：**
  - ✓ 描述某一特定应用领域中系统组织方式的惯用模式
  - ✓ 反映领域中众多系统所共有的结构和语义特性
  - ✓ 指导如何将各个模块和子系统有效地组织成完整的系统

软件设计模式

- 借用以往的**经验来解决问题**
- 以**设计重用**为目的，采用一种良定义的、正规的、一致的方式记录的软件设计经验
- 关注在一般或特定设计环境中可能重复出现的设计问题，并给出经过**充分实践考验**的软件解决方案



- 名称
- 问题
- 施用和约束条件
- 解决方案
- 效果



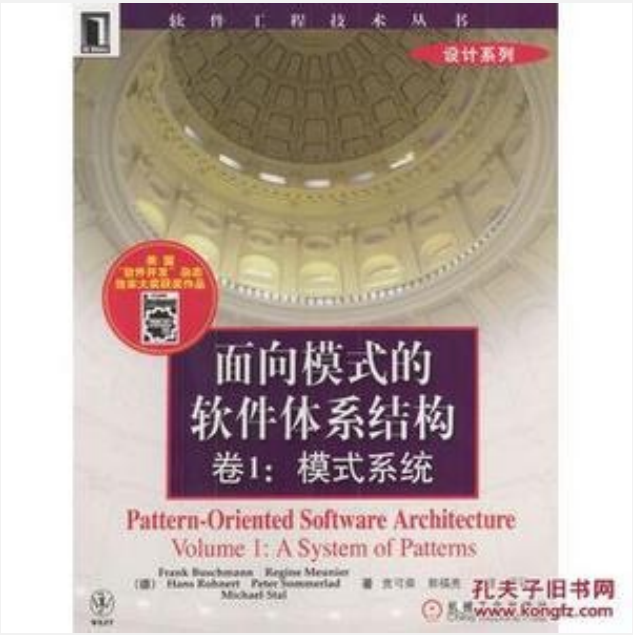


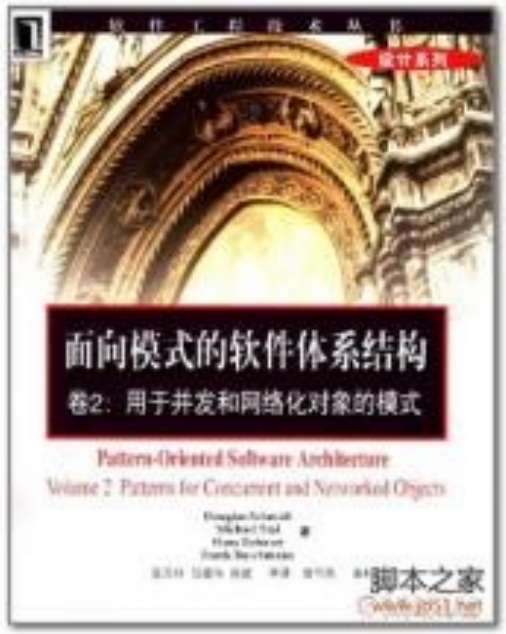
体系结构设计风格

- 面向整个软件系统，在抽象层次给出软件体系结构的结构化组织方式
- 提供一些预定义的子系统或者构件，规定其职责，明确它们之间的相互关系、协作方式的规则或指南
- 针对不同的问题采用不同的体系结构模式

常用软件体系结构风格

- 分层风格
- MVC风格
- 管道与过滤器风格
- SOA风格
- 总线风格
- ……





分层体系结构模式

❑模式思想

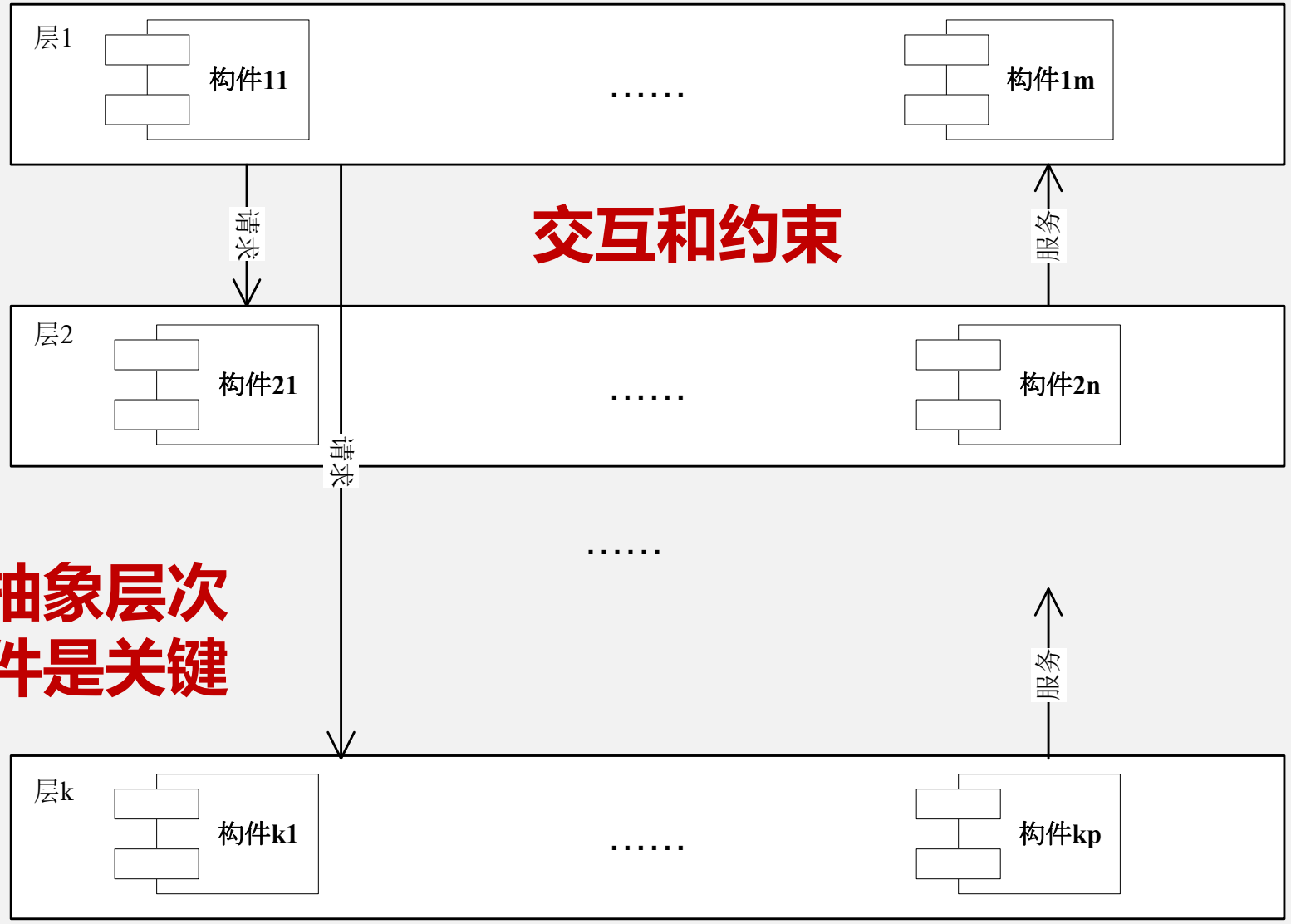
- ✓将软件系统按照**抽象级别**逐次递增或递减的顺序，组织为**若干层次**，每层由一些抽象级别相同构件组成

何为抽象级别?

❑典型层次示例

- ✓**顶层**：直接面向用户提供软件系统的交互界面
- ✓**底层**：则负责提供**基础性、公共性**的技术服务，它比较接近于硬件计算环境、操作系统或数据库管理系统
- ✓**中间层**：介乎二者之间，负责具体的**业务处理**

示例：分层体系结构风格



界面

业务

服务

合理地设计抽象层次和组织软构件是关键

分层体系结构模式的约束

❑层次间的关系

- ✓每层为其**紧邻上层**提供服务，使用**紧邻下层**所提供的服务
- ✓上层向下层发出**服务请求**，下层为上层反馈服务结果
- ✓下层向上层提供**事件信息**，上层对下层通知做出处理

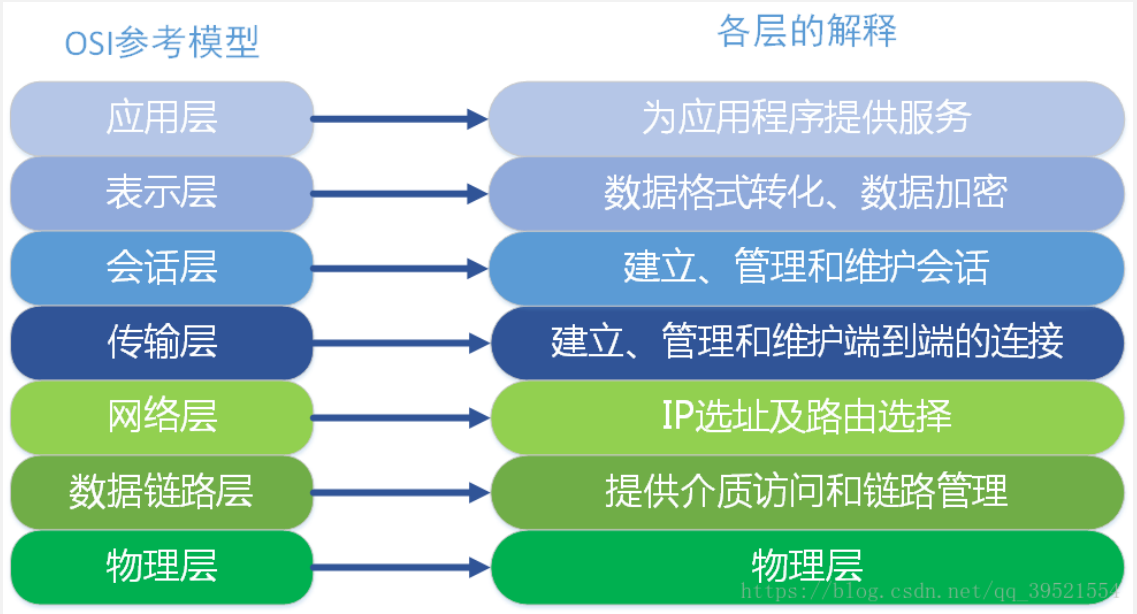
❑服务接口的组织方式

- ✓层次中的每个构件**分别**公开其服务接口
- ✓每个层次**统一**对外提供整合的服务接口

分层体系结构模式的特点

- **松耦合**
  - ✓减低整个软件系统的耦合度
- **可替换**
  - ✓一个层次可以有多个实现实例
- **可复用**
  - ✓层次和整个系统可重用
- **标准化**
  - ✓支持体系结构及其层次、接口的标准化

示例：网络OSI模型 和 云计算架构



计算机网络 OSI 七层结构



云计算经典三层架构

- 每一层实现各自的功能和协议，完成与相邻层的接口通信
- 通过接口提供给更高一层，每层所提供的服务与具体实现无关

三层结构设计

- 三层体系结构通常被划分为**表示层**、**中间层**和**数据层**三层，各个分层之间通过**对外接口**来互相访问。分层结构的主要**目的**，是为了**允许各层**可以**随着需求或技术的变化**而**独立的升级或替换**，例如当替换数据库时只需要变化数据层。
- 三层结构，就是在原来两层结构（Client/Server）的客户层与数据层之间，**加入了一个中间层（也叫业务层）**，并将应用程序的业务规则、数据访问、合法性校验等工作放到了中间层进行处理，这样就变成了三层结构。
- 三层是指逻辑上的三层，即使这三个层都放置到一台机器上。当然，三层也可以放在两台或三台机器上。

表示层

- **表示层通常采用图形化用户界面，在客户端的PC机(Desktop/Browser)或工作站上运行。**

- **站在“三个模型”建模思想上看，系统内部支持表示层的模型是“功能模型”，尽管“功能模型”中的功能实现组件都存放在业务层上，但是功能组件的表现方式却在表示层上。该层的主要功能是：**
  - ✓ 接受用户的请求，将这些请求反馈给业务逻辑层，等待业务逻辑层的回答信息
  - ✓ 对业务逻辑层的回答信息，进行显示；
  - ✓ 有时也会兼做业务逻辑层的一些小功能，比如对用户输入数据的验证，以及操作合法性的检验。 (注：RIA - Rich Internet Applications)

中间层（企业中台）

- 中间层也称业务层，它由许多构件或组件组成，这些构件或组件完全体现了用户的业务逻辑（Business Logic)或业务规则。

- 站在“三个模型”建模思想上看，系统内部支持业务层的模型是“业务模型”。尽管Java EE与.NET在实现业务层上的方法略有差异，但是，业务层本质上都是在表示层与数据层之间起桥梁作用。业务层的主要功能是：

  (1). 接受从表现层传来的用户请求信息

  (2). 根据用户的请求信息生成SQL语句(ORM)（注：RDBMS、不适用NoSQL)

  (3). 利用生成的SQL语句从数据层取数据、或修改数据、或删除数据；

  (4). 将结果返回给表现层。

数据层

- 数据层是数据库服务器上的数据库层，它包括**数据库管理系统DBMS**和**数据库DB**两部分。

- 站在“三个模型”建模思想上看，系统内部支持**数据层的模型是“数据模型”**

- 该层的主要功能是：

  (1) 接受业务层数据处理请求的SQL语句或存储过程；

  (2) 利用SQL语句或存储过程，对数据库服务器上数据库的相关表进行存储或检索；

  (3) 将存储或检索的结果信息，传递给业务层。

三层协调工作

- 三层之间，通过各自提供的接口来访问，比如用户想登录并操作系统，在表示层输入用户名和密码，**表示层会收集相关的数据传递给业务层**，业务层将数据经过一些处理和封装之后，再传递给数据层，数据层执行相应数据库中表的操作，并将结果返回业务层，业务层再返回表示层，表示层再显示给用户看。

- 对登录信息和操作信息，都是这么分层处理、协调工作的。业务层与数据层的信息交换采取批发方式，业务层与表示层的信息交换采取零售方式。

三层结构优点

(1). 三层之间的低耦合，所以互不干扰，哪一层出了问题就去找哪一层去解决；

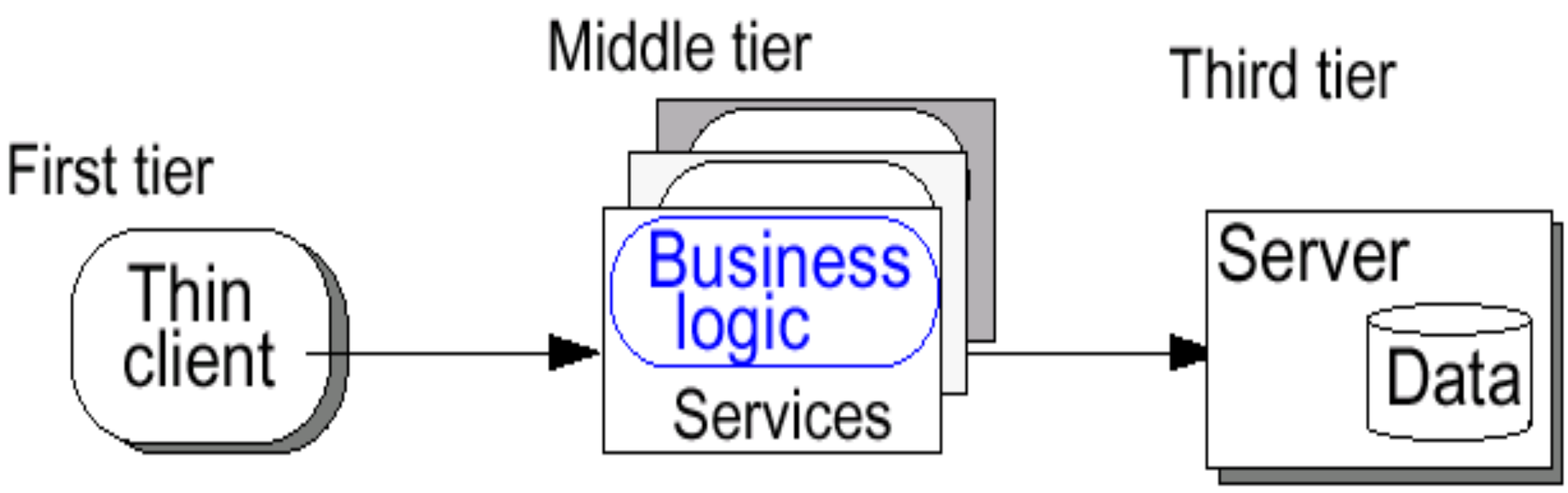
(2). 三层结构减少了客户机的工作量，提高了网络系统的运行效率；

(3).三层结构有利于系统的维护和升级，各个层的维护互不影响。例如，修改表示层，不会影响修改业务层；修改业务层，也不会影响修改数据层。而且，所有层的维护与修改，都是在服务器上进行，不需要到用户现场出差。

作业：MVC / MVP / MVVM 差别?

三层与多层的关系

若将中间层按照应用逻辑进一步划分，三层体系结构就变成了多层体系结构，如下图所示。



SpringBoot:

RESTFul (Controller Layer + Service Layer) + Repo Layer + Entity Layer

MVC风格

❑模型构件

✓负责存储业务数据并提供**业务逻辑处理功能**

❑视图构件

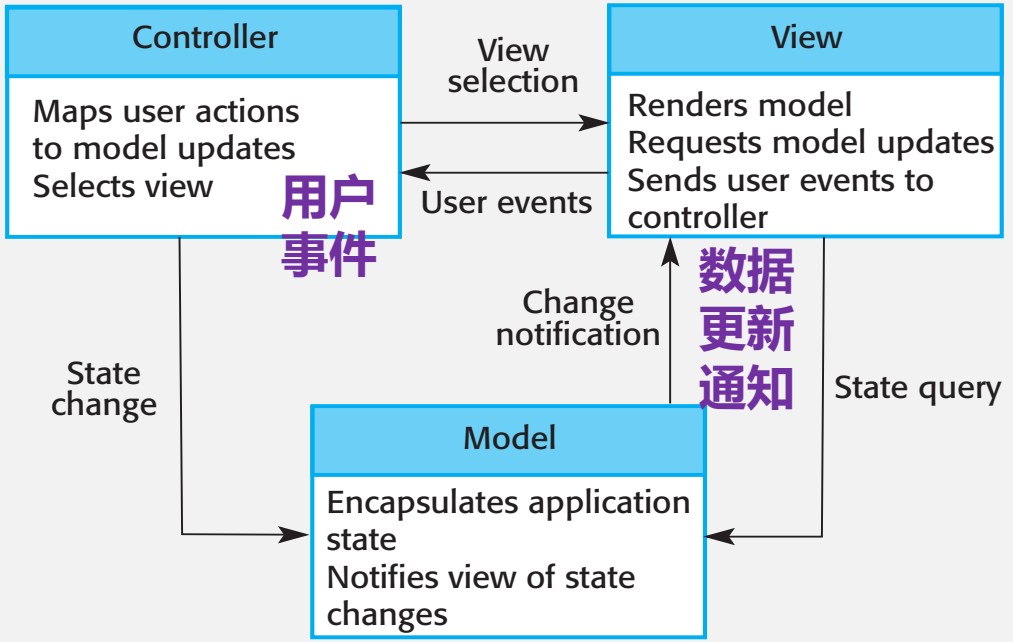
✓负责向用户**呈现**模型中的**数据**

❑控制器

✓在接获模型的业务逻辑处理结果后，负责**选择适当的视图**作为软件系统对用户的界面动作的响应

MVC风格的约束

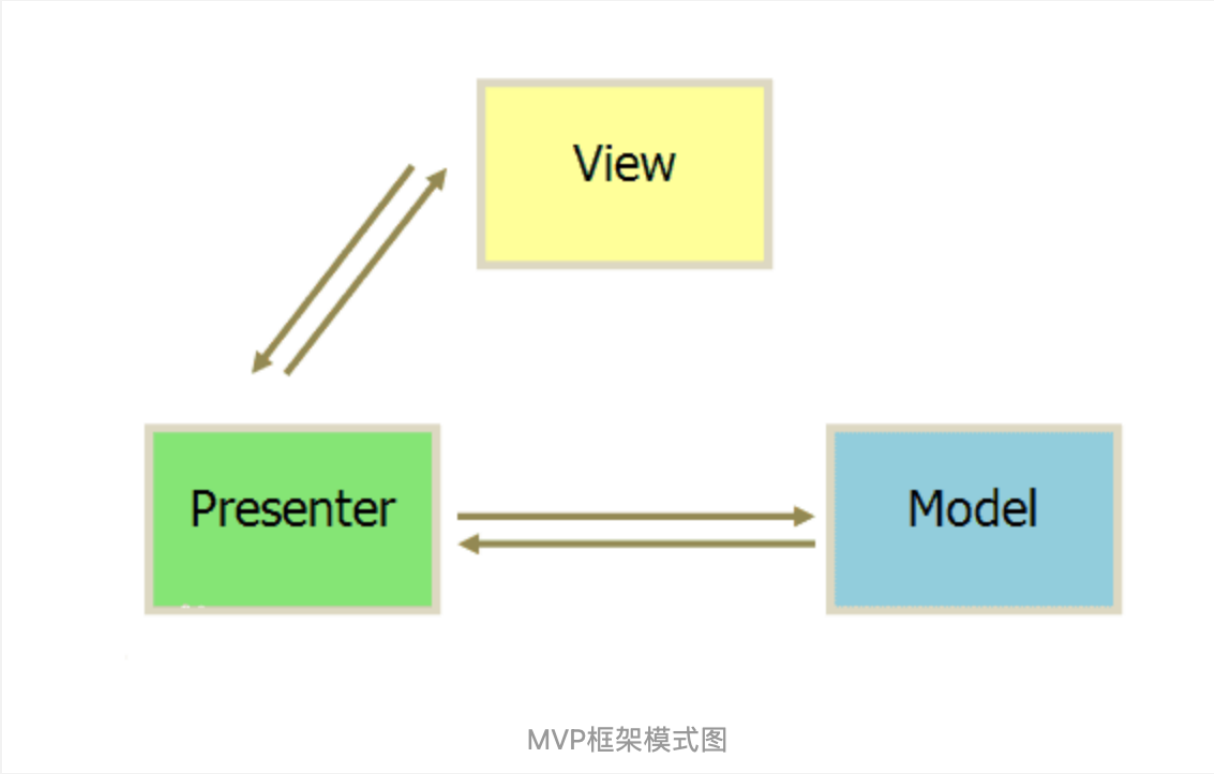
- **视图**
  - ✓ 视图接受**界面动作**，将其转换为**内部事件**传递给控制器
  - ✓ 从模型中**获取数据**并呈现回用户界面：所有视图在获得来自模型的业务数据变化通知后向模型**查询**新的数据，并据此**更新**视图
- **控制器**
  - ✓ 将来自**用户界面事件**转换为对**业务逻辑处理**功能的**调用**，将用户的请求**分发**到相应的模型
  - ✓ 控制器根据模型的处理结果： 1）**创建**新的视图、2）**选择**其他视图或 3）**维持**原有视图
- **模型**
  - ✓ 负责**存储业务数据**，提供**业务逻辑处理功能**，将处理结果回送给控制器
  - ✓ 必要时需将**业务数据变化事件**通知给所有视图



负责视图和模型之间的交互，控制对用户输入的**响应方式**和**处理逻辑**

MVP

- Model-View-Presenter ； MVP 是从经典的模式MVC演变而来，它们的基本思想有相通的地方Controller/Presenter负责逻辑的处理，Model提供数据，View负责显示。



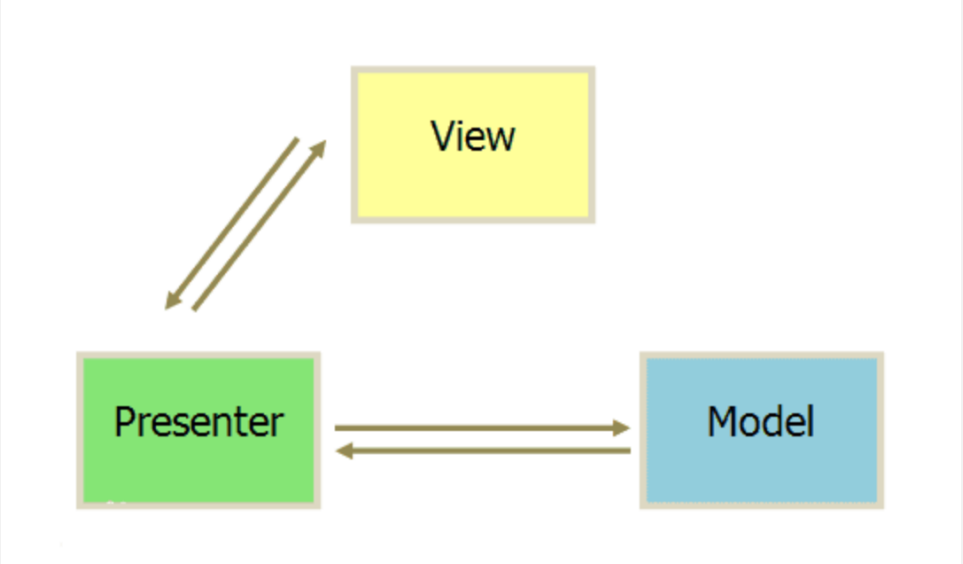
MVP框架模式图

MVP

- 优点
  - ✓1、模型与视图完全分离，我们可以修改视图而不影响模型
  - ✓2、可以更高效地使用模型，因为所有的交互都发生在一个地方——Presenter内部
  - ✓3、我们可以将一个Presenter用于多个视图，而不需要改变Presenter的逻辑。这个特性非常的有用，因为视图的变化总是比模型的变化频繁。
- 缺点
  - ✓由于对视图的渲染放在了Presenter中，所以视图和Presenter的交互会过于频繁。还有一点需要明白，如果Presenter过多地渲染了视图，往往会使得它与特定的视图的联系过于紧密。一旦视图需要变更，那么Presenter也需要变更了。

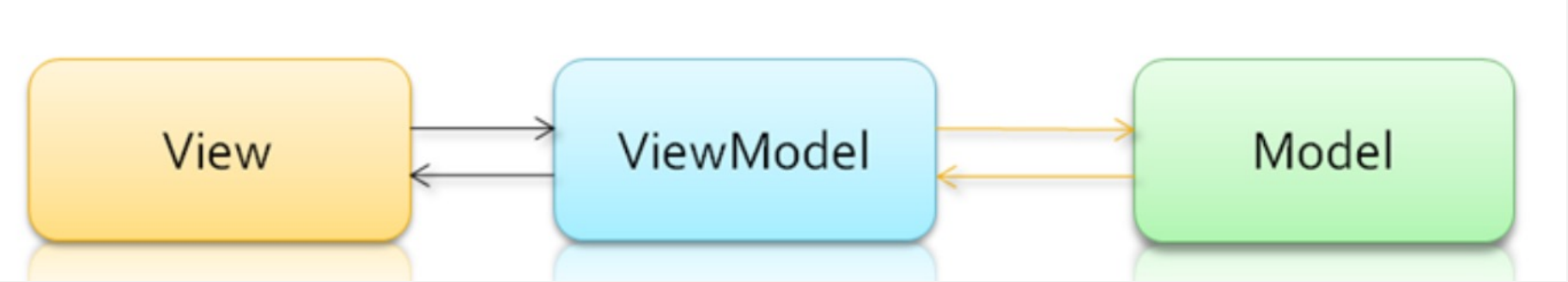
MVP与MVC区别

- 在MVP中View并不直接使用Model，它们之间的通信是通过Presenter (MVC中的Controller)来进行的，所有的交互都发生在Presenter内部，而在MVC中View会直接从Model中读取数据而不是通过 Controller。



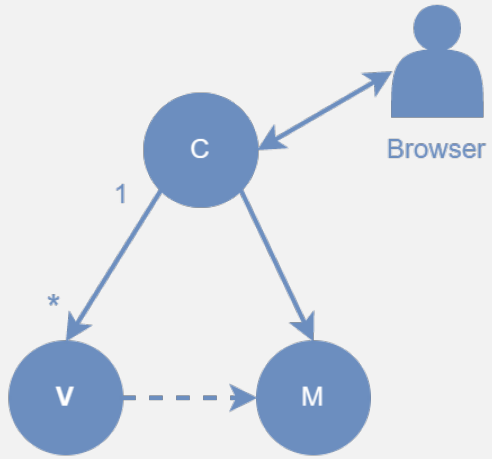
MVVM框架

- MVVM（Model-View-ViewModel）框架的由来便是MVP（Model-View-Presenter）模式与WPF结合的应用方式时发展演变过来的一种新型架构框架。MVVM 就是将其中的View 的状态和行为抽象化，让我们将视图 UI 和业务逻辑分开。

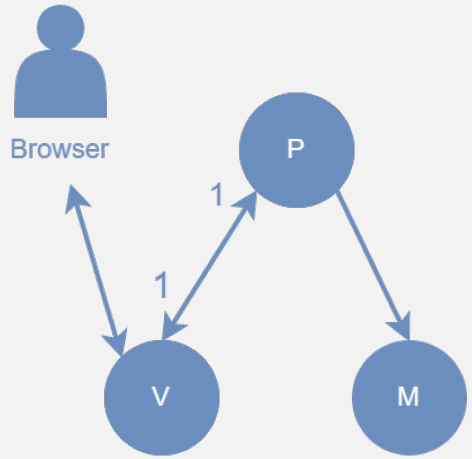


唯一的区别是，它采用双向绑定(data-binding):
View的变动，自动反映在View Model，反之亦然。这样开发者就不用处理接收事件和View更新的工作，框架已经帮你做好了。

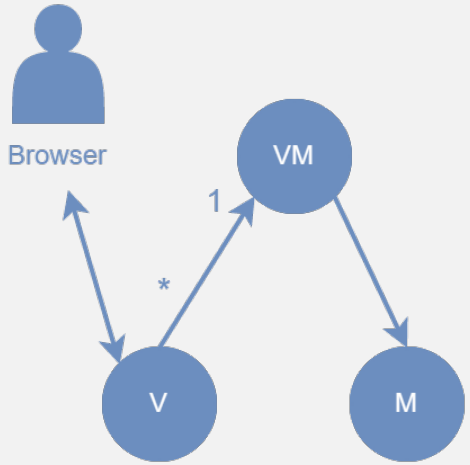
MVC-MVP-MVVM



- Controller是应用的入口
- Controller存在对于View的一对多关联
- View没有向Controller的引用
- View对Model了如指掌
- SmallTalk, ASP.Net MVC



- View是应用的入口
- View和Presenter有着一一映射
- View拥有Presenter的引用
- View对Model了如指掌

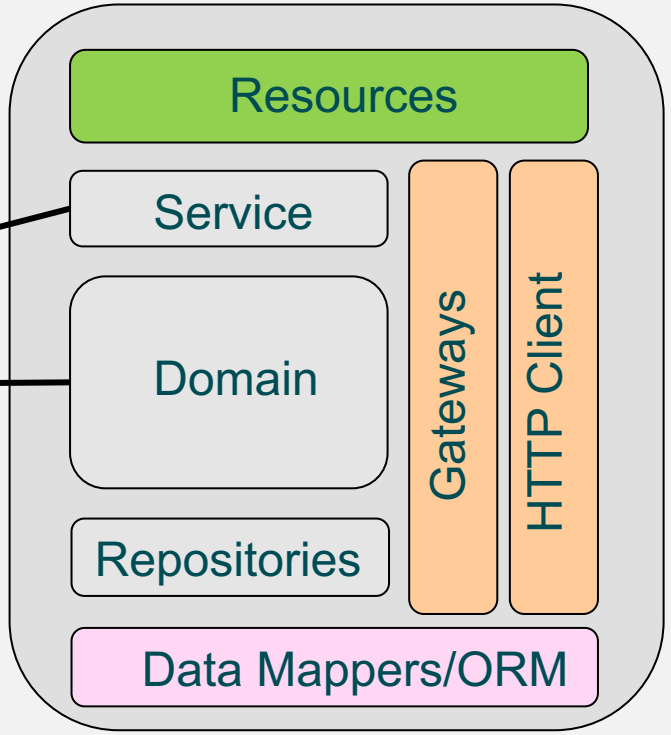


- View是应用的入口
- View存在对于ViewModel的一对多关联
- View具有ViewModel的引用
- View对Model了如指掌
- Silverlight, WPF, HTML5 with Knockout/AngularJS

贫血模型与充血模型

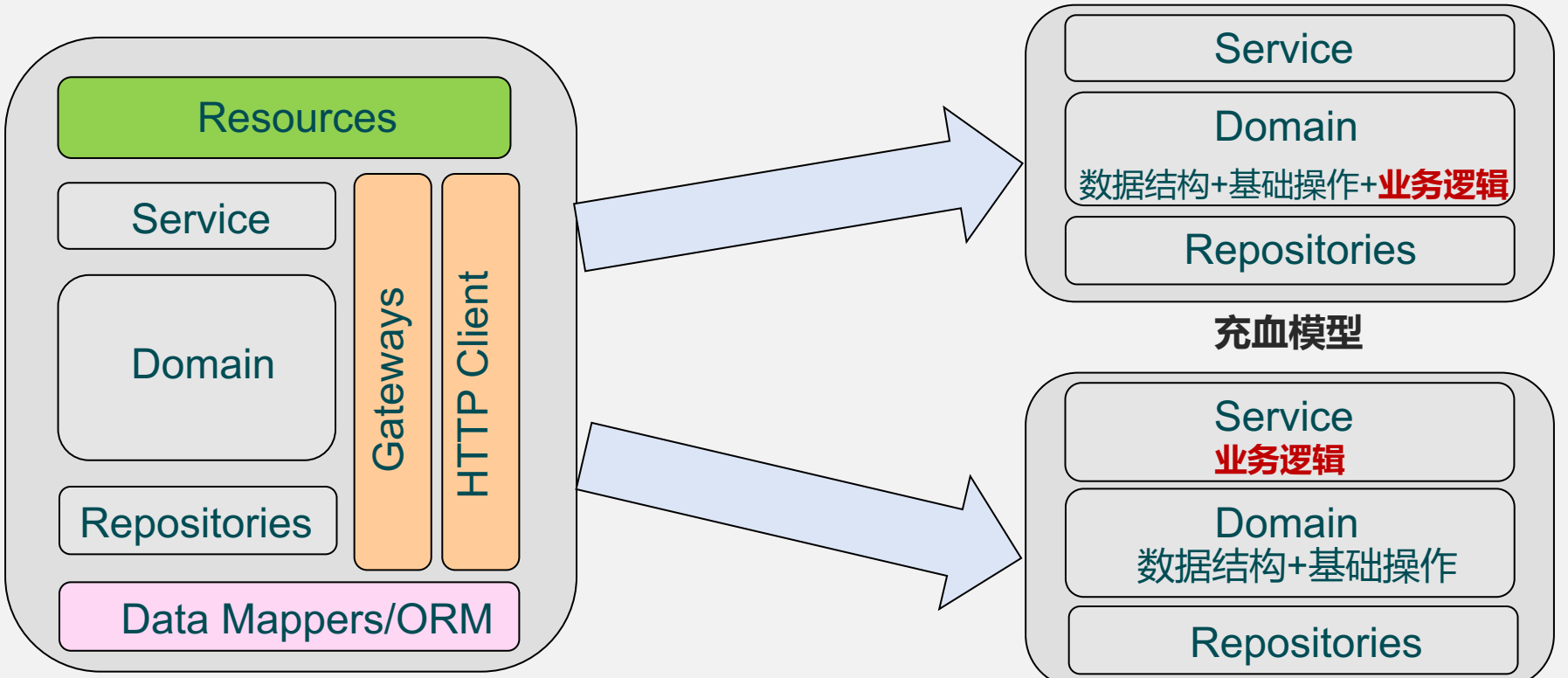
- 在介绍贫血模型与充血模型前，应了解领域模型的概念。
- 在Eric Evans的《领域驱动设计》一书中，提出领域模型的分层架构，分为用户界面层（表示层）、应用层、领域层、基础设施层。
- Martin Fowler则提出了一种领域驱动设计分层架构的实现。

用户界面层（表示层）	负责向用户显示信息和解释用户命令。
应用层	定义软件要完成的任务，并且指挥表达领域概念的对象来解决问题。
领域层（模型层）	负责表达业务概念，业务状态信息以及业务规则。
基础设施层	为上面各层提供通用的技术能力：为应用层传递消息，为领域层提供持久化机制，为用户界面绘制屏幕组件，等等。



Martin Fowler提出的DDD分层架构实现

贫血模型与充血模型



Martin Fowler提出的DDD分层架构实现

Domain中的实体Entity已经根据业务逻辑把基础的操作封装，能够完成独立功能

Domain中的实体Entity仅包含数据结构和基础操作（如属性的GET/SET方法）

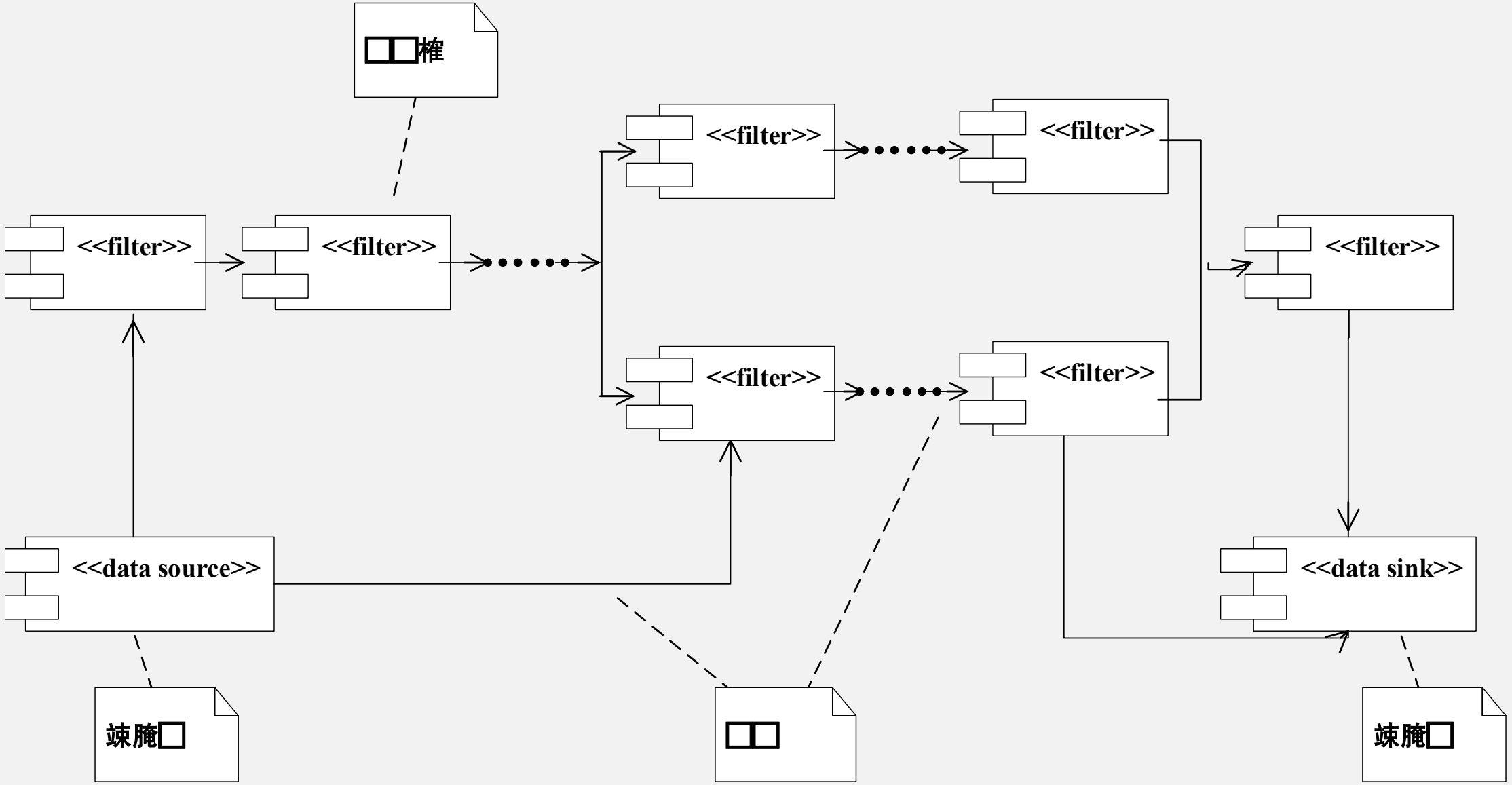
充血模型中，**领域层包含领域对象的数据结构 + 业务逻辑**。应用层（Service层）尽量简单，不包含业务规则或者知识，只为下一层中的领域对象协调任务，分配工作，使他们相互协作。

贫血模型中，**领域层仅包含对象的数据结构**，业务逻辑则放到应用层中。优点是易于开发者理解，能够与简单的数据访问层较好协作。

管道与过滤器风格

- **构件**
  - ✓将软件功能实现为一系列处理步骤，每个步骤封装在一个**过滤器构件**中
- **连接子**
  - ✓相邻过滤器间以**管道**连接，一个过滤器的输出数据借助管道流向后续过滤器，作为其输入数据
- **数据**
  - ✓软件系统的输入由**数据源**（data source）提供
  - ✓软件最终输出由源自某个过滤器的管道流向**数据汇**（data sink）
  - ✓典型数据源和数据汇包括数据库、文件、其他软件系统、物理设备等

管道与过滤器模式的示例



管道与过滤器风格的约束

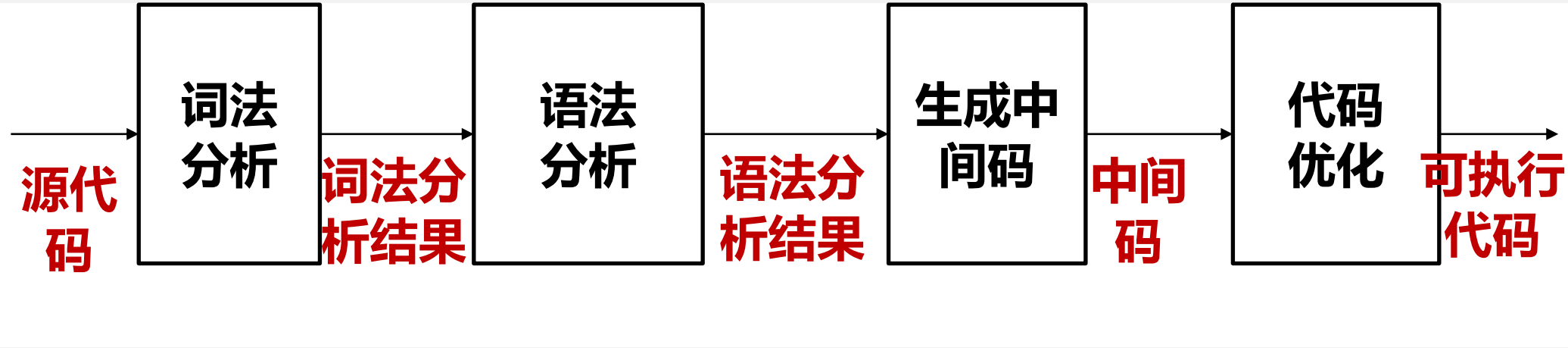
❑过滤器与管道之间的协作方式

- ✓过滤器以循环的方式不断地从管道提取输入数据，并将其输出数据压入管道，称为**主动过滤器**
- ✓管道将输入数据压入到位于其目标端过滤器，过滤器被动地等待数据，称为**被动过滤器**
- ✓管道负责提取位于其源端过滤器的输出数据

❑设计考虑

- ✓如果管道连接的两端均为主动过滤器，那么管道必须负责它们之间的同步，典型的同步方法是**先进先出缓冲器**
- ✓如果管道的一端为主动过滤器，另一端为被动过滤器，那么管道的数据流转功能可通过前者直接调用后者来实现

示例：管道与过滤器风格



编译器采用的就是一个典型的管道/过滤器风格

管道与过滤器模式的特点

- 自然地解决具有数据流特征的软件需求
- 可独立地更新、升级过滤器来实现软件系统的扩展和进化

面向服务的体系结构 (SOA)

Service Oriented Architecture - SOA

… a service?

一个**可重用的独立功能组件**，如客户信用检查、开通新帐户

… service orientation?

将**业务集成**起来形成互联服务的方法，以及形成的**系统形态**



… service oriented architecture (SOA)?

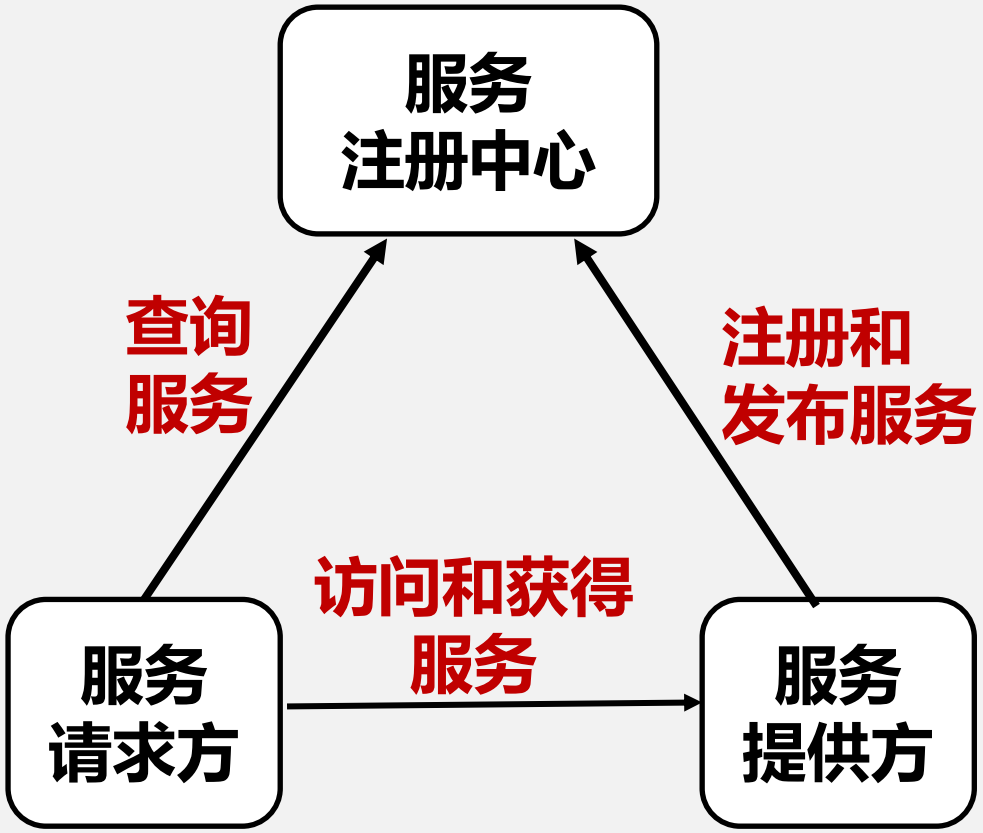
一种支持“面向服务”的IT体系结构**风格**

… a composite service?

一组**集成**的服务，它们可支持SOA中的一个复杂**业务过程**

面向服务的体系结构 (SOA)

- 将软件系统的软构件抽象为一个个的**服务**（Service），每个服务封装了特定的功能并提供了对外可访问的**接口**
- 任何一个服务既可以充当服务的**提供方**，接受其他服务的访问请求；也可充当服务的**请求方**，请求其他服务为其提供功能
- 任何服务需要向服务注册中心进行**注册登记**，描述其可提供的服务以及访问方式，才可对外提供服务

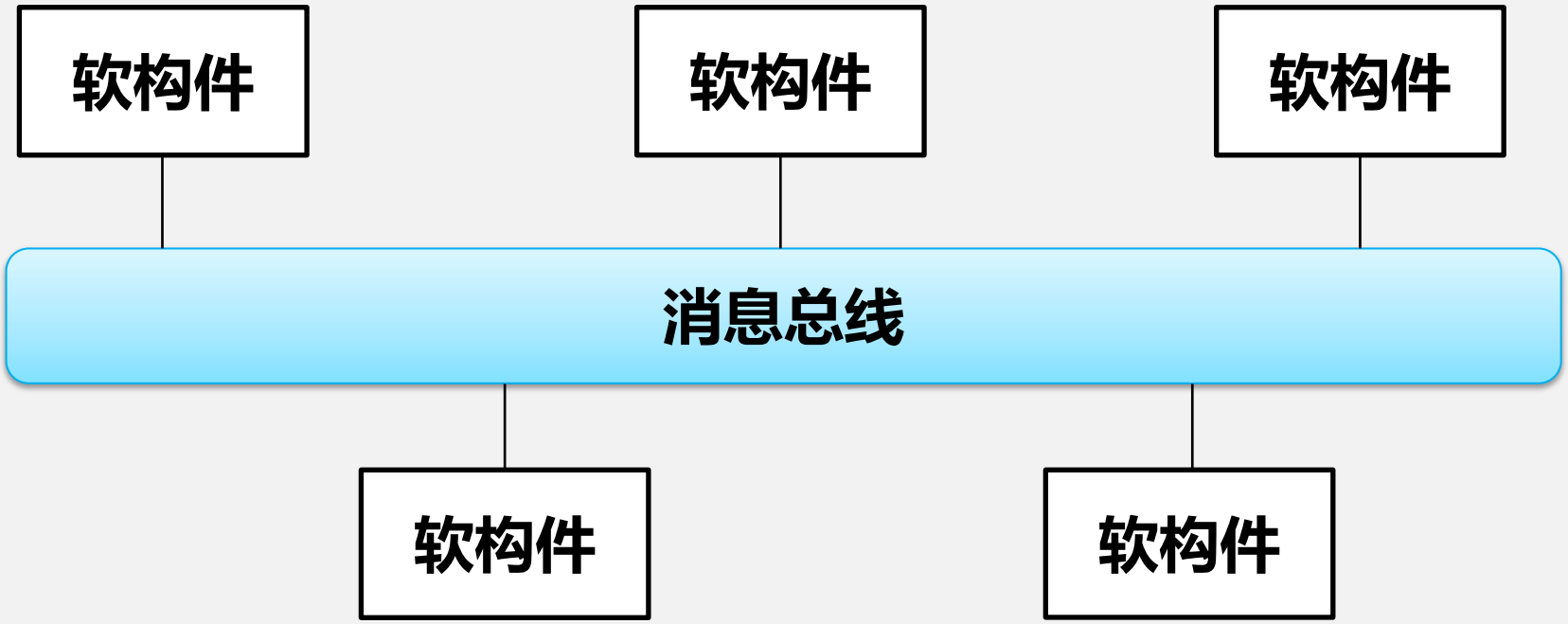


SOA风格的特点

- 将服务提供方和服务请求方独立开来，因而支持服务间的**松耦合定义**
- 允许任何一个服务在运行过程中所**扮演角色**的动态调整，支持**服务集合**在运行过程中的**动态变化**，因而具有非常强的灵活性
- 提供了诸如UDDI、SOAP、WSDL等协议来支持服务的注册、描述和绑定等，因而可有效支持**异构服务间的交互**

消息总线风格

- **包含了一组软构件和一条 “消息总线”的连接件来连接各个软构件**
  - ✓消息总线成为软构件之间的通信桥梁，实现各个软构件之间的消息发送、接收、转发、处理等功能
  - ✓每一个软构件通过接入总线，实现消息的发送和接收功能

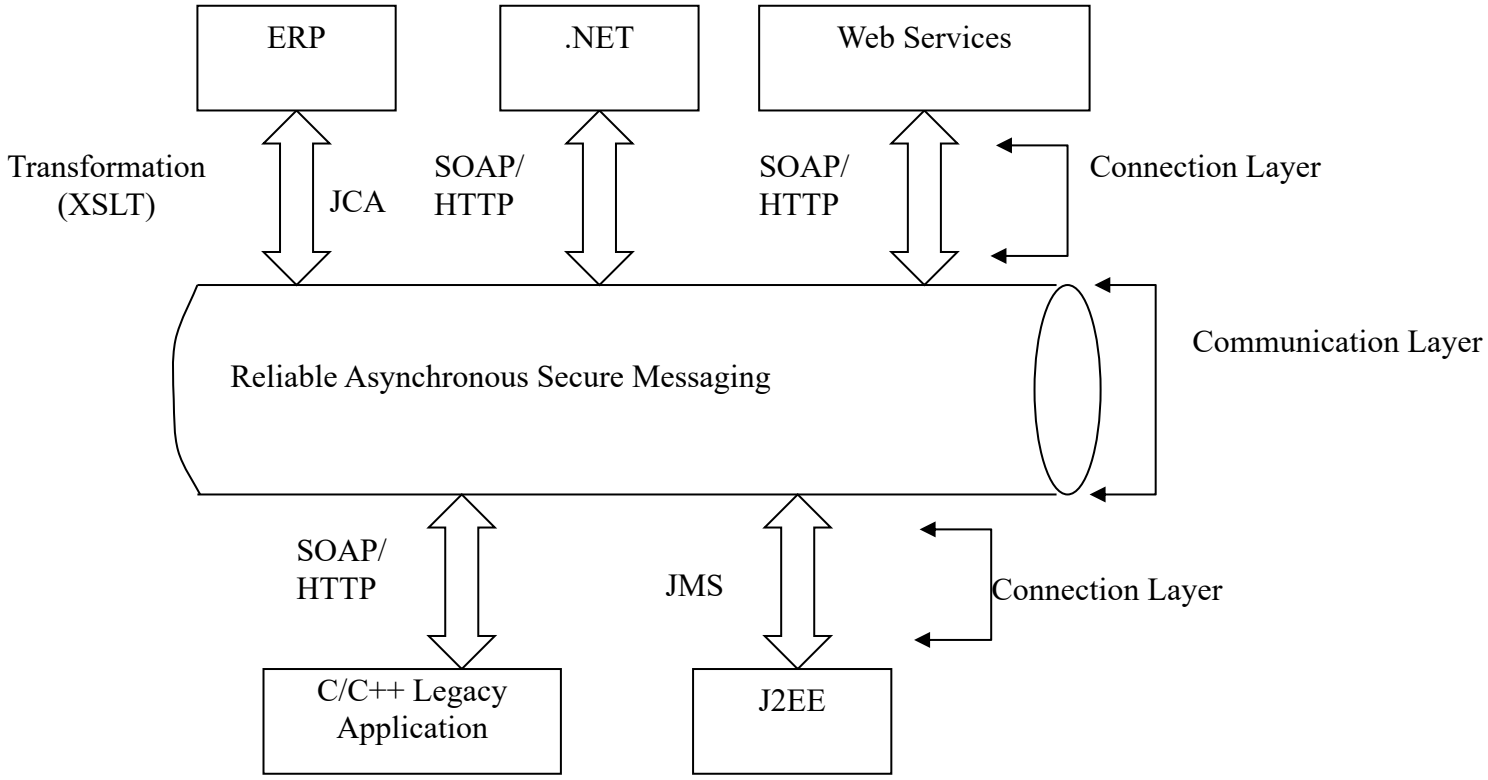


SOA 服务总线

- **ESB （Enterprise Service Bus):** 一种构建、部署企业SOA的新方法，简化Web服务部署，解决集成问题
- 基于标准的高速链路(Backbone)，利用MOM (面向消息中间件, Message Oriented Middleware) 连接异构系统, 综合不同类型中间件的特征
- **主要特征**
  - ✓ **Web服务**：SOAP, WSDL, UDDI
  - ✓ **消息**：异步store-and-forward方式的分发
  - ✓ **路由**：发布/订阅，基于内容，路线itinerary
  - ✓ **平台中立**： 连接企业中的任何资源：Java, .NET, 遗留应用, WS-*，…

ESB体系结构

支持了不同消息协议转化、不同数据格式的转换



主观题 10分

设置

基于上述软件体系结构模式，思考你的实践项目应该采用什么样的软件体系结构模式？为什么？



Answer

北京航空航天大學
BEIHANG UNIVERSITY

Thank you

北京航空航天大学